

流匹配与扩散模型导论

An Introduction to Flow Matching and Diffusion Models

Peter Holderrieth 和 Ezra Erives

https://diffusion.csail.mit.edu/

本地交付候选：全文独立审校修订版

MIT 6.S184 课程讲义 · 2026
Model: 初译模型未核实；本轮审校 gpt-6.1-sol（high）
审核：曹溪能
Updated: 2026-11-04

译本说明

本译本依据 Peter Holderrieth 与 Ezra Erives 的 MIT 6.S184（2026）课程讲义 *An Introduction to Flow Matching and Diffusion Models*。权威输入为本地冻结的 84 页 PDF，原课程来源为 `https://diffusion.csail.mit.edu/2026/`；未取得作者 TeX。本轮进行了全文中文语言、数学记号与结构、算法和图文对照及版面审校，初译模型未能核实。源文的符号、假设或表述疑点按原文保留，以译注和随附问题记录区分，未把翻译审校视为对全部源文结论的独立证明。图像保留原图并补中文说明。课程网站页脚的许可对 PDF 及第三方图像的具体适用范围尚未核实，本包只作本地交付候选；主控最终验收和公开发布另行处理。封面审核字段遵循项目格式，不表示人类已完成审核。

目录

1 引言 **3**
1.1 概览 . . . 3
1.2 课程结构 . . . 3
1.3 生成建模作为采样 . . . 4

2 流模型与扩散模型 **5**
2.1 流模型 . . . 6
2.2 扩散模型 . . . 8

3 流匹配 **12**
3.1 条件概率路径与边缘概率路径 . . . 12
3.2 条件向量场与边缘向量场 . . . 14
3.3 学习边缘向量场 . . . 17

4 得分函数与得分匹配 **21**
4.1 条件得分函数与边缘得分函数 . . . 21
4.2 使用 SDE 采样 . . . 23
4.3 得分匹配 . . . 26

5 引导：如何以提示词为条件 **29**
5.1 基本引导 . . . 29
5.2 无分类器引导 . . . 30

6 构建大型图像或视频生成器 **33**
6.1 神经网络架构 . . . 34
6.1.1 条件变量的嵌入 . . . 34
6.1.2 扩散 Transformer . . . 35
6.1.3 U-Net . . . 37
6.2 在潜空间中工作：（变分）自编码器 . . . 38

6.2.1 标准自编码器 39
6.2.2 变分自编码器 39
6.3 案例研究：Stable Diffusion 3 与 Meta Movie Gen 43
6.3.1 Stable Diffusion 3 43
6.3.2 Meta Movie Gen Video 44

7 离散扩散模型：用扩散构建语言模型 45
7.1 连续时间 Markov 链（CTMC）模型 45
7.2 训练 CTMC 模型 49
7.2.1 条件与边缘概率路径 50
7.2.2 条件速率矩阵与边缘速率矩阵 50
7.2.3 学习边缘速率矩阵 53

8 参考文献 55

A 概率论回顾 59
A.1 随机向量 59
A.2 条件密度与条件期望 59

B Fokker–Planck 方程的证明 61

C 连续时间 Markov 链的存在性与唯一性 63

D 关于 VAE 的其他视角 64

E 扩散模型文献指南 68

1 引言

从数据创造噪声很容易；从噪声创造数据就是生成建模。
Song 等人 [43]

1.1 概览

近年来，我们都见证了人工智能（AI）的一场巨大革命。Nano Banana、Stable Diffusion 3 等图像生成器可以生成各种风格的写实或艺术图像；Meta 的 VEO-3 等视频模型可以生成高度逼真的电影片段；ChatGPT 等大语言模型可以对文本提示生成看似达到人类水平的回复。这场革命的核心是人工智能系统的一种新能力：生成对象。过去的 AI 系统主要用于预测，而这些新的 AI 系统则具有创造性：它们根据用户指定的输入构想或产生新对象。这类生成式 AI 系统是近来 AI 革命的核心。

本课程的目标是教授两种使用最广泛的生成式 AI 算法：去噪扩散模型 [45] 和流匹配 [25, 27, 1, 26]。这些模型是最优秀的图像、音频和视频生成模型（例如 Nano Banana、FLUX 或 VEO-3）的基础，并且最近也在蛋白质结构等科学应用中达到当前领先水平（例如 AlphaFold3 就是一个扩散模型）。毫无疑问，理解这些模型是一项非常有用的技能。

所有这些生成模型都通过迭代地将噪声转化为数据来生成对象。这种从噪声到数据的演化通过模拟常微分方程或随机微分方程（ODE/SDE）实现。流匹配和去噪扩散模型是一类技术，使我们能够使用深度神经网络大规模构建、训练并模拟此类 ODE/SDE。虽然这些模型实现起来相当简单，但 SDE 的技术性质可能使它们难以理解。本课程自包含地介绍必要的微分方程数学工具，使读者能够系统地理解这些模型。然后我们逐步解释当前先进图像与视频生成器的现代技术栈。除了应用广泛之外，我们认为流模型与扩散模型背后的理论本身也很优美。因此，最重要的是，我们希望这门课能给你带来许多乐趣。

注 1 (其他资源)**.** 虽然本讲义是自包含的，我们仍鼓励你使用另外两种资源：

1. 课程录像：以讲课的形式引导你学习每一节。
2. 实验：指导你从零实现自己的扩散模型。我们强烈建议你“亲自动手”编程。

这些资源可在课程网站找到：*https://diffusion.csail.mit.edu/*。

1.2 课程结构

下面简要概述本文档。

- 第一节，作为采样的生成建模：我们将形式化“生成”图像、视频、蛋白质等对象的含义。例如，我们会将“如何生成一张狗的图像?”转化为从概率分布采样这一更精确的问题。
- 第二节，流模型与扩散模型：我们解释生成的机制。正如课程名称所暗示的，这一机制包括模拟常微分方程和随机微分方程。我们介绍微分方程并说明如何用它们构造生成模型。
- 第三节，流匹配：接着，我们解释并推导流匹配，这是一种简单、可扩展的算法，位于前述 Stable Diffusion、Nano Banana 或 SORA 等大型生成模型的核心。

- 第四节，得分匹配：我们研究得分函数，以及如何通过得分匹配学习它们。这不仅是扩散模型的训练算法，也使 SDE 采样和引导成为可能。
- 第五节，引导：我们学习如何以提示词（例如 “一张猫的图像”）为条件生成样本，以及如何通过无分类器引导使生成结果遵循该提示。
- 第六节，潜空间与神经网络架构：我们讨论如何构建 Nano Banana 等大型图像和视频生成器，包括常用神经网络架构以及如何在潜空间中构建模型，还会综述当前先进模型。
- 第七节（选学），离散扩散模型：我们学习如何将扩散模型原理从欧氏空间推广到语言等离散数据，从而使用扩散模型原理构造大语言模型。

所需背景。 由于主题具有技术性，我们建议读者具备一定的数学基础，尤其是对概率论有所了解。因此我们在附录 A 中加入了概率论简要回顾。如果其中有些概念不熟悉，也不必担心。

1.3 生成建模作为采样

首先考虑我们可能遇到的各种数据类型或数据模态，以及如何用数值表示它们：

1. **图像**。考虑具有 $H \times W$ 个像素的图像，其中 H 表示图像高度、W 表示宽度，每个像素有三个颜色通道（RGB）。每个像素的每个颜色通道都给出一个 $\mathbb{R}$ 中的强度值。因此图像可以表示为一个元素 $z \in \mathbb{R}^{H\times W\times 3}$。
2. **视频**。视频不过是随时间排列的一系列图像。如果有 T 个时间点或帧，视频便可表示为一个元素 $z \in \mathbb{R}^{T\times H\times W\times 3}$。
3. **分子结构**。一种朴素方法是用矩阵 $z = (z^1, \dots, z^N) \in \mathbb{R}^{3\times N}$ 表示分子结构，其中 N 为分子中的原子数，每个 $z^i \in \mathbb{R}^3$ 描述该原子的位置。当然，还有其他更复杂的分子表示方法。

在上述所有例子中，我们想生成的对象都可以在数学上表示为向量（必要时先展平）。因此，本文中始终有：

核心思想 1 (对象作为向量)**.** *我们将待生成对象表示为向量 $z \in \mathbb{R}^d$。*

上述讨论的一个显著例外是文本数据：语言模型（如 ChatGPT）通常将其建模为离散对象。虽然连续数据 $z \in \mathbb{R}^d$ 是我们的主要重点，但第七节也会研究文本生成。

生成作为采样。 让我们定义 “生成” 某个对象是什么意思。例如，假设我们想生成一张狗的图像。自然地，有许多可能的狗图像都能让我们满意，并不存在唯一一张 “最好” 的狗图像；相反，有一系列图像与要求契合的程度各不相同。在机器学习中，通常用图像空间上的概率分布来体现这种可能图像的多样性。我们称这样的分布为*数据分布*，记为 p_{data}。在数学上，可以将 p_{data} 看作概率密度，即函数 $p_{\text{data}} : \mathbb{R}^d \to \mathbb{R}_{\geq 0}$，它给每个可能对象 $z \in \mathbb{R}^d$ 赋予一个似然 $p_{\text{data}}(z) \geq 0$。在狗图像的例子中，这个分布会给看起来更像狗的图像 z 赋予更高的似然 $p_{\text{data}}(z)$。因此，一张图像、一段视频或一个分子 “符合得多好” ——这是一种相当主观的说法——被它在数据分布 p_{data} 下 “有多大可能性” 所取代。据此，我们可以将生成任务在数学上表述为从未知分布 p_{data} 中采样：

核心思想 2 (生成作为采样)**.** 生成对象 z 被建模为从数据分布采样：$z \sim p_{\text{data}}$。

生成模型是允许我们从 p_{data} 生成样本的机器学习模型。在机器学习中，我们需要数据来训练模型。在生成建模中，通常假设能够获取从 p_{data} 独立采样得到的有限多个例子，它们共同作为真实分布的近似替代。

核心思想 3 (数据集)**.** 数据集由有限多个样本 $z_1, \ldots, z_N \sim p_{\text{data}}$ 构成。

对于图像，我们可以通过收集互联网上公开可用的图像来构造数据集。对于视频，类似地可以使用 YouTube。对于蛋白质结构，RCSB 蛋白质数据库（PDB）等来源提供了数十万个经实验解析的结构。随着数据集变得非常大，它会越来越好地代表底层分布。

引导生成 / 条件生成。 在许多情形下，我们希望以某些数据 y 为条件生成对象。例如，可能希望以 $y =$ “一只狗沿着覆盖积雪的山坡奔跑，背景是群山” 这一条件生成图像。我们可以将其重新表述为从条件分布采样：

核心思想 4 (引导生成)**.** 引导生成涉及从 $z \sim p_{\text{data}}(\cdot \mid y)$ 采样，其中 y 是条件变量。

我们称 $p_{\text{data}}(\cdot \mid y)$ 为引导数据分布。引导生成建模任务通常需要学习对任意而非固定的 y 进行条件化。沿用上例，我们也可能想以另一条文本提示为条件，例如 $y =$ “一张写实图像：一只猫正在吹灭生日蛋糕上的蜡烛”。因此我们寻求一个可对任意此类 y 作条件化的单一模型。事实证明，无条件生成的技术可以直接推广到条件情形。因此，前三节我们将几乎完全聚焦无条件情形（同时记住条件生成是最终目标）。

生成模型。 抽象地说，生成模型是返回从 $z \sim p_{\text{data}}$ 所得样本的算法（或者至少近似如此）。如果 p_{data} 是狗图像的分布，这个算法就会返回随机的狗图像。本课程关注利用流模型或扩散模型构造生成模型，因为它们代表目前的先进水平。不过，务必记住人们还开发了许多其他生成模型（未来或许还会发现更多）。

总结 2 (生成作为采样)**.** 本节内容总结如下：

1. 本文主要考虑生成可表示为向量 $z \in \mathbb{R}^d$ 的对象，例如图像、视频和分子结构。
2. 生成任务是在训练期间能够获得样本数据集 $z_1, \ldots, z_N \sim p_{\text{data}}$ 的条件下，从概率分布 p_{data} 中生成样本。
3. 引导生成假设我们以标签 y 为条件，并希望在训练期间获得由样本对 $(z_1, y), \ldots, (z_N, y)$ 组成的数据集时，从 $p_{\text{data}}(\cdot \mid y)$ 中采样。
4. 我们的目标是构造一个生成模型，即训练后返回 p_{data} 样本的模型。

2 流模型与扩散模型

在上一节中，我们将生成建模形式化为从数据分布 p_{data} 采样，并形式化了目标：构建一个生成模型，即返回样本 $z \sim p_{\text{data}}$ 的算法。本节说明如何通过模拟适当构造的微分方程构建生成

模型。例如，流匹配和扩散模型分别涉及模拟常微分方程（ODE）和随机微分方程（SDE）。因此，本节的目标是按照讲义其余部分将要使用的方式定义并构造这些生成模型。具体地，我们先定义 ODE 和 SDE 并讨论如何模拟；其次，说明如何用深度神经网络参数化 ODE/SDE。由此得到流模型、扩散模型的定义和从这些模型采样的基本算法。后续章节再探讨如何训练这些模型。

2.1 流模型

我们先定义常微分方程（ODE）。ODE 的解由一条轨迹定义，也就是如下形式的函数：

$$X : [0,1] \to \mathbb{R}^d, \qquad\qquad t \mapsto X_t,$$

它将时间 t 映射为空间 $\mathbb{R}^d$ 中的某个位置。每个 ODE 由向量场 u 定义，即如下形式的函数：

$$u : \mathbb{R}^d \times [0,1] \to \mathbb{R}^d, \qquad\qquad (x,t) \mapsto u_t(x),$$

也就是说，对每个时间 t 和位置 x，我们得到指定空间速度的向量 $u_t(x) \in \mathbb{R}^d$（见图 1）。ODE 对轨迹施加一个条件：我们希望轨迹 X 从点 x_0 开始，“沿着”向量场 u_t 的线运动。可以将这条轨迹形式化为以下方程的解：

$$\frac{d}{dt} X_t = u_t(X_t) \qquad \text{ODE}, \tag{1a}$$

$$X_0 = x_0 \qquad \text{初始条件}. \tag{1b}$$

式 (1a) 要求 X_t 的导数由 u_t 给出的方向指定；式 (1b) 要求在 $t = 0$ 时从 x_0 出发。现在可以问：若在 $t = 0$ 从 $X_0 = x_0$ 出发，时间 t 时位于何处（即 X_t 是什么）？这个问题由称为流的函数回答，它是 ODE 的解：

$$\psi : \mathbb{R}^d \times [0,1] \to \mathbb{R}^d, \qquad (x_0, t) \mapsto \psi_t(x_0), \tag{2a}$$

$$\frac{d}{dt} \psi_t(x_0) = u_t(\psi_t(x_0)) \qquad \text{流的 ODE}, \tag{2b}$$

$$\psi_0(x_0) = x_0 \qquad \text{流的初始条件}. \tag{2c}$$

给定初始条件 $X_0 = x_0$，通过 $X_t = \psi_t(X_0)$ 可恢复 ODE 轨迹。因此直观而言，向量场、ODE 和流是同一对象的三种描述：向量场定义 ODE，其解是流。与所有方程一样，我们应当询问 ODE 的解是否存在，以及若存在是否唯一。数学中的一个基本结果对此均回答“是”，只需对 u_t 施加较弱的假设：

定理 3 (流的存在性与唯一性)**.** *如果 $u : \mathbb{R}^d \times [0,1] \to \mathbb{R}^d$ 连续可微且导数有界，则式 (2) 中的 ODE 有唯一的解，由流 ψ_t 给出。此时对所有 t，ψ_t 都是微分同胚，即 ψ_t 连续可微，且其逆 ψ_t^{-1} 也连续可微。*

注意，机器学习中流的存在唯一性所需的假设几乎总是满足，因为我们用神经网络参数化 $u_t(x)$，而神经网络的导数总是有界。因此定理 3 不应令你担忧，而应是好消息：在我们关心的情形下，流存在并且是 ODE 的唯一解。证明参见文献 [32, 9]。

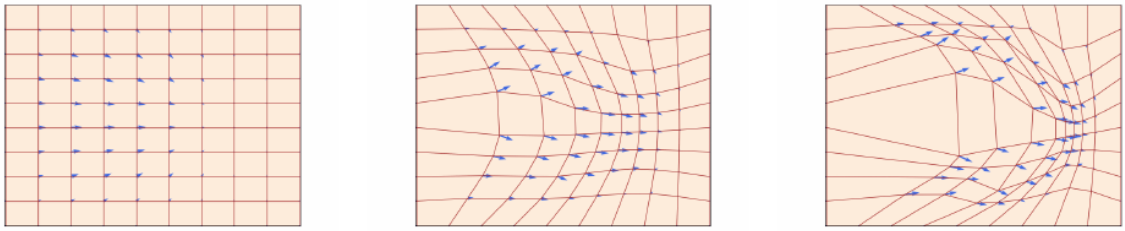


图 1: 流 $\psi_t : \mathbb{R}^d \to \mathbb{R}^d$（红色方格网）由速度场 $u_t : \mathbb{R}^d \to \mathbb{R}^d$ 定义（蓝色箭头），后者规定各处的瞬时运动；此处 $d = 2$。图中展示三个不同时间 t。可见，流是“扭曲”空间的微分同胚。图片来自文献 [26]。

例 4 (线性向量场)**.** *考虑一个简单例子：向量场 $u_t(x)$ 是关于 x 的简单线性函数，即 $u_t(x) = -\theta x$，其中 $\theta > 0$。那么函数*

$$\psi_t(x_0) = \exp(-\theta t)x_0 \tag{3}$$

定义了求解式 (2) 中 ODE 的流 ψ。可以自行验证 $\psi_0(x_0) = x_0$，并计算

$$\frac{d}{dt}\psi_t(x_0) \overset{(3)}{=} \frac{d}{dt}(\exp(-\theta t)x_0) \overset{(i)}{=} -\theta \exp(-\theta t)x_0 \overset{(3)}{=} -\theta\psi_t(x_0) = u_t(\psi_t(x_0)),$$

其中 (i) 使用了链式法则。图 3 将这种指数收敛到零的流可视化。

模拟 ODE。 一般而言，如果 u_t 不像上例那样简单，就无法显式计算流 ψ_t。这时使用数值方法模拟 ODE。幸运的是，这是数值分析中一个经典而研究充分的主题，存在大量强大的方法 [21]。其中最简单直观的方法之一是 Euler 方法。Euler 方法以 $X_0 = x_0$ 初始化，并按下式更新：

$$X_{t+h} = X_t + hu_t(X_t) \qquad (t = 0, h, 2h, 3h, \ldots, 1 - h), \tag{4}$$

其中 $h = n^{-1} > 0$ 是步长，$n \in \mathbb{N}$ 是模拟步数。对于本课程，Euler 方法已经足够好。为让读者了解更复杂的方法，考虑由以下更新规则定义的 Heun 方法：

$$\begin{aligned} X'_{t+h} &= X_t + hu_t(X_t) && \text{对新状态的初始猜测（与 Euler 步相同），} \\ X_{t+h} &= X_t + \frac{h}{2}\big(u_t(X_t) + u_{t+h}(X'_{t+h})\big) && \text{使用当前状态与猜测状态下的平均}u\text{ 更新.} \end{aligned}$$

直观而言，Heun 方法先对下一步可能的位置作出初步猜测，再用更新后的猜测修正最初采用的方向。

流模型。 现在通过将向量场变为神经网络向量场 u_t^θ，我们可以通过 ODE 构造生成模型。目前这仅意味着 u_t^θ 是具有参数 θ 的参数化函数 $u^\theta : \mathbb{R}^d \times [0, 1] \to \mathbb{R}^d$。稍后我们会讨论神经网络架构的具体选择。回顾我们的目标是从分布 p_{data} 生成样本 $z \sim p_{\text{data}}$；这些样本尤其必须是随机的。然而 ODE 本身并不随机，而是完全确定的。为注入随机性，我们只需令初始条件 X_0 随机。具体地，选择初始分布 p_{init}。多数情况下，我们取简单的标准高斯分布 $p_{\text{init}} = \mathcal{N}(0, I_d)$。最重要

的是，无论选择什么分布，都必须能在推断时轻松从中采样。流模型随后由以下 ODE 描述：

$$\begin{aligned} X_0 &\sim p_{\text{init}} && \text{随机初始化,} \\ \frac{d}{dt}X_t &= u_t^\theta(X_t) && \text{ODE.} \end{aligned}$$

我们的目标是使轨迹终点 X_1 服从 p_{data}，即

$$X_1 \sim p_{\text{data}} \quad \Longleftrightarrow \quad \psi_1^\theta(X_0) \sim p_{\text{data}},$$

其中 ψ_t^θ 描述由 u_t^θ 诱导的流。但要注意：尽管称为流模型，神经网络参数化的是向量场，而不是流。为了计算流，我们需要模拟 ODE。算法 1 总结了如何从流模型采样。

算法 1：用 Euler 方法从流模型采样。 **输入：**神经网络向量场 u_t^θ，步数 n。

1. 设 $t = 0$。
2. 设步长 $h = 1/n$。
3. 抽取样本 $X_0 \sim p_{\text{init}}$。
4. 对 $i = 1, \ldots, n$：令 $X_{t+h} = X_t + hu_t^\theta(X_t)$，再更新 $t \leftarrow t + h$。
5. 返回 X_1。

2.2 扩散模型

随机微分方程（SDE）将 ODE 的确定性轨迹扩展为随机轨迹。随机轨迹通常称为随机过程 $(X_t)_{0\le t\le 1}$，其形式为：

$$\begin{aligned} &\text{对每个} 0 \le t \le 1, \quad X_t \text{ 是随机变量,} \\ &\text{对} X \text{ 的每次抽样,} \quad X : [0,1] \to \mathbb{R}^d, \quad t \mapsto X_t \text{ 是随机轨迹.} \end{aligned}$$

特别地，当我们将同一个随机过程模拟两次时，可能得到不同结果，因为其动力学被设计为随机的。

布朗运动。 SDE 通过布朗运动构造。布朗运动是在研究物理扩散过程时出现的一种基本随机过程。你可以将布朗运动看作连续随机游走。下面给出定义：布朗运动 $W = (W_t)_{0\le t\le 1}$ 是一个随机过程，满足 $W_0 = 0$、轨迹 $t \mapsto W_t$ 连续，并满足以下两个条件：

1. **正态增量：**对所有 $0 \le s < t$，$W_t - W_s \sim \mathcal{N}(0, (t-s)I_d)$，即增量服从方差随时间线性增长的高斯分布（$I_d$ 为单位矩阵）。
2. **独立增量：**对任意 $0 \le t_0 < t_1 < \cdots < t_n = 1$，增量 $W_{t_1} - W_{t_0}, \ldots, W_{t_n} - W_{t_{n-1}}$ 是相互独立的随机变量。

布朗运动也称为 Wiener 过程，因此我们用 “W” 表示。[1]可以通过取 $W_0 = 0$ 并用步长 $h > 0$ 更新，轻松近似模拟布朗运动：

$$W_{t+h} = W_t + \sqrt{h}\,\epsilon_t, \qquad \epsilon_t \sim \mathcal{N}(0, I_d) \quad (t = 0, h, 2h, \ldots, 1-h). \tag{5}$$

图 2 绘制了几条布朗运动示例轨迹。布朗运动在随机过程研究中的核心地位，正如高斯分布在概率分布研究中的核心地位一样。从金融、统计物理到流行病学，布朗运动的研究具有远超机器学习的广泛应用。例如在金融中，布朗运动用于模拟复杂金融工具的价格。作为数学构造，布朗运动本身也很迷人：例如，虽然布朗运动路径连续（所以画它时可以不抬笔），其长度却无限（因此永远也画不完）。

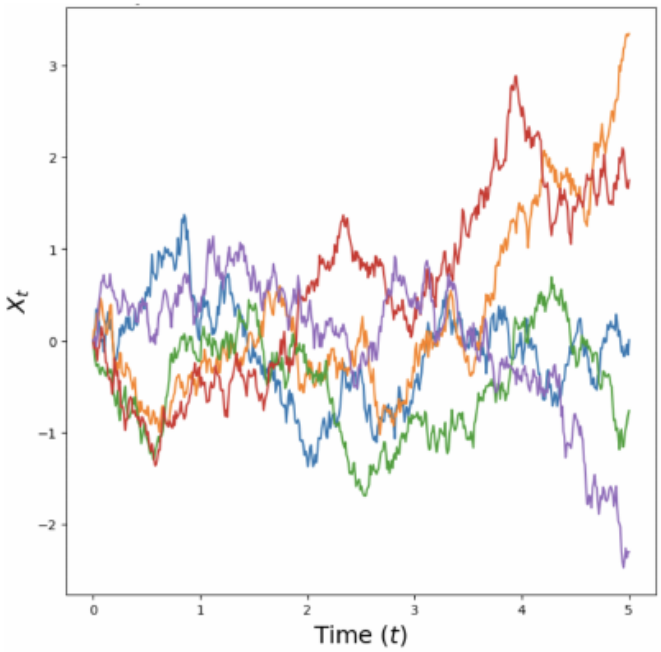


图 2: 利用式 (5) 在 $d = 1$ 维模拟得到的布朗运动 W_t 的示例轨迹。图中 Time 为时间。

从 ODE 到 SDE。 SDE 的思想是向 ODE 的确定性动力学加入由布朗运动驱动的随机动力学。由于一切都是随机的，我们不能再像式 (1a) 那样取导数。因此需要找到一种不使用导数的等价 ODE 表述。为此将 ODE 的轨迹 $(X_t)_{0\le t\le 1}$ 改写为

$$\begin{aligned} & \frac{d}{dt}X_t = u_t(X_t) && \text{通过导数表示} \\ \overset{(i)}{\iff} \quad & \frac{1}{h}(X_{t+h} - X_t) = u_t(X_t) + R_t(h) \\ \iff \quad & X_{t+h} = X_t + h u_t(X_t) + h R_t(h) && \text{通过无穷小更新表示,} \end{aligned}$$

其中 $R_t(h)$ 描述 h 很小时可忽略的函数，即满足 $\lim_{h\to 0} R_t(h) = 0$；(i) 中仅使用了导数的定义。上述推导不过是重述了已知事实：ODE 的轨迹 $(X_t)_{0\le t\le 1}$ 在每个时间步都沿 $u_t(X_t)$ 方向走一

[1]Norbert Wiener 是一位曾在 MIT 任教的著名数学家。现在仍可在 MIT 数学系看到他的画像。

小步。现在我们可以修改最后一个等式，使其变为随机的：SDE 的轨迹 $(X_t)_{0\le t\le 1}$ 在每个时间步都沿 $u_t(X_t)$ 方向走一小步，再加上来自布朗运动的贡献：

$$X_{t+h} = X_t + \underbrace{h u_t(X_t)}_{\text{确定性}} + \underbrace{\sigma_t(W_{t+h} - W_t)}_{\text{随机性}} + \underbrace{h R_t(h)}_{\text{误差项}}, \tag{6}$$

其中 $\sigma_t \ge 0$ 是扩散系数，$R_t(h)$ 是随机误差项，其标准差满足 $\mathbb{E}[\|R_t(h)\|^2]^{1/2} \to 0$，当 $h \to 0$。[2]上述式子描述一个随机微分方程（SDE），通常用下列符号记法表示：

$$dX_t = u_t(X_t)dt + \sigma_t\, dW_t \qquad \text{SDE}, \tag{7a}$$

$$X_0 = x_0 \qquad \text{初始条件}. \tag{7b}$$

不过请始终记住，上述 "dX_t" 记号只是式 (6) 的纯非正式记法。遗憾的是，SDE 不再具有流映射 ϕ_t，因为 X_t 的值不再完全由 $X_0 \sim p_{\text{init}}$ 决定，而是演化本身也具有随机性。尽管如此，与 ODE 类似，我们有：

定理 5 (SDE 解的存在性与唯一性)**.** *如果 $u : \mathbb{R}^d \times [0,1] \to \mathbb{R}^d$ 连续可微且导数有界，并且 σ_t 连续，那么式 (7) 中的 SDE 有解，即满足式 (6) 的唯一随机过程 $(X_t)_{0\le t\le 1}$。*

若这是一门随机微积分课程，我们将花数讲证明此定理并以完整的数学严格性构造 SDE，即从基本原理构造布朗运动，再通过随机积分构造过程 X_t。由于本课程聚焦机器学习，我们请读者参阅文献 [29] 了解更技术性的处理。最后注意每个 ODE 也是 SDE，只需令扩散系数 $\sigma_t = 0$。因此在本课程剩余部分中，谈及 SDE 时，我们将 ODE 视作其特殊情形。

例 6 (Ornstein–Uhlenbeck 过程)**.** *取常数扩散系数 $\sigma_t = \sigma \ge 0$ 和常数线性漂移 $u_t(x) = -\theta x$ $(\theta > 0)$，得到 SDE*

$$dX_t = -\theta X_t\, dt + \sigma\, dW_t. \tag{8}$$

上述 SDE 的解 $(X_t)_{0\le t\le 1}$ 称为 Ornstein–Uhlenbeck (OU) 过程，见图 3。向量场 $-\theta x$ 将过程推回中心 0（因为漂移总指向当前位置的反方向），而扩散系数 σ 不断加入更多噪声。若模拟至 $t \to \infty$，这一过程收敛到高斯分布 $\mathcal{N}(0, \sigma^2/(2\theta))$。注意 $\sigma = 0$ 时，我们得到式 (3) 中研究过的线性向量场流。

模拟 SDE。 如果至今仍觉得 SDE 的抽象定义难以理解，不必担心。更直观的理解方式是回答如何模拟 SDE。最简单的方案称为 Euler–Maruyama 方法，它之于 SDE 基本上就如 Euler 方法之于 ODE。使用 Euler–Maruyama 方法时，以 $X_0 = x_0$ 初始化，并按下式迭代更新：

$$X_{t+h} = X_t + h u_t(X_t) + \sqrt{h}\,\sigma_t \epsilon_t, \qquad \epsilon_t \sim \mathcal{N}(0, I_d), \tag{9}$$

其中 $h = n^{-1} > 0$ 是步长超参数，$n \in \mathbb{N}$。换言之，Euler–Maruyama 模拟沿 $u_t(X_t)$ 方向迈一小步，同时加上一点尺度为 $\sqrt{h}\sigma_t$ 的高斯噪声。在本课程（包括配套实验）中模拟 SDE 时，我们通常采用 Euler–Maruyama 方法。

[2]译注：按下方定理仅要求 σ_t 连续，这里对 $R_t(h)$ 的均方极限一般过强。例如取 $u = 0$、$\sigma_s = \sqrt{s}$、$t = 0$，则随机积分的方差为 $h^2/2$，除以 h 后的均方范数不趋于零。这不否定通常以随机积分定义的 SDE 解的存在性；它表明此处余项表述需额外条件或不同阶的误差记法。

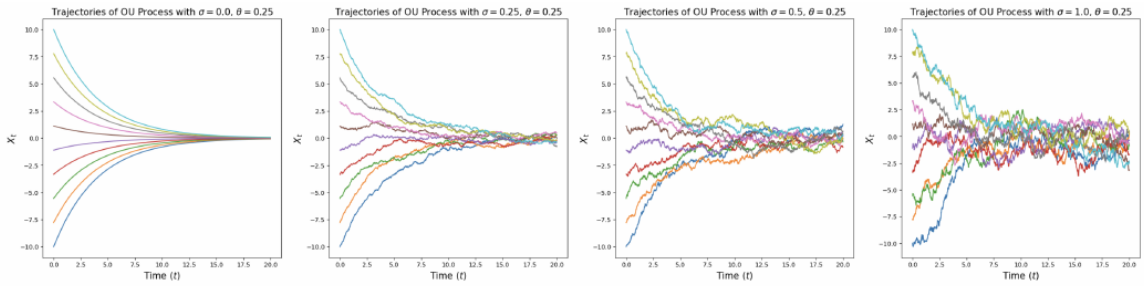


图 3: 一维 Ornstein–Uhlenbeck 过程（式 (8)）的示意，$\theta = 0.25$，σ 取不同值并从左到右递增。当 $\sigma = 0$ 时恢复收敛到原点的流（平滑的确定性轨迹）；当 $\sigma > 0$ 时，随机路径的分布趋于高斯分布 $\mathcal{N}(0, \sigma^2/(2\theta))$。图中 Time 为时间。译注：图题 Trajectories of OU Process 为 “OU 过程轨迹”；从左到右 $\sigma = 0, 0.25, 0.5, 1.0$，均有 $\theta = 0.25$。

扩散模型。 现在可以像处理 ODE 一样，通过 SDE 构造生成模型。回顾我们的目标是把简单分布 p_{init} 转化为复杂分布 p_{data}。与 ODE 一样，模拟随机初始化为 $X_0 \sim p_{\text{init}}$ 的 SDE 是实现这种转换的自然选择。为了参数化该 SDE，只需用神经网络 u_t^θ 参数化其核心成分——向量场 u_t。于是扩散模型由下式给出：

$$\begin{aligned} X_0 &\sim p_{\text{init}} && \text{随机初始化,} \\ dX_t &= u_t^\theta(X_t)dt + \sigma_t\, dW_t && \text{SDE.} \end{aligned}$$

算法 2 描述用 Euler–Maruyama 方法从扩散模型采样的过程。我们总结本节结果如下。

算法 2：从扩散模型采样（Euler–Maruyama 方法）。 **输入：** 神经网络 u_t^θ、步数 n、扩散系数 σ_t。

1. 设 $t = 0$。
2. 设步长 $h = 1/n$。
3. 抽取样本 $X_0 \sim p_{\text{init}}$。
4. 对 $i = 1, \ldots, n$：抽取 $\epsilon \sim \mathcal{N}(0, I_d)$，令 $X_{t+h} = X_t + hu_t^\theta(X_t) + \sigma_t\sqrt{h}\,\epsilon$，再更新 $t \leftarrow t + h$。
5. 返回 X_1。

总结 7 (SDE 生成模型)**.** 在本文中，扩散模型由参数为 θ 的神经网络 u_t^θ 和固定扩散系数 σ_t 构成：

$$\begin{aligned} \text{神经网络：} \quad & u^\theta : \mathbb{R}^d \times [0,1] \to \mathbb{R}^d, \quad (x,t) \mapsto u_t^\theta(x), \quad \text{参数为}\theta, \\ \text{固定：} \quad & \sigma : [0,1] \to [0,\infty), \quad t \mapsto \sigma_t. \end{aligned}$$

为从 *SDE* 模型获得样本（即生成对象），过程如下：

$$\begin{aligned} \text{初始化：} \quad & X_0 \sim p_{\text{init}} && \text{用简单分布初始化，如高斯分布,} \\ \text{模拟：} \quad & dX_t = u_t^\theta(X_t)dt + \sigma_t\, dW_t && \text{从 } 0 \text{ 到 } 1 \text{ 模拟 } SDE, \\ \text{目标：} \quad & X_1 \sim p_{\text{data}} && \text{目标是使} X_1 \text{ 服从} p_{\text{data}}. \end{aligned}$$

扩散系数 $\sigma_t = 0$ 的扩散模型是流模型。

3 流匹配

上一节，我们将流模型和扩散模型构造为由神经网络向量场 u_t^θ 参数化的生成模型。然而，我们还没有讨论如何训练它们，即如何优化参数 θ，使生成模型返回有意义的结果，例如好看的图像或精彩的视频。接下来讨论流匹配 [25, 1, 27]，这是一种简单、可扩展且处于当前先进水平的 u_t^θ 训练算法。

本节限制在流模型，即我们有神经网络 u_t^θ，通过模拟 ODE

$$X_0 \sim p_{\text{init}}, \qquad dX_t = u_t^\theta(X_t)dt \qquad \text{（流模型）} \tag{10}$$

并使用 $t = 1$ 时的终点 X_1 作为样本，从生成模型获得样本。如前所述，目标是使 X_1 服从数据分布 p_{data}，即 $X_1 \sim p_{\text{data}}$。因此，“如何训练”神经网络的问题实际上是：*如何优化 θ，使模拟式 (10) 的流模型得到来自数据分布的样本 $X_1 \sim p_{\text{data}}$？*

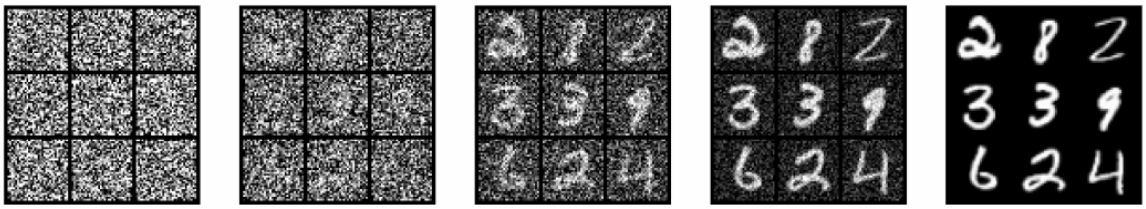


图 4: 通过高斯条件概率路径，使一组图像从噪声向数据逐渐插值。每张图像是维数为 $d = 32\times32$ 的数据点，因此这里绘制的是概率路径的各个样本；而图 5 将分布绘制为二维直方图。

3.1 条件概率路径与边缘概率路径

流匹配的第一步是指定一条概率路径。直观而言，概率路径指定了噪声 p_{init} 和数据 p_{data} 之间的逐渐插值（见图 4）。但为什么要这样做？回顾我们期望的 ODE 轨迹在 $t = 0$ 时满足 $X_0 \sim p_{\text{init}}$，在 $t = 1$ 时满足 $X_1 \sim p_{\text{data}}$。那么起点与终点之间的 $0 < t < 1$ 时刻又如何？事实证明，我们有一定自由选择中间应发生的事情，而概率路径正是对此所做的数学形式化。

下文对数据点 $z \in \mathbb{R}^d$，用 δ_z 表示 Dirac delta “分布”。这是可以想象的最简单分布：从 δ_z 采样总是返回 z（即它是确定性的）。*条件（插值）概率路径*是 $\mathbb{R}^d$ 上一组满足下式的分布 $p_t(x \mid z)$：

$$p_0(\cdot \mid z) = p_{\text{init}}, \qquad p_1(\cdot \mid z) = \delta_z \quad \text{对所有} z \in \mathbb{R}^d. \tag{11}$$

换言之，条件概率路径逐渐将初始分布 p_{init} 转换成单个数据点（例如见图 4）。你可以将概率路径看作分布空间中的一条轨迹。每条条件概率路径 $p_t(x \mid z)$ 都诱导一条*边缘概率路径* $p_t(x)$：先从数据分布采样数据点 $z \sim p_{\text{data}}$，再从 $p_t(\cdot \mid z)$ 采样，所得分布定义为边缘路径：

$$z \sim p_{\text{data}}, \quad x \sim p_t(\cdot \mid z) \quad \Longrightarrow \quad x \sim p_t \qquad \text{从边缘路径采样,} \tag{12}$$

$$p_t(x) = \int p_t(x \mid z) p_{\text{data}}(z)\, dz \qquad \text{边缘路径的密度.} \tag{13}$$

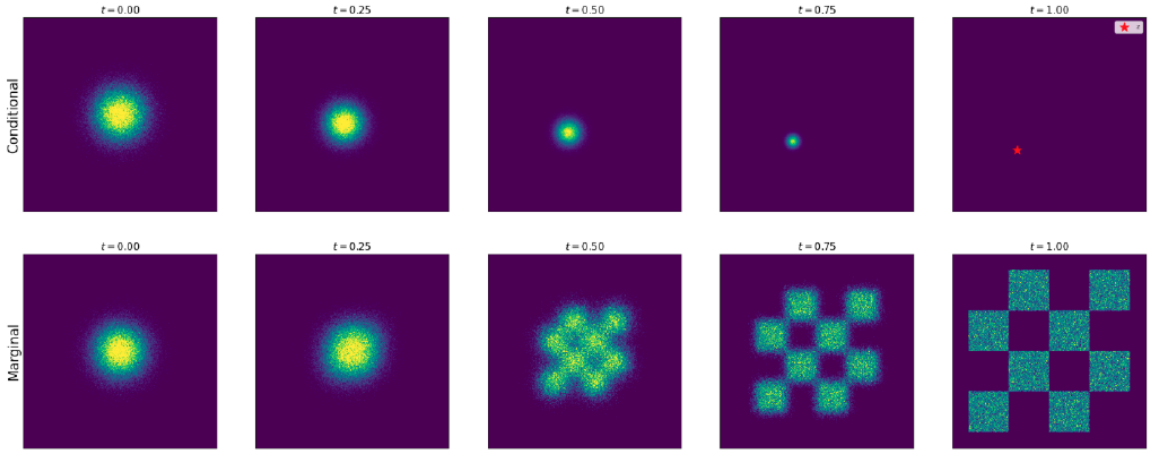


图 5: 条件概率路径（上）与边缘概率路径（下）示意。此处绘制 $\alpha_t = t$、$\beta_t = 1 - t$ 的高斯概率路径。条件概率路径在高斯分布 $p_{\text{init}} = \mathcal{N}(0, I_d)$ 与单个数据点 z 对应的 $p_{\text{data}} = \delta_z$ 之间插值；边缘概率路径在高斯分布与数据分布 p_{data} 之间插值。此处 p_{data} 是由棋盘图案表示的 $d = 2$ 玩具分布。图中 Conditional / Marginal 分别表示条件 / 边缘。

注意我们知道如何从 p_t 采样，但不知道密度值 $p_t(x)$，因为积分不可计算（即实际上可以计算式 (12)，却无法计算式 (13)）。请自行验证，由于式 (11) 中对 $p_t(\cdot \mid z)$ 的条件，边缘概率路径 p_t 在 p_{init} 与 p_{data} 之间插值：

$$p_0 = p_{\text{init}}, \qquad p_1 = p_{\text{data}} \qquad \text{噪声—数据插值.} \tag{14}$$

其中，高斯概率路径是迄今最重要的例子，因此我们强烈建议仔细阅读下例。

例 8 (高斯条件概率路径)**.** 一个特别常用的概率路径是高斯概率路径，大多数当前先进模型都使用它。设 α_t, β_t 为噪声调度函数：两个连续可微的单调函数，满足 $\alpha_0 = \beta_1 = 0$ 和 $\alpha_1 = \beta_0 = 1$。定义条件概率路径

$$p_t(\cdot \mid z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d) \qquad \text{高斯条件路径.} \tag{15}$$

根据对 α_t, β_t 施加的条件，它满足

$$p_0(\cdot \mid z) = \mathcal{N}(\alpha_0 z, \beta_0^2 I_d) = \mathcal{N}(0, I_d), \qquad p_1(\cdot \mid z) = \mathcal{N}(\alpha_1 z, \beta_1^2 I_d) = \delta_z,$$

其中使用了方差为零、均值为 z 的正态分布就是 δ_z 这一事实。因此，此 $p_t(x \mid z)$ 在 $p_{\text{init}} = \mathcal{N}(0, I_d)$ 时满足式 (11)，从而是一条有效的条件插值路径。图 4 展示了它对图像的应用。从边缘路径 p_t 采样可表示为

$$z \sim p_{\text{data}}, \quad \epsilon \sim p_{\text{init}} = \mathcal{N}(0, I_d) \quad \Longrightarrow \quad x = \alpha_t z + \beta_t \epsilon \sim p_t \qquad \text{从边缘高斯路径采样.} \tag{16}$$

直观地说，t 越低上述过程加入的噪声越多，直到 $t = 0$ 时只剩噪声。图 5 绘制了这类插值路径的例子。

3.2 条件向量场与边缘向量场

概率路径 $(p_t)_{0\le t\le 1}$ 指定了沿轨迹的点 X_t 应具有的分布 $X_t \sim p_t$。目前这只是我们“希望”发生的事情。那么，如何找到一个向量场，使轨迹 X_t 遵循概率路径？流匹配显式构造了这样一个向量场——“边缘向量场”——本节将予以解释。

对每个数据点 $z \in \mathbb{R}^d$，用 $u_t^{\text{target}}(\cdot \mid z)$ 表示一个条件向量场。它可以是任何使相应 ODE 给出条件概率路径 $p_t(\cdot \mid z)$ 的向量场，即满足

$$X_0 \sim p_{\text{init}}, \quad \frac{d}{dt}X_t = u_t^{\text{target}}(X_t \mid z) \quad \Longrightarrow \quad X_t \sim p_t(\cdot \mid z) \quad (0 \le t \le 1). \tag{17}$$

我们经常可以手工解析地找到条件向量场 $u_t^{\text{target}}(\cdot \mid z)$（即自己进行代数计算）。我们将在例 10 中为例 8 的高斯概率路径推导一个条件向量场 $u_t(x \mid z)$，以此说明。

乍看之下，条件向量场似乎没有用，因为 ODE 的所有终点 X_1 都坍缩到 $X_1 = z$，也就是说我们只是在重新生成已知数据点 z。然而，条件向量场是构造一个能够从 p_{data} 生成真实样本的向量场的构件：

定理 9 (边缘化技巧)**.** *设 $u_t^{\text{target}}(x \mid z)$ 是条件向量场（式 (17)）。则定义为*

$$u_t^{\text{target}}(x) = \int u_t^{\text{target}}(x \mid z) \frac{p_t(x \mid z)p_{\text{data}}(z)}{p_t(x)}\, dz \tag{18}$$

的边缘向量场遵循边缘概率路径，即

$$X_0 \sim p_{\text{init}}, \quad \frac{d}{dt}X_t = u_t^{\text{target}}(X_t) \quad \Longrightarrow \quad X_t \sim p_t \quad (0 \le t \le 1). \tag{19}$$

特别地，对于这个 ODE 有 $X_1 \sim p_{\text{data}}$，因此我们可以说“u_t^{target} 将噪声 p_{init} 转换为数据 p_{data}”。

例 10 (高斯概率路径的目标 ODE)**.** *如前，设 $p_t(\cdot \mid z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$，其中 α_t, β_t 是噪声调度函数（见式 (15)）。令 $\dot{\alpha}_t = \partial_t \alpha_t$ 和 $\dot{\beta}_t = \partial_t \beta_t$ 表示各自的时间导数。这里希望证明以下条件高斯向量场*

$$u_t^{\text{target}}(x \mid z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t}\alpha_t\right) z + \frac{\dot{\beta}_t}{\beta_t} x \tag{20}$$

译注：含 $1/\beta_t$ 的表示需在 $\beta_t > 0$ 的时间上使用；$\beta_1 = 0$ 时条件分布终点是单点质量，不能直接将该式作为终点的普通有界向量场。是定理 9 意义下的有效条件向量场模型：若 $X_0 \sim \mathcal{N}(0, I_d)$，其 ODE 轨迹 X_t 满足 $X_t \sim p_t(\cdot \mid z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$。图 6 通过比较条件概率路径的样本（真实值）与该流的模拟 ODE 轨迹的样本，在视觉上确认了这一点。可以看到分布匹配。下面给出证明。

证明. 先通过定义下式构造条件流模型 $\psi_t^{\text{target}}(x \mid z)$：

$$\psi_t^{\text{target}}(x \mid z) = \alpha_t z + \beta_t x. \tag{21}$$

若 X_t 是 $\psi_t^{\text{target}}(\cdot \mid z)$ 的 ODE 轨迹，且 $X_0 \sim p_{\text{init}} = \mathcal{N}(0, I_d)$，则根据定义

$$X_t = \psi_t^{\text{target}}(X_0 \mid z) = \alpha_t z + \beta_t X_0 \sim \mathcal{N}(\alpha_t z, \beta^2 I_d) = p_t(\cdot \mid z).$$

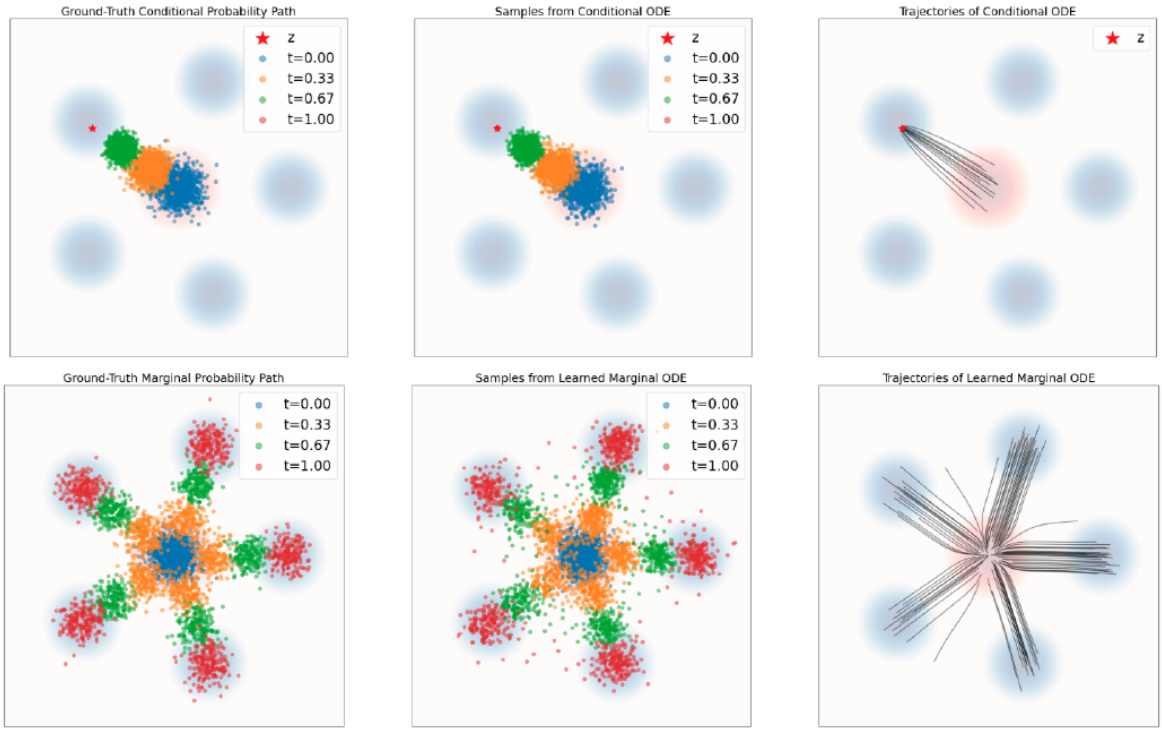


图 6: 定理 9 的示意：用 ODE 模拟概率路径。数据分布 p_{data} 为蓝色背景，高斯分布 p_{init} 为红色背景。上排是条件概率路径：左为条件路径 $p_t(\cdot \mid z)$ 的真实样本，中为随时间演化的 ODE 样本，右为模拟式 (20) 中条件向量场的 ODE 所得轨迹。下排为边缘概率路径：左为 p_t 的真实样本，中为随时间演化的 ODE 样本，右为用边缘向量场 $u_t^{\text{flow}}(x)$ 模拟 ODE 所得轨迹。条件向量场遵循条件概率路径，边缘向量场遵循边缘概率路径。译注：图中 Ground-Truth、Samples、Trajectories 分别为真实分布、样本和轨迹；Conditional / Marginal 为条件 / 边缘，Learned 为学得的。时间图例为 $t = 0, 0.33, 0.67, 1$。

我们得出轨迹按条件概率路径分布（即满足式 (17)）。剩下的任务是从 $\psi_t^{\text{target}}(x \mid z)$ 中提取向量场 $u_t^{\text{target}}(x \mid z)$。根据流的定义（式 (2)），有

$$\begin{aligned}
&\frac{d}{dt}\psi_t^{\text{target}}(x \mid z) = u_t^{\text{target}}(\psi_t^{\text{target}}(x \mid z) \mid z) \quad \text{对所有} x, z \in \mathbb{R}^d \\
\overset{(i)}{\iff} \quad &\dot{\alpha}_t z + \dot{\beta}_t x = u_t^{\text{target}}(\alpha_t z + \beta_t x \mid z) \quad \text{对所有} x, z \in \mathbb{R}^d \\
\overset{(ii)}{\iff} \quad &\dot{\alpha}_t z + \dot{\beta}_t \left(\frac{x - \alpha_t z}{\beta_t} \right) = u_t^{\text{target}}(x \mid z) \quad \text{对所有} x, z \in \mathbb{R}^d \\
\overset{(iii)}{\iff} \quad &\left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t}\alpha_t \right) z + \frac{\dot{\beta}_t}{\beta_t} x = u_t^{\text{target}}(x \mid z) \quad \text{对所有} x, z \in \mathbb{R}^d,
\end{aligned}$$

其中 (i) 使用了 $\psi_t^{\text{target}}(x \mid z)$ 的定义（式 (21)），(ii) 重新参数化 $x \mapsto (x - \alpha_t z)/\beta_t$，$(iii)$ 只是进行了代数整理。注意，最后一个等式就是我们在式 (20) 中定义的条件高斯向量场。这就证明了该命题。[3] □

图 6 是定理 9 的示意。让我们直观理解边缘向量场。统计学中的 Bayes 法则表明，下式描述了一个后验分布：

$$\frac{p_t(x \mid z) p_{\text{data}}(z)}{p_t(x)} = \text{“给定带噪数据} x \text{ 时数据点} z \text{ 的后验”},$$

其中 $p_{\text{data}}(z)$ 是先验分布。边缘向量场只是一个平均：对每个可能的数据点 z，取速度 $u_t(x \mid z)$——即将我们带向 z 的方向——再按我们相信 x 来自 z 的程度对该速度加权。对所有数据点平均后便得到边缘向量场。

本节剩余部分将使这一直觉严格化，并证明定理 9。主要数学工具是*连续性方程*，它是数学和物理中的基本方程。定义散度算符 div 为

$$\text{div}(v_t)(x) = \sum_{i=1}^{d} \frac{\partial}{\partial x_i} v_t^i(x), \tag{22}$$

其中 v_t^i 是 v_t 的第 i 个坐标。

定理 11 (连续性方程)**.** *考虑一个向量场为 u_t^{target} 的流模型，其中 $X_0 \sim p_{\text{init}} = p_0$。则对所有 $0 \le t \le 1$ 有 $X_t \sim p_t$，当且仅当*

$$\partial_t p_t(x) = -\text{div}(p_t u_t^{\text{target}})(x) \quad \text{对所有} x \in \mathbb{R}^d,\ 0 \le t \le 1, \tag{23}$$

其中 $\partial_t p_t(x) = \frac{d}{dt} p_t(x)$ 表示 $p_t(x)$ 的时间导数。式 (23) 称为连续性方程。

对于偏重数学的读者，我们在附录 B 给出连续性方程的自包含证明。

在继续之前，让我们尝试直观地理解连续性方程。左端 $\partial_t p_t(x)$ 描述 x 处概率 $p_t(x)$ 随时间的变化。直观上，这一变化应对应概率质量的净流入。对于流模型，粒子 X_t 沿向量场 u_t^{target} 运动。如你可能从物理学中记得的，散度度量向量场的某种净流出，因此负散度度量净流入。将其按当前位于 x 处的总概率质量缩放，可得净值 $-\text{div}(p_t u_t)$ 度量总流入。由于概率质量守恒（积分始终为 1），方程左右两端应当相等！下面证明定理 9 的边缘化技巧。

[3] 也可以将其代入本节稍后介绍的连续性方程进行再次检验。

定理 9 的证明. 由定理 11，我们需要证明按式 (18) 定义的边缘向量场 u_t^{target} 满足连续性方程。直接计算可得

$$\begin{aligned}
\partial_t p_t(x) &\overset{(i)}{=} \partial_t \int p_t(x \mid z) p_{\text{data}}(z)\, dz = \int \partial_t p_t(x \mid z) p_{\text{data}}(z)\, dz \\
&\overset{(ii)}{=} \int -\operatorname{div}(p_t(\cdot \mid z) u_t^{\text{target}}(\cdot \mid z))(x) p_{\text{data}}(z)\, dz \\
&\overset{(iii)}{=} -\operatorname{div}\left(\int p_t(x \mid z) u_t^{\text{target}}(x \mid z) p_{\text{data}}(z)\, dz \right) \\
&\overset{(iv)}{=} -\operatorname{div}\left(p_t(x) \int u_t^{\text{target}}(x \mid z) \frac{p_t(x \mid z) p_{\text{data}}(z)}{p_t(x)}\, dz \right)(x) \\
&\overset{(v)}{=} -\operatorname{div}(p_t u_t^{\text{target}})(x),
\end{aligned}$$

其中 (i) 使用式 (12) 中 $p_t(x)$ 的定义；(ii) 使用条件概率路径 $p_t(\cdot \mid z)$ 的连续性方程；(iii) 利用式 (22) 交换积分与散度算符；(iv) 乘以并除以 $p_t(x)$；(v) 使用式 (18)。上述等式链的首尾表明 u_t^{target} 满足连续性方程。由定理 11，这足以推出式 (19)，证明完毕。 □

3.3 学习边缘向量场

现在我们可以描述训练算法。流匹配的目标是训练神经网络 u_t^θ，使其等于边缘向量场 u_t^{target}。如果达到这一点，由定理 9 可知，终点 $X_1 \sim p_{\text{data}}$ 具有期望的分布。下文用 $\text{Unif} = \text{Unif}_{[0,1]}$ 表示区间 $[0,1]$ 上的均匀分布，用 $\mathbb{E}$ 表示随机变量的期望。获得 $u_t^\theta \approx u_t^{\text{target}}$ 的直观方式是使用均方误差，即使用定义如下的流匹配损失：

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t \sim \text{Unif},\, x \sim p_t}\left[\| u_t^\theta(x) - u_t^{\text{target}}(x) \|^2 \right] \tag{24}$$

$$\overset{(i)}{=} \mathbb{E}_{t \sim \text{Unif},\, z \sim p_{\text{data}},\, x \sim p_t(\cdot \mid z)}\left[\| u_t^\theta(x) - u_t^{\text{target}}(x) \|^2 \right], \tag{25}$$

其中 $p_t(x) = \int p_t(x \mid z) p_{\text{data}}(z)\, dz$ 是边缘概率路径；(i) 使用了式 (12) 给出的采样过程。直观地说，这个损失表示：首先抽取随机时间 $t \in [0,1]$；其次从数据集随机抽取点 z，从 $p_t(\cdot \mid z)$ 采样（例如加入一些噪声），并计算 $u_t^\theta(x)$；最后计算神经网络输出与边缘向量场 $u_t^{\text{target}}(x)$ 的均方误差。不幸的是，我们还没有完成。虽然定理 9 给出了 u_t^{target} 的公式，但由于积分不可计算，我们无法高效求出它。相反，我们将利用条件速度场 $u_t^{\text{target}}(x \mid z)$ 可以计算这一事实。

为此定义条件流匹配损失：

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t \sim \text{Unif},\, z \sim p_{\text{data}},\, x \sim p_t(\cdot \mid z)}\left[\| u_t^\theta(x) - u_t^{\text{target}}(x \mid z) \|^2 \right]. \tag{26}$$

注意它与式 (24) 的差别：我们使用条件向量场 $u_t^{\text{target}}(x \mid z)$，而不是边缘向量场 $u_t^{\text{target}}(x)$。由于我们有 $u_t^{\text{target}}(x \mid z)$ 的解析公式，可以轻松最小化上述损失。但且慢，既然我们关心的是边缘向量场，对条件向量场做回归又有什么意义？事实证明，显式地对可计算的条件向量场回归，实际上隐式地对不可计算的边缘向量场回归。下一个结果使这一直觉精确化。

定理 12. *边缘流匹配损失与条件流匹配损失仅相差一个常数，即*

$$\mathcal{L}_{\text{FM}}(\theta) = \mathcal{L}_{\text{CFM}}(\theta) + C,$$

其中 C 与 θ 无关。因此，它们的梯度相同：

$$\nabla_\theta \mathcal{L}_{\mathrm{FM}}(\theta) = \nabla_\theta \mathcal{L}_{\mathrm{CFM}}(\theta).$$

因此，例如使用随机梯度下降（SGD）最小化 $\mathcal{L}_{\mathrm{CFM}}(\theta)$ 等价于以同样方式最小化 $\mathcal{L}_{\mathrm{FM}}(\theta)$。特别地，对于 $\mathcal{L}_{\mathrm{CFM}}(\theta)$ 的最小化解 θ^*，将有 $u_t^{\theta^*} = u_t^{\mathrm{target}}$，即神经网络将等于边缘向量场（假设参数化具有无限表达能力）。

直接证明. 证明方法是将均方误差展开为三个部分并移除常数：

$$\begin{aligned}
\mathcal{L}_{\mathrm{FM}}(\theta) &\overset{(i)}{=} \mathbb{E}_{t\sim\mathrm{Unif},x\sim p_t}[\|u_t^\theta(x) - u_t^{\mathrm{target}}(x)\|^2] \\
&\overset{(ii)}{=} \mathbb{E}_{t\sim\mathrm{Unif},x\sim p_t}[\|u_t^\theta(x)\|^2 - 2u_t^\theta(x)^T u_t^{\mathrm{target}}(x) + \|u_t^{\mathrm{target}}(x)\|^2] \\
&\overset{(iii)}{=} \mathbb{E}_{t\sim\mathrm{Unif},x\sim p_t}[\|u_t^\theta(x)\|^2] - 2\mathbb{E}_{t\sim\mathrm{Unif},x\sim p_t}[u_t^\theta(x)^T u_t^{\mathrm{target}}(x)] \\
&\quad + \underbrace{\mathbb{E}_{t\sim\mathrm{Unif}_{[0,1]},x\sim p_t}[\|u_t^{\mathrm{target}}(x)\|^2]}_{=:C_1} \\
&\overset{(iv)}{=} \mathbb{E}_{t\sim\mathrm{Unif},z\sim p_{\mathrm{data}},x\sim p_t(\cdot|z)}[\|u_t^\theta(x)\|^2] \\
&\quad - 2\mathbb{E}_{t\sim\mathrm{Unif},x\sim p_t}[u_t^\theta(x)^T u_t^{\mathrm{target}}(x)] + C_1,
\end{aligned}$$

其中 (i) 由定义成立；(ii) 使用公式 $\|a-b\|^2 = \|a\|^2 - 2a^Tb + \|b\|^2$；$(iii)$ 定义常数 C_1；(iv) 使用式 (12) 给出的 p_t 采样过程。我们重新表达第二项：

$$\begin{aligned}
&\mathbb{E}_{t\sim\mathrm{Unif},x\sim p_t}[u_t^\theta(x)^T u_t^{\mathrm{target}}(x)] \\
&\overset{(i)}{=} \int_0^1 \int p_t(x) u_t^\theta(x)^T u_t^{\mathrm{target}}(x)\, dx\, dt \\
&\overset{(ii)}{=} \int_0^1 \int p_t(x) u_t^\theta(x)^T \left[\int u_t^{\mathrm{target}}(x \mid z) \frac{p_t(x \mid z) p_{\mathrm{data}}(z)}{p_t(x)}\, dz\right] dx\, dt \\
&\overset{(iii)}{=} \int_0^1 \iint u_t^\theta(x)^T u_t^{\mathrm{target}}(x \mid z) p_t(x \mid z) p_{\mathrm{data}}(z)\, dz\, dx\, dt \\
&\overset{(iv)}{=} \mathbb{E}_{t\sim\mathrm{Unif},z\sim p_{\mathrm{data}},x\sim p_t(\cdot|z)}[u_t^\theta(x)^T u_t^{\mathrm{target}}(x \mid z)],
\end{aligned}$$

其中 (i) 将期望表示为积分，(ii) 使用式 (18)，(iii) 使用积分的线性性，(iv) 将积分表示为期望。注意这正是证明的关键一步：等式开头使用边缘向量场 $u_t^{\mathrm{target}}(x)$，而结尾使用条件向量场 $u_t^{\mathrm{target}}(x \mid z)$。将其代回 $\mathcal{L}_{\mathrm{FM}}$ 的表达式：

$$\begin{aligned}
\mathcal{L}_{\mathrm{FM}}(\theta) &\overset{(i)}{=} \mathbb{E}_{t,z,x}[\|u_t^\theta(x)\|^2] - 2\mathbb{E}_{t,z,x}[u_t^\theta(x)^T u_t^{\mathrm{target}}(x \mid z)] + C_1 \\
&\overset{(ii)}{=} \mathbb{E}_{t,z,x}[\|u_t^\theta(x)\|^2 - 2u_t^\theta(x)^T u_t^{\mathrm{target}}(x \mid z) + \|u_t^{\mathrm{target}}(x \mid z)\|^2 - \|u_t^{\mathrm{target}}(x \mid z)\|^2] + C_1 \\
&\overset{(iii)}{=} \mathbb{E}_{t,z,x}[\|u_t^\theta(x) - u_t^{\mathrm{target}}(x \mid z)\|^2] + \underbrace{\mathbb{E}_{t,z,x}[-\|u_t^{\mathrm{target}}(x \mid z)\|^2]}_{C_2} + C_1 \\
&\overset{(iv)}{=} \mathcal{L}_{\mathrm{CFM}}(\theta) + \underbrace{C_2 + C_1}_{=:C},
\end{aligned}$$

这里的 $\mathbb{E}_{t,z,x}$ 沿用上述联合采样分布。其中 (i) 代入刚推导的等式，(ii) 加上并减去相同的值，(iii) 再次使用平方范数公式，(iv) 定义关于 θ 的常数。证明完毕。 □

因此，流匹配训练就是最小化条件流匹配损失。训练过程在算法 3 中总结，并在图 7 中展示。这个算法有几个显著特点：首先，训练期间实际上从不模拟任何 ODE，人们称这一特点为免模拟。这使训练非常便宜，因为不必在训练期间展开 ODE 轨迹（那需要许多步）。其次，训练是简单的回归目标——我们只是在对 $u_t^{\text{target}}(x \mid z)$ 做回归，因此与监督学习并无太大不同。最后，这一算法极为简单，很难想象还有简单得多的训练目标。所有这些使流匹配成为大型机器学习模型中极具吸引力的方法。训练好 u_t^θ 后，可以模拟流模型

$$dX_t = u_t^\theta(X_t)\,dt, \qquad X_0 \sim p_{\text{init}} \tag{27}$$

例如用算法 1 获得样本 $X_1 \sim p_{\text{data}}$。在文献中，整个流程称为流匹配 [25, 27, 1, 26]。现在将条件流匹配损失具体化到高斯概率路径。

算法 3：流匹配训练过程（对于高斯 CondOT 路径 $p_t(x \mid z) = \mathcal{N}(tz, (1-t)^2)$）。 **输入：** 样本 $z \sim p_{\text{data}}$ 构成的数据集，神经网络 u_t^θ。

1. 对每个数据小批量，执行以下步骤：
2. 从数据集采样一个数据例子 z。
3. 采样随机时间 $t \sim \text{Unif}_{[0,1]}$。
4. 采样噪声 $\epsilon \sim \mathcal{N}(0, I_d)$。
5. 令 $x = tz + (1-t)\epsilon$（一般情形：$x \sim p_t(\cdot \mid z)$）。
6. 计算损失 $\mathcal{L}(\theta) = \|u_t^\theta(x) - (z - \epsilon)\|^2$（一般情形：$\|u_t^\theta(x) - u_t^{\text{target}}(x \mid z)\|^2$）。
7. 更新 $\theta \leftarrow \text{grad_update}(\mathcal{L}(\theta))$；对每个小批量重复。

例 13 (高斯条件概率路径的流匹配)**.** 回到高斯概率路径 $p_t(\cdot \mid z) = \mathcal{N}(\alpha_t z; \beta_t^2 I_d)$ 的例子；我们可以通过以下方式从条件路径采样：

$$\epsilon \sim \mathcal{N}(0, I_d) \quad \Longrightarrow \quad x_t = \alpha_t z + \beta_t \epsilon \sim \mathcal{N}(\alpha_t z, \beta_t^2 I_d) = p_t(\cdot \mid z). \tag{28}$$

如式 (20) 所推导，条件向量场 $u_t^{\text{target}}(x \mid z)$ 为

$$u_t^{\text{target}}(x \mid z) = \left(\dot\alpha_t - \frac{\dot\beta_t}{\beta_t}\alpha_t\right) z + \frac{\dot\beta_t}{\beta_t} x, \tag{29}$$

其中 $\dot\alpha_t = \partial_t \alpha_t$ 和 $\dot\beta_t = \partial_t \beta_t$ 是各自的时间导数。代入该公式，条件流匹配损失为

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t\sim\text{Unif}, z\sim p_{\text{data}}, x\sim\mathcal{N}(\alpha_t z, \beta_t^2 I_d)} \left[\left\| u_t^\theta(x) - \left(\dot\alpha_t - \frac{\dot\beta_t}{\beta_t}\alpha_t\right) z - \frac{\dot\beta_t}{\beta_t} x \right\|^2 \right] \tag{30}$$

$$\overset{(i)}{=} \mathbb{E}_{t\sim\text{Unif}, z\sim p_{\text{data}}, \epsilon\sim\mathcal{N}(0, I_d)} \left[\|u_t^\theta(\alpha_t z + \beta_t \epsilon) - (\dot\alpha_t z + \dot\beta_t \epsilon)\|^2 \right], \tag{31}$$

其中 (i) 代入式 (28)，将 x 替换为 $\alpha_t z + \beta_t \epsilon$。注意 $\mathcal{L}_{\text{CFM}}$ 的简单性：采样数据点 z，采样一些噪声 ϵ，然后取均方误差。对于 $\alpha_t = t$、$\beta_t = 1 - t$ 的特殊情况，让我们再具体一些。对应概率 $p_t(x \mid z) = \mathcal{N}(tz, (1-t)^2)$ 有时称为（高斯）*CondOT* 概率路径。此时 $\dot\alpha_t = 1$、$\dot\beta_t = -1$，因此

$$\mathcal{L}_{\text{cfm}}(\theta) = \mathbb{E}_{t\sim\text{Unif}, z\sim p_{\text{data}}, \epsilon\sim\mathcal{N}(0, I_d)} \left[\|u_t^\theta(tz + (1-t)\epsilon) - (z - \epsilon)\|^2 \right].$$

许多著名的先进模型使用这种简单而有效的过程训练，例如 *Stable Diffusion 3*、*Meta* 的 *Movie Gen Video*，以及很可能还有许多专有模型。图 7 将其可视化为一个简单例子，算法 3 总结了训练过程。

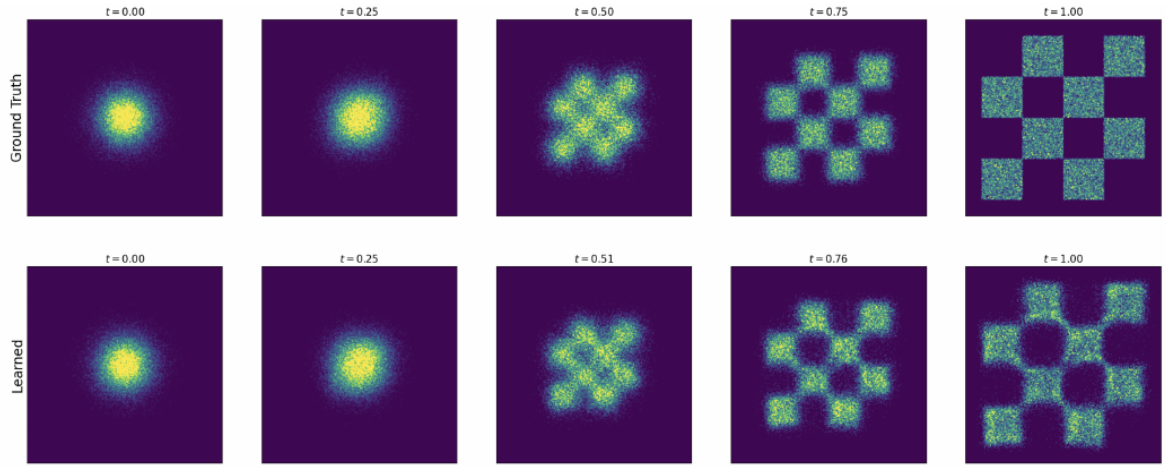


图 7: 高斯 CondOT 概率路径下的定理 12 示意：模拟经过流匹配训练的模型对应的 ODE。数据分布为棋盘图案（右上）。上排是真实边缘概率路径 $p_t(x)$ 的直方图，下排是流匹配模型样本的直方图。训练后，上下两排在训练误差范围内匹配。模型使用算法 3 训练。图中 Ground Truth / Learned 分别表示真实分布 / 学得的分布。译注：原图上排时间为 $0, 0.25, 0.50, 0.75, 1.00$，下排为 $0, 0.25, 0.51, 0.76, 1.00$。

下面总结本节结果。

总结 14 (流匹配)**.** 流匹配训练由学习边缘向量场 u_t^{target} 构成。为构造它，先选择满足 $p_0(\cdot \mid z) = p_{\text{init}}$、$p_1(\cdot \mid z) = \delta_z$ 的条件概率路径 $p_t(x \mid z)$。接着找到条件向量场 $u_t^{\text{target}}(x \mid z)$，使其对应的流 $\psi_t^{\text{target}}(x \mid z)$ 满足

$$X_0 \sim p_{\text{init}} \quad \Longrightarrow \quad X_t = \psi_t^{\text{target}}(X_0 \mid z) \sim p_t(\cdot \mid z),$$

或者等价地使 u_t^{target} 满足连续性方程。那么由下式定义的边缘向量场

$$u_t^{\text{target}}(x) = \int u_t^{\text{target}}(x \mid z) \frac{p_t(x \mid z) p_{\text{data}}(z)}{p_t(x)} \, dz \tag{32}$$

遵循边缘概率路径，即

$$X_0 \sim p_{\text{init}}, \quad dX_t = u_t^{\text{target}}(X_t) dt \quad \Longrightarrow \quad X_t \sim p_t \quad (0 \le t \le 1). \tag{33}$$

特别地，此 *ODE* 满足 $X_1 \sim p_{\text{data}}$，因此 u_t^{target} 如所期望地“将噪声转换为数据”。为学习它，我们最小化条件流匹配损失

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t \sim \text{Unif}, z \sim p_{\text{data}}, x \sim p_t(\cdot \mid z)} \left[\| u_t^\theta(x) - u_t^{\text{target}}(x \mid z) \|^2 \right]. \tag{34}$$

最广泛使用的例子是高斯概率路径。此时公式为

$$p_t(x \mid z) = \mathcal{N}(x; \alpha_t z, \beta_t^2 I_d), \tag{35}$$

$$u_t^{\text{flow}}(x \mid z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t}\alpha_t\right) z + \frac{\dot{\beta}_t}{\beta_t} x, \tag{36}$$

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t\sim\text{Unif}, z\sim p_{\text{data}}, \epsilon\sim\mathcal{N}(0,I_d)}\left[\|u_t^\theta(\alpha_t z + \beta_t \epsilon) - (\dot{\alpha}_t z + \dot{\beta}_t \epsilon)\|^2\right], \tag{37}$$

其中噪声调度函数 $\alpha_t, \beta_t \in \mathbb{R}$ 是连续可微单调函数，选择其满足 $\alpha_0 = \beta_1 = 0$、$\alpha_1 = \beta_0 = 1$（例如 $\alpha_t = t, \beta_t = 1 - t$）。

4 得分函数与得分匹配

上一节我们展示了如何用流匹配训练流模型。本节讨论扩散模型，并展示如何使用得分匹配训练它们。

4.1 条件得分函数与边缘得分函数

迄今为止，我们研究的核心对象是向量场 $u_t(x)$。扩散模型 [45, 44] 采用不同视角，聚焦于得分函数。因此本节将用得分函数的语言重新表述目前学到的内容，提供一种新视角。

设 $q(x)$ 是任意概率分布。则 q 的得分函数定义为 $\nabla \log q(x)$，即 q 的对数似然关于 x 的梯度。得分具有直观含义：$\nabla \log q(x)$ 是对数似然增长最快的方向。图 8 展示了这一点。回到第

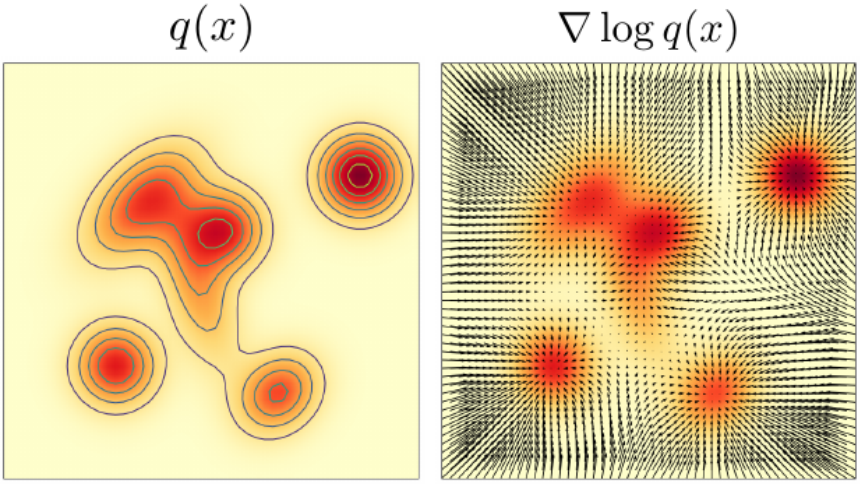


图 8: 一般概率分布 $q(x)$（左）的得分函数 $\nabla \log q(x)$，右图用黑色箭头绘出。

三节的条件概率路径 $p_t(x \mid z)$ 和边缘概率路径 $p_t(x)$。我们可以相应地将条件得分函数定义为 $\nabla \log p_t(x \mid z)$，将边缘得分函数定义为 $\nabla \log p_t(x)$。类似于式 (18)，边缘得分可通过条件得分函数 $\nabla \log p_t(x \mid z)$ 表达：

$$\nabla \log p_t(x) = \int \nabla \log p_t(x \mid z) \frac{p_t(x \mid z) p_{\text{data}}(z)}{p_t(x)}\, dz. \tag{38}$$

因此，条件得分与边缘得分之间的关系类似于条件向量场与边缘向量场之间的关系。注意，可通过下式证明式 (38)：

$$\begin{aligned}\nabla \log p_t(x) &= \frac{\nabla p_t(x)}{p_t(x)} = \frac{\nabla \int p_t(x \mid z) p_{\text{data}}(z)\, dz}{p_t(x)} = \frac{\int \nabla p_t(x \mid z) p_{\text{data}}(z)\, dz}{p_t(x)} \\ &= \int \nabla \log p_t(x \mid z) \frac{p_t(x \mid z) p_{\text{data}}(z)}{p_t(x)}\, dz, \end{aligned} \tag{39}$$

其中两次使用了规则 $\partial_y \log y = 1/y$ 与链式法则。

例 15 (高斯概率路径的得分函数)**.** 对于高斯路径 $p_t(x \mid z) = \mathcal{N}(x; \alpha_t z, \beta_t^2 I_d)$，利用高斯概率密度的形式（见式 (97)）得到

$$\nabla \log p_t(x \mid z) = \nabla \log \mathcal{N}(x; \alpha_t z, \beta_t^2 I_d) = -\frac{x - \alpha_t z}{\beta_t^2}. \tag{40}$$

注意，高斯概率路径的得分函数是 x 与 z 的线性函数，条件向量场 $u_t(x \mid z)$ 也是如此（见式 (20)）。因此两者之间可以互相转换，如下一个命题所示。

命题 1 (高斯概率路径的转换公式)**.** 对于高斯概率路径 $p_t(x \mid z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$，条件（分别为边缘）向量场与条件（分别为边缘）得分由下列等式联系：

$$u_t^{\text{target}}(x \mid z) = a_t \nabla \log p_t(x \mid z) + b_t x, \qquad a_t = \left(\beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t\right), \qquad b_t = \frac{\dot{\alpha}_t}{\alpha_t}, \tag{41}$$

$$u_t^{\text{target}}(x) = a_t \nabla \log p_t(x) + b_t x. \tag{42}$$

特别地，我们注意到可以由条件（分别为边缘）得分恢复条件（分别为边缘）向量场，反之亦然。

证明. 对条件向量场和条件得分，我们可以推导：

$$\begin{aligned} u_t^{\text{target}}(x \mid z) &= \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t}\alpha_t\right) z + \frac{\dot{\beta}_t}{\beta_t} x \\ &\overset{(i)}{=} \left(\beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t\right) \left(\frac{\alpha_t z - x}{\beta_t^2}\right) + \frac{\dot{\alpha}_t}{\alpha_t} x \\ &= \left(\beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t\right) \nabla \log p_t(x \mid z) + \frac{\dot{\alpha}_t}{\alpha_t} x, \end{aligned}$$

其中 (i) 只进行了代数整理。通过取积分，同一恒等式也适用于边缘流向量场与边缘得分函数：

$$\begin{aligned} u_t^{\text{target}}(x) &= \int u_t^{\text{target}}(x \mid z) \frac{p_t(x \mid z) p_{\text{data}}(z)}{p_t(x)}\, dz \\ &= \int [a_t \nabla \log p_t(x \mid z) + b_t x] \frac{p_t(x \mid z) p_{\text{data}}(z)}{p_t(x)}\, dz \overset{(i)}{=} a_t \nabla \log p_t(x) + b_t x, \end{aligned}$$

其中 (i) 使用式 (38) 以及后验密度积分为 1 的事实。 □

命题 1 引人注目，因为它表明一旦学得 u_t^{target}，我们也就学得了得分函数 $\nabla \log p_t(x)$，反之亦然。因此，许多扩散模型转而通过神经网络学习得分函数 $\nabla \log p_t(x)$。我们将在第 4.3 节讨论这一点。

注 16 (得分的重新参数化)**.** 式 (41) 中针对高斯概率路径的重新参数化公式之所以可行，是因为两边（条件向量场和条件得分）都是 x, z 的线性函数。边缘化后，两边（边缘向量场和边缘得分）都只是后验均值 $\mathbb{E}_{z|x}[z]$ 的线性重新参数化。因此，任何能够恢复 $\mathbb{E}_{z|x}[z]$ 的量，都能进一步用于恢复无条件向量场和得分。此外，从数值 / 训练稳定性的角度看，这样做甚至可能更好。

一种常见选择是后验均值本身，通常称为去噪器。形式上，条件和边缘去噪器定义为

$$D_t(x \mid z) = z, \qquad D_t(x) = \int z \frac{p_t(x \mid z) p_{\text{data}}(z)}{p_t(x)}\, dz \overset{(i)}{=} \frac{\beta_t u_t^{\text{target}}(x_t) - \dot{\beta}_t x_t}{\dot{\alpha}_t \beta_t - \alpha_t \dot{\beta}_t}. \tag{43}$$

这里 (i) 可由与命题 *1* 等价的推导得到。去噪器具有非常直观的解释：给定带噪数据 x，它是干净数据 z 的期望值。[4]人们通常称这类模型为去噪扩散模型，因为学习 D_t 与学习 u_t^{target} 在理论上等价。

4.2 使用 SDE 采样

到目前为止，我们已经展示了如何通过边缘向量场 u_t^{target} 构造遵循期望概率路径 p_t 的 ODE 轨迹 X_t。但这种方法仅限于流模型。扩散模型又如何？现在利用得分函数将这一结果扩展到 SDE。

定理 17 (SDE 扩展技巧)**.** 如前定义条件向量场 $u_t^{\text{target}}(x \mid z)$ 和边缘向量场 $u_t^{\text{target}}(x)$。那么，对任意扩散系数 $\sigma_t \geq 0$，我们可以通过向原 *ODE* 的动力学添加随机动力学构造如下 *SDE*:

$$X_0 \sim p_{\text{init}}, \qquad dX_t = u_t^{\text{target}}(X_t)dt + \frac{\sigma_t^2}{2} \nabla \log p_t(X_t) dt + \sigma_t\, dW_t \tag{44}$$

$$= \left[u_t^{\text{target}}(X_t) + \frac{\sigma_t^2}{2} \nabla \log p_t(X_t) \right] dt + \sigma_t\, dW_t$$

$$\implies \quad X_t \sim p_t \quad (0 \leq t \leq 1). \tag{45}$$

特别地，此 *SDE* 有 $X_1 \sim p_{\text{data}}$。我们注意到随机动力学与 *Langevin* 动力学密切相关，可以将其理解为在保持边缘分布 p_t 的同时注入噪声。关于 *Langevin* 动力学将在注 *20* 中简要讨论。

图 9 展示了定理 17 描述的动力学。可以看到轨迹如今呈锯齿状，说明 SDE 演化的随机性质。然而，如定理 17 所建立的，边缘分布 p_t 保持不变。值得注意的是，即便在网络训练之后，我们仍可选择任意扩散系数 $\sigma_t \geq 0$。理论上，定理对任意 σ_t 的选择都成立。然而在实践中，我们既有训练误差（神经网络并未完美近似边缘向量场和得分），也有模拟误差（例如对于 $\sigma_t \gg 0$，算法 2 需要小到代价难以承受的步长）。实践中，对固定的已训练模型，存在可以通过经验确定的最优 $\sigma_t \geq 0$ [23, 1, 28]。[5]

对于高斯概率路径，学习边缘向量场的同时就能免费得到得分函数。

例 18 (高斯 SDE 扩展技巧)**.** 根据命题 *1*，对于高斯概率路径，可以仅用得分函数表达定理 *17* 中的 *SDE*:

$$X_0 \sim p_{\text{init}}, \qquad dX_t = \left[\left(a_t + \frac{\sigma_t^2}{2} \right) \nabla \log p_t(X_t) + b_t X_t \right] dt + \sigma_t\, dW_t \tag{46}$$

$$\implies \quad X_t \sim p_t \quad (0 \leq t \leq 1), \tag{47}$$

[4]思考题：去噪器是否总会输出一个“干净”的数据点？为什么或为什么不？这可能取决于什么？

[5]再次强调，“最优 σ_t” 的存在是模型训练不完美和计算预算有限带来的结果，而非连续极限下动力学的理论断言。

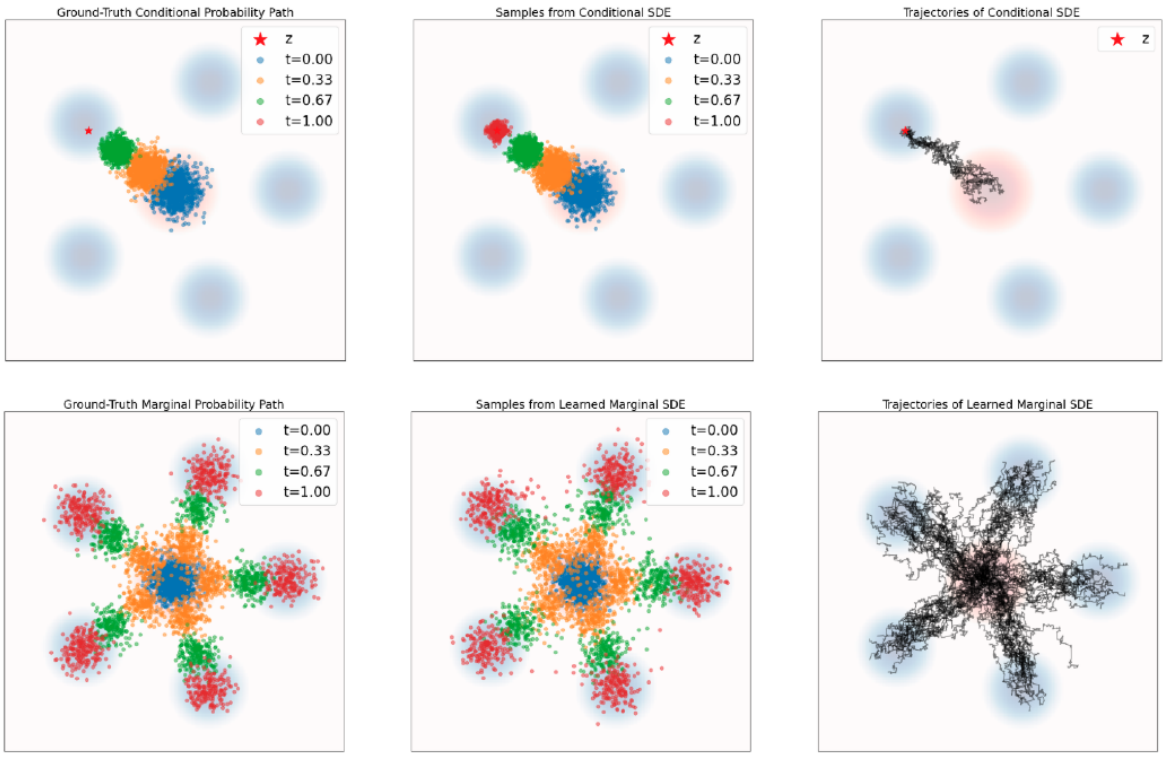


图 9: 定理 17 的示意：用 SDE 模拟概率路径。这里使用式 (44) 的 SDE 采样重复图 6 的绘图。蓝色背景为数据分布 p_{data}，红色背景为高斯分布 p_{init}。上排为条件路径，下排为边缘路径。SDE 将来自 p_{init} 的样本转换为来自 δ_z（条件路径）及 p_{data}（边缘路径）的样本。译注：Ground-Truth、Samples、Trajectories、Learned 分别表示真实分布、样本、轨迹、学得的；Conditional / Marginal 为条件 / 边缘，SDE 为随机微分方程。时间图例为 $t = 0, 0.33, 0.67, 1$。

其中 a_t, b_t 按命题 *1* 定义。

本节余下部分将通过 Fokker–Planck 方程证明定理 17，该方程将 ODE 的连续性方程扩展到 SDE。为此先定义 Laplace 算符 Δ：

$$\Delta w_t(x) = \sum_{i=1}^{d} \frac{\partial^2}{\partial x_i^2} w_t(x) = \operatorname{div}(\nabla w_t)(x), \tag{48}$$

其中 $w_t : \mathbb{R}^d \to \mathbb{R}$ 是标量场。

定理 19 (Fokker–Planck 方程)**.** *设 p_t 是概率路径，并考虑 SDE*

$$X_0 \sim p_{\text{init}}, \qquad dX_t = u_t(X_t)dt + \sigma_t\, dW_t.$$

那么，对所有 $0 \le t \le 1$，X_t 服从 p_t，当且仅当 Fokker–Planck 方程成立：

$$\partial_t p_t(x) = -\operatorname{div}(p_t u_t)(x) + \frac{\sigma_t^2}{2}\Delta p_t(x) \quad \text{对所有} x \in \mathbb{R}^d,\ 0 \le t \le 1. \tag{49}$$

附录 B 给出了 Fokker–Planck 方程的自包含证明。注意当 $\sigma_t = 0$ 时，Fokker–Planck 方程退化为定理 11。附加的 Laplace 项 Δp_t 起初可能难以理解。熟悉物理学的读者会注意到同一项也出现在热方程中（它实际上是 Fokker–Planck 方程的特殊情况）。热在介质中扩散；我们也加入了扩散过程（并非物理上的，而是数学上的），因此加入额外的 Laplace 项。现在利用 Fokker–Planck 方程证明定理 17。

定理 17 的证明. 由定理 19，我们需要证明式 (44) 定义的 SDE 对 p_t 满足 Fokker–Planck 方程。直接计算可得：

$$\begin{aligned}
\partial_t p_t(x) &\overset{(i)}{=} -\operatorname{div}(p_t u_t^{\text{target}})(x) \\
&\overset{(ii)}{=} -\operatorname{div}(p_t u_t^{\text{target}})(x) - \frac{\sigma_t^2}{2}\Delta p_t(x) + \frac{\sigma_t^2}{2}\Delta p_t(x) \\
&\overset{(iii)}{=} -\operatorname{div}(p_t u_t^{\text{target}})(x) - \operatorname{div}\left(\frac{\sigma_t^2}{2}\nabla p_t\right)(x) + \frac{\sigma_t^2}{2}\Delta p_t(x) \\
&\overset{(iv)}{=} -\operatorname{div}(p_t u_t^{\text{target}})(x) - \operatorname{div}\left(p_t\left[\frac{\sigma_t^2}{2}\nabla \log p_t\right]\right)(x) + \frac{\sigma_t^2}{2}\Delta p_t(x) \\
&\overset{(v)}{=} -\operatorname{div}\left(p_t\left[u_t^{\text{target}} + \frac{\sigma_t^2}{2}\nabla \log p_t\right]\right)(x) + \frac{\sigma_t^2}{2}\Delta p_t(x),
\end{aligned}$$

其中 (i) 使用定理 11；(ii) 加上并减去同一项；(iii) 使用 Laplace 算符的定义（式 (48)）；(iv) 使用 $\nabla \log p_t = \nabla p_t / p_t$；$(v)$ 使用散度算符的线性性。上述推导表明式 (44) 定义的 SDE 对 p_t 满足 Fokker–Planck 方程。由定理 19，这推出对 $0 \le t \le 1$ 有 $X_t \sim p_t$，正如所需。 □

注 20 (选学：Langevin 动力学)**.** *上述构造有一个著名的特殊情况：概率路径为常数，即某个固定分布 p 满足 $p_t = p$。此时取 $u_t^{\text{target}} = 0$，得到 SDE*

$$dX_t = \frac{\sigma_t^2}{2}\nabla \log p(X_t)dt + \sigma_t\, dW_t, \tag{50}$$

它通常称为 *Langevin* 动力学。p_t 为常数意味着 $\partial_t p_t(x) = 0$。由定理 *17* 立即可知这些动力学对定理中的静态路径 $p_t = p$ 满足 *Fokker–Planck* 方程。因此可得 p 是 *Langevin* 动力学的平稳分布：

$$X_0 \sim p \quad \Longrightarrow \quad X_t \sim p \quad (t \geq 0).$$

如同许多 *Markov* 过程，在相当一般的条件下，这些动力学收敛到平稳分布 p。也就是说，如果改取 $X_0 \sim p' \neq p$，使 $X_t \sim p'_t$，则在较弱条件下 $p_t \to p$。这一事实使 *Langevin* 动力学极为有用，因此它构成例如分子动力学模拟，以及贝叶斯统计和自然科学中许多其他 *Markov* 链 *Monte Carlo*（*MCMC*）方法的基础。特别地，当 p 为高斯分布时，恢复作为 *Langevin* 动力学特殊情况的 *Ornstein–Uhlenbeck* 过程，而它们构成扩散模型初始表述的基础。

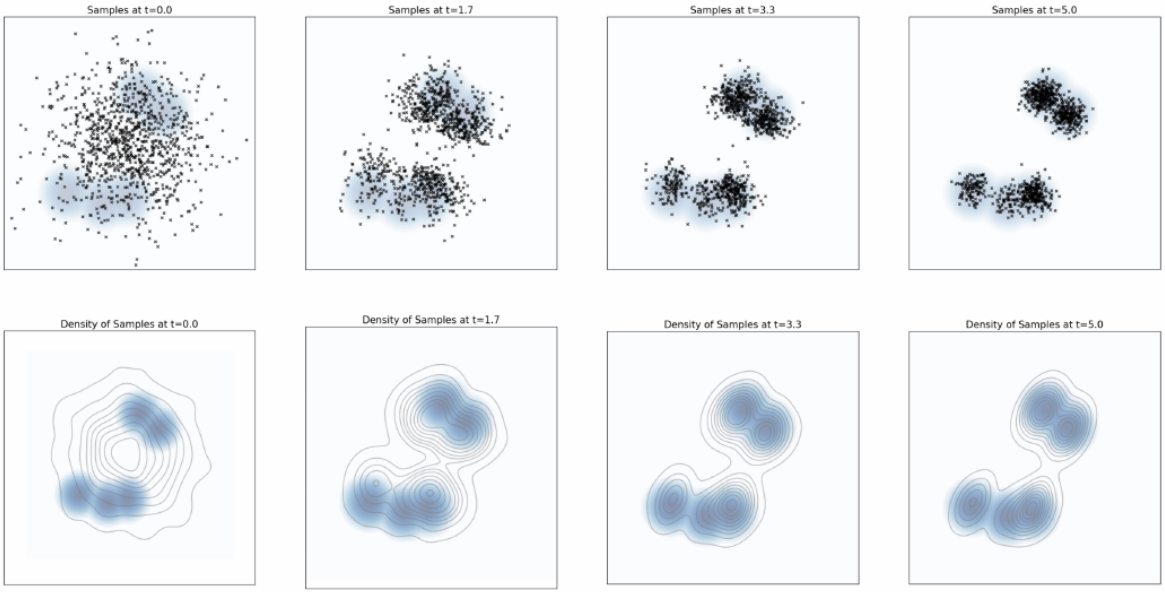


图 10: 上排：按照式 (50) 给出的 Langevin 动力学演化的粒子，其中 $p(x)$ 取为具有五个模态的高斯混合分布。下排：对上排所示同一样本的核密度估计。样本分布收敛到平衡分布 p（蓝色背景）。译注：Samples at / Density of Samples at 分别为该时刻的样本 / 样本密度；四列时间为 $t = 0, 1.7, 3.3, 5.0$。

注 21 (选学：GLASS Flows，使用 ODE 的随机演化)**.** 与 *ODE* 相比，*SDE* 采样的显著性质是演化变为随机的，即初始点 X_0 不再完全决定 $t > 0$ 的 X_t。也许令人惊讶，通过一种称为 *GLASS Flows [20]* 的简单采样技巧，也可以仅通过 *ODE* 获得相同的随机跃迁。这使我们能够在保持 *ODE* 效率的同时利用 *SDE* 的随机性质（例如通过搜索算法）。

4.3 得分匹配

剩下的问题是如何学习边缘得分函数 $\nabla \log p_t(x)$。当然，对高斯概率路径，我们可以通过命题 1 直接变换 $u_t^{\text{target}}(x)$。但一般情形又如何? 事实证明，我们也可以直接学习边缘得分函数。为近似边缘得分 $\nabla \log p_t$，使用称为得分网络的神经网络 $s_t^\theta : \mathbb{R}^d \times [0,1] \to \mathbb{R}^d$。与前面相同，我们

可以设计得分匹配损失和去噪得分匹配损失：

$$\mathcal{L}_{\mathrm{SM}}(\theta) = \mathbb{E}_{t\sim\mathrm{Unif},z\sim p_{\mathrm{data}},x\sim p_t(\cdot|z)}\left[\|s_t^\theta(x) - \nabla \log p_t(x)\|^2\right] \qquad \text{得分匹配损失,}$$
$$\mathcal{L}_{\mathrm{CSM}}(\theta) = \mathbb{E}_{t\sim\mathrm{Unif},z\sim p_{\mathrm{data}},x\sim p_t(\cdot|z)}\left[\|s_t^\theta(x) - \nabla \log p_t(x \mid z)\|^2\right] \qquad \text{条件得分匹配损失,}$$

区别仍在于使用边缘得分 $\nabla \log p_t(x)$ 还是条件得分 $\nabla \log p_t(x \mid z)$。如前，理想中我们希望最小化得分匹配损失，但由于不知道 $\nabla \log p_t(x)$ 而无法做到。同样地，去噪得分匹配损失是一个可计算的替代方案：

定理 22. 得分匹配损失与去噪得分匹配损失仅相差一个常数：

$$\mathcal{L}_{\mathrm{SM}}(\theta) = \mathcal{L}_{\mathrm{CSM}}(\theta) + C,$$

其中 C 与参数 θ 无关。因此，它们的梯度相同：

$$\nabla_\theta \mathcal{L}_{\mathrm{SM}}(\theta) = \nabla_\theta \mathcal{L}_{\mathrm{CSM}}(\theta).$$

特别地，对最小化解 θ^*，有 $s_t^{\theta^*} = \nabla \log p_t$。

证明. 注意 $\nabla \log p_t$ 的公式（式 (38)）与 u_t^{target} 的公式（式 (18)）形式相同。因此，将 u_t^{target} 替换为 $\nabla \log p_t$ 后，证明与定理 12 的证明相同。 □

例 23 (去噪扩散模型：高斯概率路径的得分匹配). 将去噪得分匹配损失具体化到 $p_t(x \mid z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$ 的情形。如式 (40) 所推导，条件得分 $\nabla \log p_t(x \mid z)$ 的公式为

$$\nabla \log p_t(x \mid z) = -\frac{x - \alpha_t z}{\beta_t^2}. \tag{51}$$

代入该公式，条件得分匹配损失变为

$$\begin{aligned}
\mathcal{L}_{\mathrm{CSM}}(\theta) &= \mathbb{E}_{t\sim\mathrm{Unif},z\sim p_{\mathrm{data}},x\sim p_t(\cdot|z)}\left[\left\|s_t^\theta(x) + \frac{x - \alpha_t z}{\beta_t^2}\right\|^2\right] \\
&\overset{(i)}{=} \mathbb{E}_{t\sim\mathrm{Unif},z\sim p_{\mathrm{data}},\epsilon\sim\mathcal{N}(0,I_d)}\left[\left\|s_t^\theta(\alpha_t z + \beta_t \epsilon) + \frac{\epsilon}{\beta_t}\right\|^2\right] \\
&= \mathbb{E}_{t\sim\mathrm{Unif},z\sim p_{\mathrm{data}},\epsilon\sim\mathcal{N}(0,I_d)}\left[\frac{1}{\beta_t^2}\left\|\beta_t s_t^\theta(\alpha_t z + \beta_t \epsilon) + \epsilon\right\|^2\right],
\end{aligned}$$

其中 (i) 代入式 (28)，将 x 换为 $\alpha_t z + \beta_t \epsilon$。注意网络 s_t^θ 本质上学会预测用于污染数据样本 z 的噪声。这解释了为何上述训练损失称为去噪得分匹配。人们很快意识到，当 $\beta_t \approx 0$ 接近零时，上述损失在数值上不稳定（也就是说，只有加入足够的噪声，去噪得分匹配才有效）。因此，在最早的一些去噪扩散模型工作中（见去噪扩散概率模型 *[17]*），提出去掉损失中的常数 $1/\beta_t^2$，[6]并通过下式将 s_t^θ 重新参数化为噪声预测网络 $\epsilon_t^\theta : \mathbb{R}^d \times [0,1] \to \mathbb{R}^d$：

$$-\beta_t s_t^\theta(x) = \epsilon_t^\theta(x) \quad \Longrightarrow \quad \mathcal{L}_{\mathrm{DDPM}}(\theta) = \mathbb{E}_{t\sim\mathrm{Unif},z\sim p_{\mathrm{data}},\epsilon\sim\mathcal{N}(0,I_d)}\left[\|\epsilon_t^\theta(\alpha_t z + \beta_t \epsilon) - \epsilon\|^2\right].$$

和前面一样，网络 ϵ_t^θ 本质上学会预测用于污染数据样本 z 的噪声。算法 *4* 总结了训练过程。

[6]译注：该因子与模型参数无关，但通常随时间 t 变化；从时间期望中去掉它会改变训练的时间权重，并非只给总损失乘一个固定常数。

算法 4：高斯概率路径的得分匹配训练过程。 **输入：**样本 $z \sim p_{\text{data}}$ 构成的数据集，得分网络 s_t^θ 或噪声预测器 ϵ_t^θ。

1. 对每个数据小批量，从数据集采样一个数据例子 z。
2. 采样随机时间 $t \sim \text{Unif}_{[0,1]}$。
3. 采样噪声 $\epsilon \sim \mathcal{N}(0, I_d)$。
4. 令 $x_t = \alpha_t z + \beta_t \epsilon$（一般情形：$x_t \sim p_t(\cdot \mid z)$）。
5. 计算损失 $\mathcal{L}(\theta) = \|s_t^\theta(x_t) + \epsilon/\beta_t\|^2$（一般情形：$\|s_t^\theta(x_t) - \nabla \log p_t(x_t \mid z)\|^2$）；也可使用 $\mathcal{L}(\theta) = \|\epsilon_t^\theta(x_t) - \epsilon\|^2$。
6. 通过对 $\mathcal{L}(\theta)$ 进行梯度下降更新模型参数 θ；对每个小批量重复。

下面总结本节结果：

总结 24 (得分函数、得分匹配与随机采样)**.** 设 $p_t(x \mid z), p_t(x)$ 分别为条件和边缘概率路径。条件得分函数为 $\nabla \log p_t(x \mid z)$，边缘得分函数为 $\nabla \log p_t(x)$。对每个扩散系数 $\sigma_t \geq 0$，以下 *SDE* 的轨迹遵循概率路径：

$$X_0 \sim p_{\text{init}}, \quad dX_t = \left[u_t^{\text{target}}(X_t) + \frac{\sigma_t^2}{2} \nabla \log p_t(X_t) \right] dt + \sigma_t \, dW_t, \tag{52}$$

$$\implies \quad X_t \sim p_t \quad (0 \leq t \leq 1), \tag{53}$$

其中 $u_t^{\text{target}}(x)$ 仍为前述边缘向量场（见式 (18)）。

得分匹配。为学习边缘得分函数 $\nabla \log p_t(x)$，可以使用得分网络 s_t^θ，通过去噪得分匹配训练：

$$\mathcal{L}_{\text{CSM}}(\theta) = \mathbb{E}_{z \sim p_{\text{data}}, t \sim \text{Unif}, x \sim p_t(\cdot \mid z)} [\|s_t^\theta(x) - \nabla \log p_t(x \mid z)\|^2]. \tag{54}$$

高斯概率路径。对于最重要的高斯概率路径 $p_t(x \mid z) = \mathcal{N}(x; \alpha_t z, \beta_t^2 I_d)$，不必分别训练 s_t^θ 和 u_t^θ，因为可通过以下公式转换：

$$u_t^\theta(x) = a_t s_t^\theta(x) + b_t x, \qquad a_t = \left(\beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t \right), \qquad b_t = \frac{\dot{\alpha}_t}{\alpha_t}.$$

训练后，可针对任意扩散系数 $\sigma_t \geq 0$ 模拟以下 *SDE*，得到近似样本 $X_1 \sim p_{\text{data}}$：

$$X_0 \sim p_{\text{init}}, \quad dX_t = \left[\left(1 + \frac{\sigma_t^2}{2a_t} \right) u_t^\theta(X_t) - \frac{\sigma_t^2 b_t}{2a_t} X_t \right] dt + \sigma_t \, dW_t \tag{55}$$

$$= \left[\left(a_t + \frac{\sigma_t^2}{2} \right) s_t^\theta(X_t) + b_t X_t \right] dt + \sigma_t \, dW_t. \tag{56}$$

可以通过经验找到最优的 $\sigma_t \geq 0$。

5 引导：如何以提示词为条件

到目前为止，我们考虑的生成模型都是无引导的。例如，图像模型只是生成某张图像。数学上，这意味着模型返回来自无条件数据分布 $p_{\text{data}}(z)$ 的样本。然而在大多数情况下，我们的目标不只是生成任意对象，而是以某些额外信息为条件生成对象。换言之，我们希望引导模型生成某一类对象。例如，可以设想一个以文本提示 y 为输入，进而生成符合该文本提示 y 的图像 x 的图像生成模型。如第一节所述，这意味着我们希望从 $p_{\text{data}}(z \mid y)$ 中采样，即以 y 为条件的引导数据分布。本节将对此展开讨论。

注 25 (术语)**.** 为避免“条件”一词与以 $z \sim p_{\text{data}}$ 为条件（条件概率路径 / 向量场）的记号和术语冲突，我们使用引导专门表示以文本提示等 y 为条件。

5.1 基本引导

首先讨论构建引导生成模型的“标准”方法。简短回答如下：只需在训练和推断期间将输入提示 y 提供给网络，其余操作与之前完全相同。下面将其形式化。设条件变量或提示 y 属于某个空间 $\mathcal{Y}$。例如，当 y 是文本提示时，$\mathcal{Y}$ 是所有文本的空间；当 y 是某个离散类别标签时，$\mathcal{Y}$ 为离散空间。我们对 $\mathcal{Y}$ 不施加约束。

定义引导扩散模型由一个神经网络参数化的引导向量场 $u_t^\theta(\cdot \mid y)$ 和一个含时扩散系数 σ_t 构成，共同写为

$$\begin{aligned} \text{神经网络：} \quad & u^\theta : \mathbb{R}^d \times \mathcal{Y} \times [0,1] \to \mathbb{R}^d, \quad (x, y, t) \mapsto u_t^\theta(x \mid y), \\ \text{固定：} \quad & \sigma : [0,1] \to [0, \infty), \quad t \mapsto \sigma_t. \end{aligned}$$

注意其与总结 7 的区别：这里额外用输入 $y \in \mathcal{Y}$ 引导 u_t^θ。对于任意此类 $y \in \mathcal{Y}$，可按以下方式从模型生成样本：

$$\begin{aligned} \text{初始化：} \quad & X_0 \sim p_{\text{init}} && \text{用简单分布初始化（如高斯分布），} \\ \text{模拟：} \quad & dX_t = u_t^\theta(X_t \mid y)dt + \sigma_t \, dW_t && \text{从} t = 0 \text{ 到} t = 1 \text{ 模拟 SDE,} \\ \text{目标：} \quad & X_1 \sim p_{\text{data}}(\cdot \mid y) && \text{目标是使} X_1 \text{ 服从} p_{\text{data}}(\cdot \mid y). \end{aligned}$$

当 $\sigma_t = 0$ 时，称这样的模型为引导流模型。下文为简洁起见限制在流匹配和流模型，但一切都可类似地应用于一般情形。

接下来讨论：如何训练引导流模型 $u_t^\theta(x \mid y)$？一个简单方法是固定 y 的选择，并将数据分布取为 $p_{\text{data}}(x \mid y)$。这样就恢复了先前的无引导生成问题，因此可以用条件流匹配目标构造生成模型，即

$$\mathbb{E}_{z \sim p_{\text{data}}(\cdot \mid y), x \sim p_t(\cdot \mid z)} \| u_t^\theta(x \mid y) - u_t^{\text{target}}(x \mid z) \|^2. \tag{57}$$

注意标签 y 不影响条件概率路径 $p_t(\cdot \mid z)$ 或条件向量场 $u_t^{\text{target}}(x \mid z)$（尽管原则上可以让它们依赖于 y）。对所有此类 y 的选择展开期望，便得到引导条件流匹配目标：

$$\mathcal{L}_{\text{CFM}}^{\text{guided}}(\theta) = \mathbb{E}_{(z,y) \sim p_{\text{data}}(z,y),\, t \sim \text{Unif}_{[0,1]},\, x \sim p_t(\cdot \mid z)} \| u_t^\theta(x \mid y) - u_t^{\text{target}}(x \mid z) \|^2. \tag{58}$$

式 (58) 的引导目标与式 (26) 的无引导目标的主要差别之一，是这里采样 $(z, y) \sim p_{\text{data}}$，而非只采样 $z \sim p_{\text{data}}$。原因是现在的数据分布原则上是例如图像 z 和文本提示 y 的联合分布。在实践中，这意味着式 (58) 的 PyTorch 实现会使用一个同时返回 z 和 y 批量的数据加载器。

5.2 无分类器引导

理论上，基本引导应产生忠实于 $p_{\text{data}}(\cdot \mid y)$ 的生成过程。然而，人们很快在经验上发现，以这种方式采样的图像与期望标签 y 的符合程度不够（见图 11）。原因可能多种多样：模型可能欠拟合（即我们实际上没有学到真正的边缘向量场），或数据可能不完美（例如，来自互联网的文本—图像对包含许多错误）。因此，要真正生成更符合提示的样本，必须想办法人为增强提示变量 y。主要技术称为*无分类器引导*，它广泛用于当前先进的扩散模型，下面将进行讨论。



图 11：以提示 / 类别 $y =$ “柯基犬”生成图像。左：基本引导生成的样本，与提示符合得不够好。右：分类器引导且 $w = 4$ 时生成的样本。无分类器引导提升了对提示的遵循程度。图取自文献 [18]。

分类器引导。 为简单起见，这里重点考虑高斯概率路径。回顾式 (15)，高斯条件概率路径为 $p_t(\cdot \mid z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$，其中噪声调度函数 α_t, β_t 连续可微、单调，并满足 $\alpha_0 = \beta_1 = 0$、$\alpha_1 = \beta_0 = 1$。另回顾可以使用命题 1，用引导得分函数 $\nabla \log p_t(x \mid y)$ 将引导向量场 $u_t^{\text{target}}(x \mid y)$ 改写为

$$u_t^{\text{target}}(x \mid y) = a_t \nabla \log p_t(x \mid y) + b_t x. \tag{59}$$

接下来注意 $p_t(x \mid y)$ 是条件密度，因此可以使用 Bayes 法则改写引导得分：

$$p_t(x \mid y) = \frac{p_t(x) p_t(y \mid x)}{p_t(y)}, \tag{60}$$

$$\nabla \log p_t(x \mid y) = \nabla \log \left(\frac{p_t(x) p_t(y \mid x)}{p_t(y)} \right) = \nabla \log p_t(x) + \nabla \log p_t(y \mid x), \tag{61}$$

其中使用了梯度 ∇ 关于变量 x 计算，因此 $\nabla \log p_t(y) = 0$。于是可改写为

$$u_t^{\text{target}}(x \mid y) = b_t x + a_t \big(\nabla \log p_t(x) + \nabla \log p_t(y \mid x) \big) = u_t^{\text{target}}(x) + a_t \nabla \log p_t(y \mid x).$$

注意上述等式的形式：引导向量场 $u_t^{\text{target}}(x \mid y)$ 是无引导向量场 $u_t^{\text{target}}(x)$ 与引导变量 y 的似然 $p_t(y \mid x)$ 的梯度之和。由于人们观察到图像 x 与提示 y 的符合程度不够，自然的想法是放大 $\nabla \log p_t(y \mid x)$ 项的贡献，得到

$$\widetilde{u}_t(x \mid y) = u_t^{\text{target}}(x) + w a_t \nabla \log p_t(y \mid x) \qquad \text{（分类器引导），} \tag{62}$$

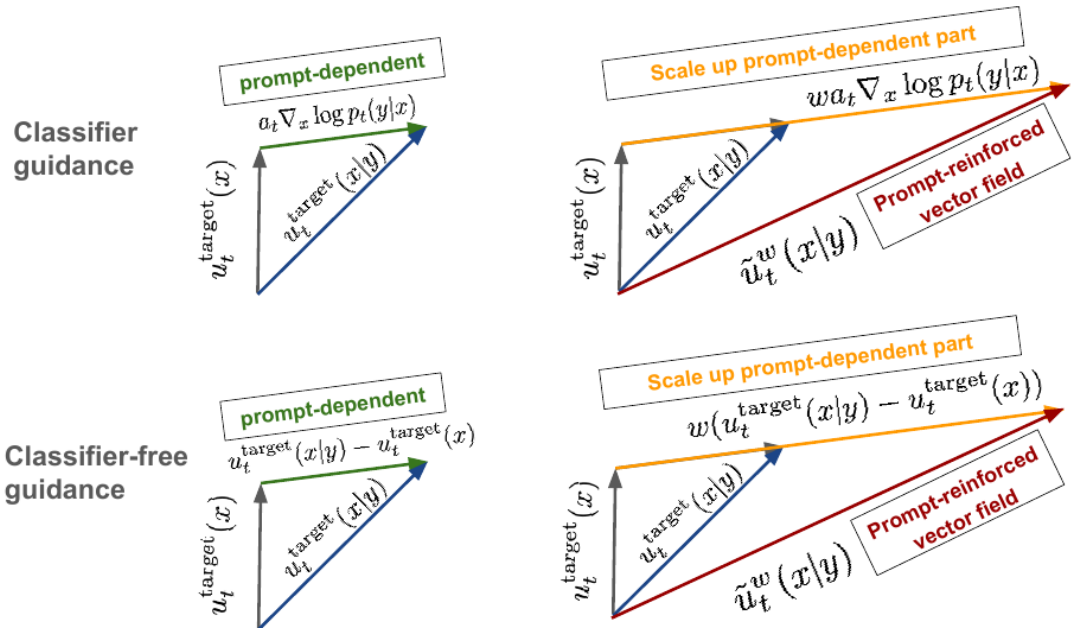


图 12: 分类器引导与无分类器引导示意。分类器引导将引导向量场 $u_t^{\text{target}}(x \mid y)$ 分解，分离出分类器的对数概率梯度 $\nabla_x \log p_t(y \mid x)$ 对应的项，并以引导尺度 $w > 1$ 放大分类器项。无分类器引导放大两个向量场之差，从而在不训练独立分类器模型的情况下达到相同效果。图中 prompt-dependent、scale up prompt-dependent part 和 prompt-reinforced vector field 分别表示依赖提示、放大依赖提示的部分和提示增强的向量场。

其中 $w > 1$ 称为*引导尺度*。如何学习 $\log p_t(y \mid x)$？注意可以将它看作带噪数据的某种分类器（即它给出给定 x 下 y 的对数似然），因此可以直接通过监督学习来学习它。这就得到分类器引导 [11, 43]（示意见图 12）。

分类器引导在很大程度上已被无分类器引导取代，因此这里不再进一步讨论。然而，它是无分类器引导的基础，下面即可看到。最后注意，这是一种启发式方法：对于 $w \neq 1$，有 $\widetilde{u}_t(x \mid y) \neq u_t^{\text{target}}(x \mid y)$，因此它不是“真正的”引导向量场。

无分类器引导。 虽然分类器引导原则上可行，但也有困难：首先，我们需要在流 / 扩散模型之外再训练一个分类器，因此有两个网络而非一个。此外，如果 y 是高维对象，例如文本提示而不只是类别，那么 $p_t(y \mid x)$ 可能非常难学，其梯度 $\nabla \log p_t(y \mid x)$ 也难以得到。因此，人们提出了无分类器引导 [18]。无分类器引导在理论上达到与分类器引导等价的效果，但不必训练独立分类器。

为此，再次应用等式

$$\nabla \log p_t(x \mid y) = \nabla \log p_t(x) + \nabla \log p_t(y \mid x),$$

得到

$$\begin{aligned}\widetilde{u}_t(x \mid y) &= u_t^{\text{target}}(x) + w a_t \nabla \log p_t(y \mid x) \\ &= u_t^{\text{target}}(x) + w a_t \big(\nabla \log p_t(x \mid y) - \nabla \log p_t(x)\big) \\ &= u_t^{\text{target}}(x) - \big(w b_t x + w a_t \nabla \log p_t(x)\big) + \big(w b_t x + w a_t \nabla \log p_t(x \mid y)\big) \\ &= (1-w) u_t^{\text{target}}(x) + w u_t^{\text{target}}(x \mid y).\end{aligned}$$

因此可将缩放后的引导向量场 $\widetilde{u}_t(x \mid y)$ 表示为两个向量场的线性组合：无引导的 $u_t^{\text{target}}(x)$ 和引导的 $u_t^{\text{target}}(x \mid y)$。由此可以考虑同时训练无引导的 $u_t^{\text{target}}(x)$（例如使用式 (26)）及引导的 $u_t^{\text{target}}(x \mid y)$（例如使用式 (58)），然后在推断时组合它们以得到 $\widetilde{u}_t(x \mid y)$。“等等！”你可能会问，“那不是需要训练两个模型吗!?”事实证明，我们可以在一个模型中同时训练两者：为标签集合增加一个新的 $\varnothing$ 标签，表示没有条件。于是可以令 $u_t^{\text{target}}(x) = u_t^{\text{target}}(x \mid \varnothing)$。这样就不需要训练独立模型来增强假想分类器的效果。这种在一个模型中同时训练条件与无条件模型（随后加强条件）的方式称为*无分类器引导*（CFG） [18]，见图 12。

注 26 (一般概率路径的推导)**.** *注意构造*

$$\widetilde{u}_t(x \mid y) = (1-w) u_t^{\text{target}}(x) + w u_t^{\text{target}}(x \mid y)$$

对任何概率路径的选择都同样有效，不仅限于高斯路径。当 $w = 1$ 时，可以直接验证 $\widetilde{u}_t(x \mid y) = u_t^{\text{target}}(x \mid y)$。我们用高斯路径推导只是为了说明这一构造的直觉，特别是放大假想“分类器” $\nabla \log p_t(y \mid x)$ 的贡献。

训练与无分类器引导。 现在必须修改式 (58) 的引导条件流匹配目标，以考虑 $y = \varnothing$ 的可能性。困难在于，从 $(z, y) \sim p_{\text{data}}$ 采样时永远不会得到 $y = \varnothing$。因此必须人为引入 $y = \varnothing$ 的可能性。为此定义超参数 η 为丢弃原始标签 y 并用 $\varnothing$ 替换的概率。于是得到 CFG 条件流匹配训练目标：

$$\mathcal{L}_{\text{CFM}}^{\text{CFG}}(\theta) = \mathbb{E}_{\square} \| u_t^\theta(x \mid y) - u_t^{\text{target}}(x \mid z) \|^2, \tag{63}$$

$$\square = (z, y) \sim p_{\text{data}}(z, y), \quad t \sim \text{Unif}[0, 1], \quad x \sim p_t(\cdot \mid z), \quad \text{以概率} \eta \text{ 将} y = \varnothing. \tag{64}$$

算法 5：高斯概率路径 $p_t(x \mid z) = \mathcal{N}(x; \alpha_t z, \beta_t^2 I_d)$ 的无分类器引导训练。 **输入：** 成对数据集 $(z, y) \sim p_{\text{data}}$，神经网络 u_t^θ。

1. 对每个数据小批量，从数据集中采样一个数据例子 (z, y)。
2. 采样随机时间 $t \sim \text{Unif}[0, 1]$。
3. 采样噪声 $\epsilon \sim \mathcal{N}(0, I_d)$。
4. 令 $x = \alpha_t z + \beta_t \epsilon$。
5. 以概率 p 丢弃标签：$y \leftarrow \varnothing$。
6. 计算损失 $\mathcal{L}(\theta) = \| u_t^\theta(x \mid y) - (\dot{\alpha}_t z + \dot{\beta}_t \epsilon) \|^2$。
7. 通过对 $\mathcal{L}(\theta)$ 进行梯度下降更新模型参数 θ；对每个小批量重复。

下面总结我们的结果。

总结 27 (流模型的无分类器引导)**.** 给定无引导边缘向量场 $u_t^{\text{target}}(x \mid \varnothing)$、引导边缘向量场 $u_t^{\text{target}}(x \mid y)$ 和引导尺度 $w > 1$，定义无分类器引导向量场 $\widetilde{u}_t(x \mid y)$ 为

$$\widetilde{u}_t(x \mid y) = (1-w)u_t^{\text{target}}(x \mid \varnothing) + wu_t^{\text{target}}(x \mid y). \tag{65}$$

通过用同一个神经网络近似 $u_t^{\text{target}}(x \mid \varnothing)$ 和 $u_t^{\text{target}}(x \mid y)$，我们可以使用以下无分类器引导 *CFM*（*CFG-CFM*）目标：

$$\mathcal{L}_{\text{CFM}}^{\text{CFG}}(\theta) = \mathbb{E}_{\square}\|u_t^\theta(x \mid y) - u_t^{\text{target}}(x \mid z)\|^2, \tag{66}$$

$$\square = (z,y) \sim p_{\text{data}}(z,y), \quad t \sim \text{Unif}[0,1], \quad x \sim p_t(\cdot \mid z), \quad \text{以概率}\eta \text{ 将}y = \varnothing. \tag{67}$$

用通俗的语言，$\mathcal{L}_{\text{CFM}}^{\text{CFG}}$ 可按以下过程近似：

$$\begin{aligned}
(z,y) &\sim p_{\text{data}}(z,y) && \text{从数据分布采样}(z,y),\\
t &\sim \text{Unif}[0,1) && \text{在}[0,1) \text{ 上均匀采样}t,\\
x &\sim p_t(x \mid z) && \text{从条件概率路径}p_t(x \mid z) \text{ 采样}x,\\
\text{以概率}\eta,\quad y &\leftarrow \varnothing && \text{以概率}\eta \text{ 用}\varnothing \text{ 替换}y,\\
\widehat{\mathcal{L}}_{\text{CFM}}^{\text{CFG}}(\theta) &= \|u_t^\theta(x \mid y) - u_t^{\text{target}}(x \mid z)\|^2 && \text{将模型对条件向量场回归.}
\end{aligned}$$

推断时，对固定的 y，可以通过下列方式采样：

$$\begin{aligned}
\text{初始化：}&\quad X_0 \sim p_{\text{init}}(x) && \text{用简单分布初始化（如高斯分布），}\\
\text{模拟：}&\quad dX_t = \widetilde{u}_t^\theta(X_t \mid y)dt && \text{从}t=0 \text{ 到}t=1 \text{ 模拟 } ODE,\\
\text{样本：}&\quad X_1 && \text{目标是使}X_1 \text{ 符合引导变量}y.
\end{aligned}$$

注意，如果使用权重 $w > 1$，X_1 的分布不再必然与 $X_1 \sim p_{\text{data}}(\cdot \mid y)$ 一致。然而经验上，这样做与条件更为吻合。因此，无分类器引导是一种启发式方法，其主要依据是优异的经验结果。事实上，你看到的几乎任何 AI 生成图像或视频都严重依赖 $w \geq 4$ 的无分类器引导。图 11 按文献 [18] 展示了在 128×128 ImageNet 上基于类别的无分类器引导。类似地，图 13 展示在手写数字 MNIST 数据集上应用无分类器引导采样时，不同引导尺度 w 的影响。

注 28 (扩散模型的引导)**.** 将流模型的讨论扩展到扩散模型是直接的：只需将 $u_t^\theta(x \mid y)$ 替换为 $\widetilde{u}_t^\theta(x \mid y)$，并按照第四节讨论的方法使用 *SDE* 采样。

6 构建大型图像或视频生成器

前几节我们学习了如何训练流匹配模型或扩散模型，从分布 $p_{\text{data}}(x \mid y)$ 中采样。这一方法具有一般性，可应用于多种不同数据类型和应用。本节深入研究大型图像和视频生成的具体情形，包括 FLUX 2.0、Stable Diffusion 3、Nano Banana、VEO-3 及 Meta Movie Gen Video 等著名模型。最后，我们将在实验中应用迄今所学，从零构建自己的这类模型！本节大致安排如下：



图 13: 对手写数字 MNIST 数据集使用不同引导尺度的无分类器引导的效果。左：$w = 1.0$；中：$w = 2.0$；右：$w = 4.0$。你将在实验三中自己生成类似图像。

1. 神经网络架构：先讨论如何将原始条件输入，包括时间 t 和引导变量 y_{raw}（即离散类别标签或原始文本），转换或嵌入为模型 $u_t^\theta(x \mid y)$ 本身能够处理的向量形式。随后讨论 $u_t^\theta(x \mid y)$ 的常用架构选择，包括 U-Net 和扩散 Transformer。

2. 潜空间：讨论变分自编码器，它允许在更低维的潜空间中进行生成建模，从而实现超高分辨率图像生成。

3. 案例研究：最后深入考察上述两个先进模型 Stable Diffusion 和 Meta MovieGen，让你了解大规模图像、视频应用中的具体做法。

6.1 神经网络架构

首先关注针对图像类模态（如图像和视频）的流与扩散模型中，可扩展神经网络架构的设计。具体而言，我们研究参数为 θ 的（引导）向量场 $u_t^\theta(x \mid y)$ 的任务在实践中如何实现。注意神经网络必须具有三个输入：向量 $x \in \mathbb{R}^d$、条件变量 $y \in \mathcal{Y}$ 和时间值 $t \in [0, 1]$，以及一个输出向量 $u_t^\theta(x \mid y) \in \mathbb{R}^d$。对于低维分布（如前几节的玩具分布），将 $u_t^\theta(x \mid y)$ 参数化为多层感知机（MLP），也称全连接神经网络，就已足够。即在此简单设定中，对 $u_t^\theta(x \mid y)$ 的前向传递包括拼接输入 x, y, t 并将它们传入 MLP。然而对于图像、视频和蛋白质等复杂高维分布，MLP 很可能不够，通常使用专门针对应用的架构。本小节后续考虑图像情形（并由此扩展到视频）。首先研究如何将原始条件信息——时间 t 和条件变量 y——嵌入为实际模型能够处理的向量形式；其次研究两种常见模型架构：U-Net [38, 17, 22, 11] 和扩散 Transformer（DiT） [12, 30, 28]。

6.1.1 条件变量的嵌入

时间嵌入。 对于简单的玩具模型，把 t 的原始值拼接到输入上便足以训练出性能合理的网络。实践中，通常使用 Fourier 特征将标量时间嵌入更高维空间，使模型更忠实地捕捉高频时间依

赖 [46]。显式地，特征化由下式给出：

$$\mathrm{TimeEmb}(t) = \sqrt{\frac{2}{d}}\big[\cos(2\pi w_1 t)\ \cdots\ \cos(2\pi w_{d/2} t)\ \sin(2\pi w_1 t)\ \cdots\ \sin(2\pi w_{d/2} t)\big]^T, \tag{68}$$

$$w_i = w_{\min}\left(\frac{w_{\max}}{w_{\min}}\right)^{(i-1)/(d/2-1)}, \qquad i = 1, \ldots, d/2. \tag{69}$$

这种 TimeEmb 选择是标准做法，但其精确形式并非严格必要。相反，上述方法只是获得维数为 d 的归一化嵌入的一种方便方式，即 $\|\mathrm{TimeEmb}(t)\| = 1$（因为 $\sin^2 + \cos^2 = 1$）。

类别标签嵌入。 当 $y_{\mathrm{raw}} \in \mathcal{Y} \triangleq \{0, \ldots, N\}$ 只是一个类别标签时，通常最简单的是为 y_{raw} 的 $N+1$ 个可能值分别学习一个嵌入向量，并将 y 设为该嵌入向量。我们将这些嵌入的参数视为 $u_t^\theta(x \mid y)$ 参数的一部分，因此在训练期间学习。

文本输入嵌入。 当 y_{raw} 是文本提示时，情况更复杂，方法主要依赖冻结的预训练模型。这些模型经训练将离散文本输入嵌入为捕获相关信息的连续向量。一个此类模型称为 CLIP（对比语言—图像预训练）。CLIP 经训练为图像和文本提示学习共享嵌入空间，其训练损失旨在使图像嵌入接近相应提示的嵌入，同时远离其他图像和提示的嵌入 [34]。因此可以取 $y = \mathrm{CLIP}(y_{\mathrm{raw}}) \in \mathbb{R}^{d_{\mathrm{CLIP}}}$，作为冻结的预训练 CLIP 模型产生的嵌入。某些情况下，将整个序列压缩为单一表示可能并不理想。此时还可以考虑使用预训练 Transformer 嵌入提示，以获得一个嵌入序列。条件化时结合多个此类预训练嵌入也很常见，以同时获得每个模型的优势 [14, 33]。就我们的目的而言，可以直接假设应用这样的模型后，提示嵌入的形状为 $\mathrm{PromptEmbed}(y_{\mathrm{raw}}) \in \mathbb{R}^{S \times k}$。

6.1.2 扩散 Transformer

深入这些架构的具体内容之前，回顾引言：图像只是一个向量 $x \in \mathbb{R}^{C_{\mathrm{image}} \times H \times W}$。这里 C_{image} 是通道数（RGB 图像通常有 $C_{\mathrm{input}} = 3$ 个颜色通道），H, W 分别表示图像以像素计的高度和宽度。一类特别重要的架构是所谓的扩散 Transformer（DiT）及其变体，它们使用注意力机制构造网络 [49, 30, 28]。扩散 Transformer 有不同形式。这里解释一种通用设计，但请注意 DiT 的具体实现可能随模型和应用而异。本节余下部分用 d 表示隐藏维数，L 表示 Transformer 层数，h 表示每层的注意力头数。扩散 Transformer 以视觉 Transformer（ViT）为基础，其主要思想基本上是将图像切分为图像块，嵌入这些块得到一个 token 序列，并通过标准注意力机制处理所得 token [13]。最后执行逆图像块化操作以恢复正确形状的图像。最初的图像块化操作只是对图像张量 $x \in \mathbb{R}^{C \times H \times W}$ 进行重排：

$$\mathrm{Patchify}(x) \in \mathbb{R}^{N \times C'},$$

其中 $C' = CP^2$、$N = (H/P) \cdot (W/P)$，P 是图像块大小。接着对输出应用线性变换，得到最终图像块嵌入：

$$\mathrm{PatchEmb}(x) = \mathrm{Patchify}(x)W \in \mathbb{R}^{N \times d},$$

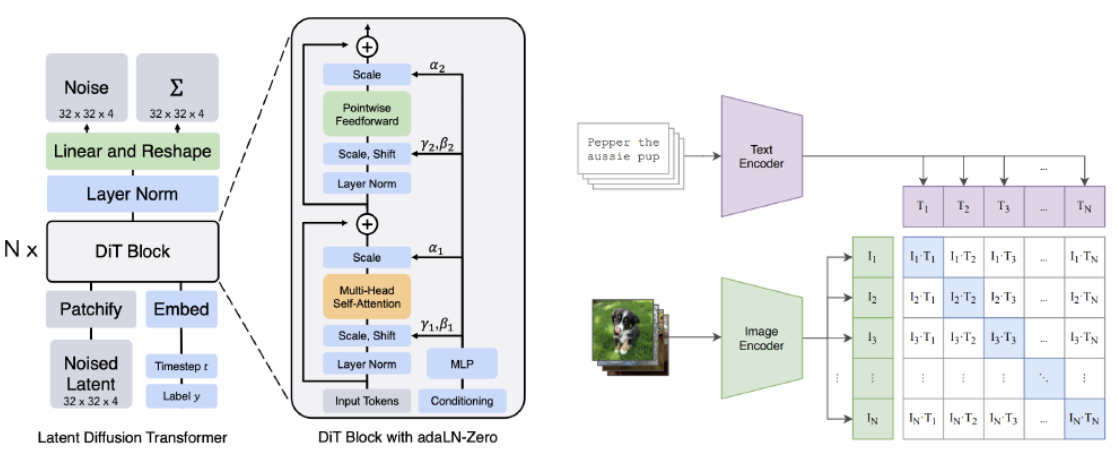


图 14: 左：扩散 Transformer 架构概览，取自文献 [30]；右：对比式 CLIP 损失的示意，模型学习共享的图像—文本嵌入空间，取自文献 [34]。图中 Patchify 为图像块化、Embed 为嵌入、Layer Norm 为层归一化，Text Encoder / Image Encoder 分别为文本 / 图像编码器。译注：其余图中文字：Noise / Noised Latent 为噪声 / 加噪潜变量；Linear and Reshape 为线性变换与重塑；Timestep / Label 为时间步 / 标签；Scale / Shift 为缩放 / 平移；Pointwise Feedforward 为逐位置前馈；Multi-Head Self-Attention 为多头自注意力；Input Tokens / Conditioning 为输入词元 / 条件；Latent Diffusion Transformer 为潜扩散 Transformer；DiT Block with adaLN-Zero 为采用 adaLN-Zero 的 DiT 块。右图例句 Pepper the aussie pup 为 “澳牧幼犬 Pepper”；其余模型名、矩阵元素与尺寸保留原记号。

其中 $W \in \mathbb{R}^{C' \times d}$ 是可学习权重矩阵。扩散 Transformer 的输入是时间嵌入、提示嵌入和图像块化图像张量，见第 6.1.1 节：

$$\begin{aligned} \widetilde{t} &= \mathrm{TimeEmb}(t) \in \mathbb{R}^{d}, \\ \widetilde{y} &= \mathrm{PromptEmb}(y) \in \mathbb{R}^{S \times d}, \\ \widetilde{x}_0 &= \mathrm{PatchEmb}(x) \in \mathbb{R}^{N \times d}. \end{aligned}$$

注意所有元素现在都具有 Transformer 所需的隐藏维数。扩散 Transformer 接着通过 DiTBlock 中的 Transformer 层，针对 $i = 0, \ldots, L-1$ 迭代更新 $\widetilde{z}_i$（细节见注 29）：

$$\widetilde{x}_{i+1} = \mathrm{DiTBlock}(\widetilde{x}_i, \widetilde{t}, \widetilde{y}) \in \mathbb{R}^{N \times d} \qquad (i = 0, \ldots, L-1), \tag{70}$$

其中 N 是层数。最后应用逆图像块化操作，将 DiT 输出映射回期望的输出形状：

$$u = \mathrm{Depatchify}(\widetilde{x}_N \widetilde{W}) \in \mathbb{R}^{C \times H \times W},$$

其中 $\widetilde{W} \in \mathbb{R}^{d \times C'}$。最终张量 u 即作为模型输出及预测速度 $u_t^\theta(x \mid y)$。

注 29 (DiT 块)**.** *为完整起见，我们简要给出单个 DiT 层的数学描述。虽然我们试图提供足够的细节以便总体理解 DiT 模型族，但提醒读者，这些描述着重关键算法选择而非架构细节。设 $x \in \mathbb{R}^{N \times d}$ 为当前图像块 token 序列（此处 $x = \widetilde{x}_i$），$y \in \mathbb{R}^{S \times d}$ 为嵌入后的引导变量（此处 $y = \widetilde{y}$）。典型 DiT 块通过以下方式更新 x：对图像块进行自注意力、对提示进行交叉注意力，以及通过自适应归一化（AdaLN）进行时间条件化。*

缩放点积注意力。给定查询 $Q \in \mathbb{R}^{N\times d_h}$、键 $K \in \mathbb{R}^{M\times d_h}$ 和值 $V \in \mathbb{R}^{M\times d_h}$，

$$\mathrm{Attn}(Q,K,V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right)V \in \mathbb{R}^{N\times d_h},$$

其中 *softmax* 按行应用。

多头注意力。设 h 为头数，$d_h = d/h$ 为每个头的维数。对每个头 $h \in \{1,\dots,n_{\text{heads}}\}$，学习投影矩阵 $W_Q^{(h)}, W_K^{(h)}, W_V^{(h)} \in \mathbb{R}^{k\times d_h}$。定义

$$\mathrm{head}_h(x,z) = \mathrm{Attn}(xW_Q^{(h)}, zW_K^{(h)}, zW_V^{(h)}),$$

其中源序列 z 为 $z = x$（图像块的自注意力）或 $z = y$（对提示的交叉注意力）。拼接各个头并施加输出投影 $W_O \in \mathbb{R}^{d\times d}$：

$$\mathrm{MultiHeadattention}(x,z) = \mathrm{Concat}(\mathrm{head}_1(x,z),\dots,\mathrm{head}_h(x,z))W_O \in \mathbb{R}^{N\times d}.$$

通过自适应归一化进行时间条件化。设 $\widetilde{t} \in \mathbb{R}^d$ 为时间步嵌入。*DiT* 中的标准选择是使用 $\widetilde{t}$ 产生逐通道缩放 / 偏移参数，调制归一化的激活 *[31]*。具体地，设 $g : \mathbb{R}^d \to \mathbb{R}^{2d}$ 为 *MLP*，并令 $(\gamma,\beta) = g(\widetilde{t})$，其中 $\gamma,\beta \in \mathbb{R}^d$（或者依实现而定，对不同子层如注意力和 *MLP* 分别使用独立的 (γ,β)）。给定 *token* 矩阵 $x \in \mathbb{R}^{N\times d}$ 和归一化算符 $\mathrm{Norm}(\cdot)$（如 *LayerNorm*），定义调制归一化为

$$\mathrm{AdaNorm}_{\widetilde{t}}(x) = (1+\gamma)\odot \mathrm{Norm}(H) + \beta,$$

其中 $\odot$ 表示逐元素乘法，并在 *token* 维度上广播。

组合起来。组合操作，也就是 *DiTBlock*，由下式给出：

$$\begin{aligned}
x &\leftarrow x + g_{\text{self}}(\widetilde{t}) \odot \mathrm{MultiHeadattention}(\mathrm{AdaNorm}_{\widetilde{t}}(x), \mathrm{AdaNorm}_{\widetilde{t}}(x)),\\
x &\leftarrow x + g_{\text{cross}}(\widetilde{t})\, \mathrm{MultiHeadattention}(\mathrm{AdaNorm}_{\widetilde{t}}(x), y),\\
x &\leftarrow x + g_{\text{MLP}}(\widetilde{t})\, \mathrm{MLP}(\mathrm{AdaNorm}_{\widetilde{t}}(x)),
\end{aligned}$$

其中 *MLP* 是逐位置的前馈网络，$g_{\dots}$ 是可学习门控参数。输出 $x \in \mathbb{R}^{N\times d}$ 成为下一层的图像块 *token* 序列（在我们的记号中为 $\widetilde{x}_{i+1}$）。最后注意，类别条件化的 *DiT*，例如实验中实现的版本，通常更简单：它们省略交叉注意力层，转而使用基于时间和类别的 *AdaNorm* 条件化。

6.1.3 U-Net

U-Net 架构 [38] 是 DiT 架构的一种替代方案，也是一类特定的卷积神经网络。它最初为图像分割而设计，其关键特征是输入和输出均具有图像形状（通道数可以不同）。这使其非常适合参数化向量场 $x \mapsto u_t^\theta(x \mid y)$：对固定的 y,t，输入和输出均具有图像形状。因此，U-Net 在早期扩散模型文献中得到广泛使用 [17, 22, 11]。U-Net 包含一系列编码器 E_i 及相应的解码器 D_i，中间还有一个潜在处理块，我们称之为中间编码器。[7]

[7] “中间编码器”（midcoder）是这里仿照 encoder 和 decoder、用于指 U-Net 堆栈最底部的完全非标准术语。

举例来说，跟踪图像 $x_t \in \mathbb{R}^{3\times256\times256}$（我们取 $(C_{\text{input}}, H, W) = (3, 256, 256)$）通过 U-Net 的路径：

$$
\begin{aligned}
x_t^{\text{input}} &\in \mathbb{R}^{3\times256\times256} && \text{U-Net 的输入,}\\
x_t^{\text{latent}} = E(x_t^{\text{input}}) &\in \mathbb{R}^{512\times32\times32} && \text{经编码器得到潜变量,}\\
x_t^{\text{latent}} = M(x_t^{\text{latent}}) &\in \mathbb{R}^{512\times32\times32} && \text{潜变量通过中间编码器,}\\
x_t^{\text{output}} = D(x_t^{\text{latent}}) &\in \mathbb{R}^{3\times256\times256} && \text{通过解码器得到输出.}
\end{aligned}
$$

注意输入通过编码器时，表示中的通道数增加，而图像高度和宽度减小。编码器和解码器通常都由一系列卷积层构成（其间插有激活函数、池化操作等）。上面没有展示两点：首先，输入 $x_t^{\text{input}} \in \mathbb{R}^{3\times256\times256}$ 在进入第一个编码器块之前，通常先送入初始预编码块以增加通道数；其次，编码器和解码器常由残差连接相连。完整图景见图 15。从较高层面看，多数 U-Net 都是上述描述的某种变体，但其中某些设计选择在实践中的不同实现中可能有所不同。特别是上面选择了纯卷积架构，而实践中通常在编码器和解码器中也加入注意力层。U-Net 得名于其编码器和解码器形成的“U”形（见图 15）。

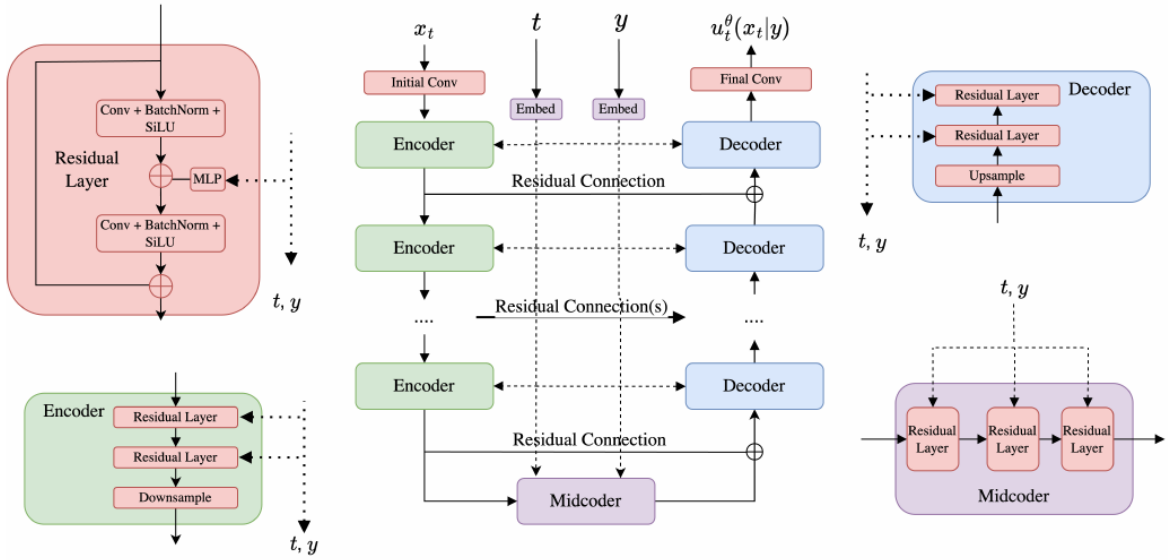


图 15: 简化的 U-Net 架构（本课程 2025 年版的实验 03 使用了类似架构）。图中 Encoder / Decoder / Midcoder 分别为编码器 / 解码器 / 中间编码器，Residual Connection 为残差连接，Downsample / Upsample 为下采样 / 上采样，Embed 为嵌入。译注：Initial Conv / Final Conv 为初始 / 最终卷积；Conv / BatchNorm 为卷积 / 批归一化；Residual Layer 为残差层；MLP 为多层感知机，SiLU 为激活函数名称。

6.2 在潜空间中工作：（变分）自编码器

到目前为止，我们一直在数据空间 $\mathbb{R}^d$ 中操作。然而，随着图像分辨率不断提高，直接在此空间中建模的成本会迅速高得难以承受。例如，具有三个 RGB 颜色通道的 1024×1024 图像对应总维数 $d = H \cdot W \cdot 3 \approx 3 \times 10^6$！注意对视频而言，所有东西都随帧数 T 缩放，因此维数会进一步增加。可以想象，在这样的空间上训练很快会变得不可行。图像分类的低维输出允许卷积堆

栈逐渐收窄；与之不同，基于流的建模方法要求输出 $u_t^\theta(x) \in \mathbb{R}^d$ 与输入同样大。于是重要问题变为：如何在合理的内存和计算预算内对高维图像建模？

6.2.1 标准自编码器

一个自然答案在于压缩：例如，实际图像空间或许位于高维图像空间的一个低得多维的流形附近。更具体地，可以考虑编码器 $\mu_\phi : \mathbb{R}^d \to \mathbb{R}^k$ 以及某个解码器 $\mu_\theta : \mathbb{R}^k \to \mathbb{R}^d$，它们共同将原始图像 $x \in \mathbb{R}^d$ 分别映射到潜变量 $z \in \mathbb{R}^k$ 及从中映射回来。维数 k 通常选得比 d 小得多。例如，对于 $d = 3 \times 1024 \times 1024$ 的图像，下采样到 $k = 3 \times (1024/16) \times (1024/16)$ 并不罕见。

μ_ϕ 与 μ_θ 合称为*自编码器*。理想情况下，选择它们以实现高重建质量，换言之，使 $\mu_\theta(\mu_\phi(x))$ 在平均意义上与 x 相似。因此通常用重建损失训练自编码器：

$$\mathcal{L}_{\text{Recon}}(\phi, \theta) = \mathbb{E}_{x \sim p_{\text{data}}} \left[\|\mu_\theta(\mu_\phi(x)) - x\|^2 \right],$$

它度量原始数据点 x 与重建数据点 $\mu_\theta(\mu_\phi(x))$ 之间的平方误差。

对生成建模的适用性。 遗憾的是，上述重建损失不足以训练出一个“好”的自编码器。回顾我们的最终目标是在潜空间中训练生成模型，目标是由 $z = \mu_\phi(x)$、$x \sim p_{\text{data}}$ 给出的潜分布 $p_{\text{latent}}(z)$。通过将潜空间生成模型的输出传入解码器 μ_θ，便实现了针对 $p_{\text{data}}(x)$ 的生成模型。按目前的方式表述自编码器会出现一个微妙问题：我们对 $p_{\text{latent}}(z)$ 几乎无法控制，因此基本无法保证它足够良好、适于训练这样的生成模型（即良好、简单、类似高斯分布）。虽然转换到潜空间可能压缩了数据，但也可能把数据分布 p_{data} 转换为非常难学的分布 p_{latent}。因此问题是：如何保证潜分布 p_{latent} 仍然良好且易于学习？为了更显式地正则化潜分布，下面在更一般的概率框架中重新表述自编码器概念，由此得到变分自编码器。

6.2.2 变分自编码器

从确定性的标准自编码器出发，放松编码器与解码器必须是确定性函数的约束，就得到变分自编码器（VAE）。具体地，考虑参数为 ϕ 的编码器 $q_\phi(z \mid x)$ 和参数为 θ 的解码器 $p_\theta(x \mid z)$。最常见的选择是

$$q_\phi(z \mid x) = \mathcal{N}(z; \mu_\phi(x), \text{diag}(\sigma_\phi^2(x))), \qquad p_\theta(x \mid z) = \mathcal{N}(x; \mu_\theta(z), \sigma_\theta^2(z) I_d), \tag{71}$$

其中 $\mu_\phi(x) \in \mathbb{R}^k$、$\sigma_\phi^2(x) \in \mathbb{R}^k_{\geq 0}$、$\mu_\theta(z) \in \mathbb{R}^d$、$\sigma_\theta^2(z) \in \mathbb{R}_{\geq 0}$ 都由神经网络参数化，diag 表示对角矩阵。要编码或解码一个变量，进行采样

$$\begin{aligned} z &\sim q_\phi(\cdot \mid x) && \text{编码,} \\ x &\sim p_\theta(\cdot \mid z) && \text{解码.} \end{aligned}$$

最后注意，当始终有 $\sigma_\phi(x) = 0$ 且 $\sigma_\theta(x) = 0$ 时，恢复标准自编码器。让我们考察重建损失的形式。一个自然的目标是

$$\mathcal{L}_{\text{VAE-Recon}}(\phi, \theta) = -\mathbb{E}_{x \sim p_{\text{data}}(x), z \sim q_\phi(\cdot \mid x)} [\log p_\theta(x \mid z)]. \tag{72}$$

注意两个变化：我们不再进行确定性编码，而是采样 $z \sim q_\phi(z \mid x)$；此外，现在取解码下 x 的负对数似然，也就是说损失实际上问的是，如果对原始数据点 x 编码并解码，它会有多大可能性。由于现在变成了随机过程，我们考虑所有可能的编码 / 解码。对于高斯情形，此重建损失变为

$$\mathcal{L}_{\mathrm{VAE-Recon}}(\phi,\theta) = \mathbb{E}_{x\sim p_{\mathrm{data}}(x), z\sim q_\phi(z|x)}\left[\frac{1}{2\sigma_\theta^2(z)}\|x-\mu_\theta(z)\|^2 + \frac{d}{2}\log\sigma_\theta^2(z)\right] + \text{const}, \tag{73}$$

其中使用了正态分布的密度（见式 (97)）。因此 VAE 重建损失与标准 AE 重建损失并没有那么不同，只需考虑所有可能的编码 $z \sim q_\phi(\cdot \mid x)$。依赖解码器方差的第二项控制重建准确性与预测不确定性之间的权衡。许多实现（包括实验中的实现）将 $\sigma_\phi(x)$ 和 $\sigma_\theta(z)$ 固定为可学习的标量常数（即分别独立于 x 和 z），从而避免学习方差时出现病态行为和数值稳定性问题。因此，此时 VAE 重建损失除编码的随机性和常数外，基本成为标准自编码器重建损失：

$$\mathcal{L}_{\mathrm{VAE-Recon}}(\phi,\theta) = \mathbb{E}_{x\sim p_{\mathrm{data}}(x), z\sim q_\phi(z|x)}\left[\frac{1}{2\sigma_\theta^2}\|x-\mu_\theta(z)\|^2\right] + \text{const}. \tag{74}$$

现在回顾我们的目标：希望对数据分布 $p_{\mathrm{data}}(x)$ 编码，使得将其映射到潜空间后，分布变得“良好”或易于学习。为此引入潜变量 z 的先验分布 $p_{\mathrm{prior}}(z)$。就我们的目的，取 $p_{\mathrm{prior}} = \mathcal{N}(0, I_k)$ 为各向同性高斯分布。选择这个先验分布实际上代表潜分布应具有的“理想”情况。正态分布很容易学习，因此满足我们获得“可训练”潜分布的目标。核心思想是正则化编码器，确保编码后的数据分布尽可能接近 p_{prior}，为此使用辅助损失

$$\mathcal{L}_{\mathrm{VAE-Prior}}(\phi) = \mathbb{E}_{x\sim p_{\mathrm{data}}(x)}[D_{\mathrm{KL}}(q_\phi(\cdot \mid x)\|p_{\mathrm{prior}})], \tag{75}$$

其中 D_{KL} 是 Kullback–Leibler（KL）散度。KL 散度是衡量两个概率分布差异的一种基本方式；详细解释超出本文范围，但注 30 简要回顾了相关背景。这里定义的损失 $\mathcal{L}_{\mathrm{VAE-Prior}}$ 很直观：我们希望对每个数据点 x，其编码分布都像高斯分布。如果对所有 x 都这样做，自然可以期待潜分布也会像高斯分布。

注 30 (KL 散度背景)**.** *对于两个概率密度 q, p，Kullback–Leibler 散度（KL 散度）定义为*

$$D_{\mathrm{KL}}(q(x)\|p(x)) = \int q(x)\log\frac{q(x)}{p(x)} = \mathbb{E}_{X\sim q}\left[\log\frac{q(X)}{p(X)}\right].$$

KL 散度是度量分布差异的标准量。特别地，它满足以下有用性质：

$$D_{\mathrm{KL}}(q(x)\|p(x)) \geq 0, \tag{76}$$

$$D_{\mathrm{KL}}(q(x)\|p(x)) = 0 \quad \Longleftrightarrow \quad q = p. \tag{77}$$

即它始终非负，并且当且仅当两个概率分布一致时为零。

为了定义变分自编码器的损失函数，现在可以使用权重参数 $\beta \geq 0$ 将重建损失和先验损失结合，得到 VAE 训练目标：

$$\mathcal{L}_{\mathrm{VAE}}(\phi,\theta) = \mathcal{L}_{\mathrm{VAE-Recon}}(\phi,\theta) + \beta\mathcal{L}_{\mathrm{VAE-Prior}}(\phi) \tag{78}$$

$$= -\mathbb{E}_{x\sim p_{\mathrm{data}}(x), z\sim q_\phi(z|x)}[\log p_\theta(x \mid z)] + \beta\mathbb{E}_{x\sim p_{\mathrm{data}}(x)}[D_{\mathrm{KL}}(q_\phi(\cdot \mid x)\|p_{\mathrm{prior}})], \tag{79}$$

其中第一项保证潜变量可以高效解码回数据，第二项保证我们的潜分布接近高斯分布；参数 β 控制各项强度。为使该损失更具体，我们推导高斯情形的 KL 散度。

例 31 (各向同性高斯分布之间的 KL 散度)**.**

设 $q(x)=\mathcal{N}(x;\mu_q,\mathrm{diag}(\sigma_q^2))$ 和 $p(x)=\mathcal{N}(x;\mu_p,\mathrm{diag}(\sigma_p^2))$ 是对角协方差矩阵的高斯分布，其中 $\sigma_q,\sigma_p\in\mathbb{R}^d_{\geq 0}$、$x\in\mathbb{R}^d$。则

$$D_{\mathrm{KL}}(q\|p)=\frac{1}{2}\left(K\left(\frac{\sigma_q^2}{\sigma_p^2}\right)+\frac{\|\mu_q-\mu_p\|^2}{\sigma_p^2}\right),\qquad K(\alpha)=\sum_{i=1}^{d}\alpha_i-\log\alpha_i-1. \tag{80}$$

上式很直观：如果均值和方差一致，$D_{\mathrm{KL}}(q\|p)=0$。此外，它随均值向量之间的平方误差 $\|\mu_q-\mu_p\|^2$ 增大。最后，函数 $K(\alpha)$ 在 $\alpha=1$ 处有唯一最小值，因此当 $\sigma_q=\sigma_p$ 时 $D_{\mathrm{KL}}(q\|p)$ 最小。

证明. 我们对 $d=1$ 证明（$d>1$ 时对各维求和，证明类似）。由正态分布密度，见式 (97)，有

$$\log q(x)=-\frac{1}{2}\log(2\pi\sigma_q^2)-\frac{1}{2\sigma_q^2}\|x-\mu_q\|^2,\qquad \log p(x)=-\frac{1}{2}\log(2\pi\sigma_p^2)-\frac{1}{2\sigma_p^2}\|x-\mu_p\|^2.$$

于是

$$\begin{aligned}D_{\mathrm{KL}}(q\|p)&=\mathbb{E}_{x\sim q}[\log q(x)-\log p(x)]\\&=\frac{1}{2}\log\frac{\sigma_p^2}{\sigma_q^2}+\frac{1}{2\sigma_p^2}\mathbb{E}_q[\|x-\mu_p\|^2]-\frac{1}{2\sigma_q^2}\mathbb{E}_q[\|x-\mu_q\|^2].\end{aligned} \tag{81}$$

对 $x\sim\mathcal{N}(\mu_q,\sigma_q^2 I)$，有

$$\mathbb{E}_q[\|x-\mu_q\|^2]=\mathrm{tr}(\sigma_q^2 I)=\sigma_q^2.$$

将此与 $x-\mu_p=(x-\mu_q)+(\mu_q-\mu_p)$ 及 $\mathbb{E}_q[x-\mu_q]=0$ 结合，得到

$$\mathbb{E}_q[\|x-\mu_p\|^2]=\mathbb{E}_q[\|x-\mu_q\|^2]+\|\mu_q-\mu_p\|^2=\sigma_q^2+\|\mu_q-\mu_p\|^2.$$

将这些代入上式即可得到式 (80)。 □

现在假设编码器是高斯形式，那么得到

$$\mathcal{L}_{\mathrm{VAE-Prior}}(\phi)=\mathbb{E}_{x\sim p_{\mathrm{data}}(x)}[D_{\mathrm{KL}}(q_\phi(\cdot\mid x)\|\mathcal{N}(0,I_k))]=\mathbb{E}\left[\frac{1}{2}K(\sigma_\phi^2(x))+\frac{1}{2}\|\mu_\phi(x)\|^2\right]. \tag{82}$$

这个损失很直观：均值 $\mu_\phi(x)$ 偏离零会受惩罚，方差偏离 1 也会受惩罚。作为 VAE 的总损失，得到

$$\begin{aligned}\mathcal{L}_{\mathrm{VAE}}(\phi,\theta)&=\mathcal{L}_{\mathrm{VAE-Recon}}(\phi,\theta)+\beta\mathcal{L}_{\mathrm{VAE-Prior}}(\phi)\\&=\mathbb{E}_{x\sim p_{\mathrm{data}}(x),z\sim q_\phi(z|x)}\Bigg[\underbrace{\frac{1}{2\sigma_\theta^2(z)}\|x-\mu_\theta(z)\|^2}_{\text{重建误差}}+\underbrace{\frac{d}{2}\log\sigma_\theta^2(z)}_{\text{解码器置信度}}\\&\qquad+\underbrace{\frac{\beta}{2}K(\sigma_\phi^2(x))}_{\text{使潜变量方差为 1}}+\underbrace{\frac{\beta}{2}\|\mu_\phi(x)\|^2}_{\text{使潜变量均值为 0}}\Bigg].\end{aligned} \tag{83}$$

上述损失函数的四项都很直观：第一项就是重建误差；第二项描述解码器的不确定性，较小方差使解码器更有“信心”，但也更强烈地惩罚重建误差。此外，我们希望使潜变量方差为 1、均值为 0，从而保证潜空间中的分布接近高斯分布。

训练 VAE。 还需讨论如何最小化 VAE 损失 $\mathcal{L}_{\mathrm{VAE}}(\phi,\theta)$。这一损失的问题是，迄今取期望所依据的分布 $q_\phi(z \mid x)$ 仍依赖参数 ϕ。不过，我们可以应用所谓的*重参数化技巧*改写它。具体地，对于

$$q_\phi(z \mid x) = \mathcal{N}(z; \mu_\phi(x), \sigma_\phi^2(x) I_k),$$

可以通过以下方式获得样本：

$$\epsilon \sim \mathcal{N}(0, I_k), \quad z = \mu_\phi(x) + \sigma_\phi(x)\epsilon \quad \Longrightarrow \quad z \sim q_\phi(\cdot \mid x).$$

注意在这个等式中，唯一的噪声 / 随机性来源是 ϵ，其分布与 ϕ 无关。因此可以将损失改写为

$$\mathcal{L}_{\mathrm{VAE}}(\phi,\theta) = \mathbb{E}_{x\sim p_{\mathrm{data}}(x),\epsilon\sim\mathcal{N}(0,I_k)}\left[\frac{1}{2\sigma_\theta^2(z)}\|x - \mu_\theta(\mu_\phi(x)+\sigma_\phi(x)\epsilon)\|^2 + \frac{d}{2}\log\sigma_\theta^2(z) + \frac{\beta}{2}K(\sigma_\phi^2(x)) + \frac{\beta}{2}\|\mu_\phi(x)\|^2\right].$$

重参数化后，随机性仅来自 $\epsilon \sim \mathcal{N}(0, I_k)$，其分布不依赖 ϕ。因此可以用标准深度学习工具最小化该损失。为进一步简化，可以再次令 $\sigma_\theta^2(z) = \sigma^2$ 在各处恒定，得到

$$\mathcal{L}_{\mathrm{VAE}}(\phi,\theta) = \mathbb{E}_{x\sim p_{\mathrm{data}}(x),\epsilon\sim\mathcal{N}(0,I_k)}\left[\frac{1}{2\sigma^2}\|x - \mu_\theta(\mu_\phi(x)+\sigma_\phi(x)\epsilon)\|^2 + \frac{\beta}{2}K(\sigma_\phi^2(x)) + \frac{\beta}{2}\|\mu_\phi(x)\|^2\right].$$

算法 6 总结 VAE 的训练过程。

算法 6：β-VAE 训练过程（固定方差高斯解码器 $p_\theta(x \mid z) = \mathcal{N}(x; \mu_\theta(z), \widetilde{\sigma}^2 I_d)$）。 **输入：** 样本 $x \sim p_{\mathrm{data}}$ 构成的数据集，编码器网络 $(\mu_\phi(x), \log\sigma_\phi^2(x))$，解码器网络 $\mu_\theta(z)$，潜空间维数 k，常数 $\beta \geq 0, \sigma^2 > 0$。

1. 对每个小批量 $\{x_i\}_{i=1}^B$，对每个 x_i 编码：$\mu_i \leftarrow \mu_\phi(x_i)$、$\log\sigma_i^2 \leftarrow \log\sigma_\phi^2(x_i)$。
2. 采样噪声 $\epsilon_i \sim \mathcal{N}(0, I_k)$。
3. 重参数化：$z_i \leftarrow \mu_i + \sigma_i \odot \epsilon_i$，其中 $\sigma_i = \exp(\frac{1}{2}\log\sigma_i^2)$。
4. 解码均值：$\widehat{x}_i \leftarrow \mu_\theta(z_i)$。
5. 重建损失：$\mathcal{L}_{\mathrm{recon}} \leftarrow \frac{1}{B}\sum_{i=1}^B \frac{1}{2\widetilde{\sigma}^2}\|x_i - \widehat{x}_i\|^2$。
6. 相对于先验 $p_{\mathrm{prior}}(z) = \mathcal{N}(0, I_k)$ 的 KL 损失：$\mathcal{L}_{\mathrm{KL}} \leftarrow \frac{1}{B}\sum_{i=1}^B \frac{1}{2}\sum_{j=1}^k(\mu_{i,j}^2 + \sigma_{i,j}^2 - \log\sigma_{i,j}^2 - 1)$。
7. 总损失：$\mathcal{L} \leftarrow \mathcal{L}_{\mathrm{recon}} + \beta\mathcal{L}_{\mathrm{KL}}$；更新 $(\phi,\theta) \leftarrow \mathrm{grad_update}(\mathcal{L})$。对各小批量重复。

实践说明。 这里的构造展示了自编码器设计的原理。当然在实践中，人们可能加入更多损失项或其他约束。因此最后补充几点关于自编码器的实践说明：

1. **选择 β（及 KL 预热）。**较大的 β 迫使潜变量更接近先验，但可能损害重建并引发后验坍缩（编码器忽略 x，输出 $q_\phi(z \mid x) \approx \mathcal{N}(0, I_k)$）。常见的稳定化方法是 KL 预热：从 $\beta = 0$ 开始，在最初几个 epoch 中逐渐增加至目标值。然而，在所有现代自编码器中，β 值都很小，即 $\beta \ll 1$。

2. **解码器方差**。学习高斯解码器方差 σ_θ^2 在数值上可能很棘手，除非加以正则化，否则可能导致退化解。为稳定起见，许多实现将 $p_\theta(x \mid z) = \mathcal{N}(x; \mu_\theta(z), \sigma^2 I_d)$ 的 σ^2 固定为常数，使重建项与均方误差成正比（相差常数）。

3. **像素 MSE 以外的重建损失**。对于图像，逐像素高斯似然（均方误差）常产生过于平滑的重建。实践中人们加入感知损失（使用预训练网络的特征空间损失）来改善清晰度和语义保真度。

4. **对抗与混合目标**。为进一步改善视觉真实感，可将 VAE 目标与对抗损失结合（VAE-GAN 风格），在解码样本上使用判别器。这通常使输出更清晰，但引入额外的优化不稳定性和超参数。

注 32 (在潜空间中工作)**.** 要训练潜空间生成模型，只需遵循已有训练方法，但直接在潜空间中工作。训练时，对 $x \sim p_{\text{data}}$ 从 $q_\phi(z \mid x)$ 抽取样本；推断时，从潜空间扩散或流模型采样 z，再用 $x = \mu_{\text{mean}}(z)$ 解码（注意取均值而非随机样本，以避免噪声引入的伪影）。直观地说，训练良好的自编码器可视为滤除了高频或其他语义上无意义的细节，使生成模型能够“聚焦”重要的、感知上相关的特征 *[36]*。在本文写作时，几乎所有先进图像和视频生成方法都遵循所谓的潜扩散范式，即在自编码器的潜空间内训练流模型或扩散模型 *[36, 48]*。但必须注意：还需要在训练扩散模型之前训练自编码器。关键是，性能现在还依赖于自编码器将图像压缩至潜空间以及恢复美观图像的能力。

附录 D 对 VAE 作了额外讨论。

6.3 案例研究：Stable Diffusion 3 与 Meta Movie Gen

本节最后简要考察两个大型生成模型：用于图像生成的 Stable Diffusion 3 和用于视频生成的 Meta Movie Gen Video [14, 33]。正如我们将看到的，这些模型使用本文描述的技术，并加入额外的架构增强，以扩展规模并适应文本输入等结构丰富的条件模态。

6.3.1 Stable Diffusion 3

Stable Diffusion 是一系列先进的图像生成模型，属于最早使用大规模潜扩散模型生成图像的一批模型。如果还没有试过，我们强烈建议你自己在线体验（`https://stability.ai/news/stable-diffusion-3`）。

Stable Diffusion 3 使用与本文研究的相同的条件流匹配目标（见算法 4）。[8]正如其论文所述，作者广泛测试了各种流和扩散替代方案，发现流匹配效果最好。训练时采用上文所述的无分类器引导训练（丢弃类别标签）。此外，Stable Diffusion 3 遵循第 6.1 节所述方法，在预训练自编码器的潜空间中训练。训练一个好的自编码器是最早的 Stable Diffusion 论文的重要贡献。

为增强文本条件化，Stable Diffusion 3 使用三种不同类型的文本嵌入（包括 CLIP 嵌入，以及 Google 预训练 T5-XXL 编码器输出的序列 [35]；与文献 [3, 39] 的方法类似）。CLIP 嵌入为输入文本提供粗粒度、整体性的嵌入，而 T5 嵌入提供更细粒度的上下文，使模型能够关注条件文本中的特定元素。为了适应这些序列上下文嵌入，作者提出扩展扩散 Transformer，使其不仅

[8]在他们的工作中，使用了另一种对噪声作条件化的约定。但这只是记号不同，算法相同。

关注图像块，也关注文本嵌入，从而将条件化能力从最初为 DiT 提出的基于类别的方案扩展到序列上下文嵌入。这种修改后的 DiT 称为多模态 DiT（MM-DiT），见图 16。最终最大模型具有 80 亿参数。

采样时，作者使用 50 步（即需要评估网络 50 次），采用 Euler 模拟方案和介于 2.0–5.0 之间的无分类器引导权重。

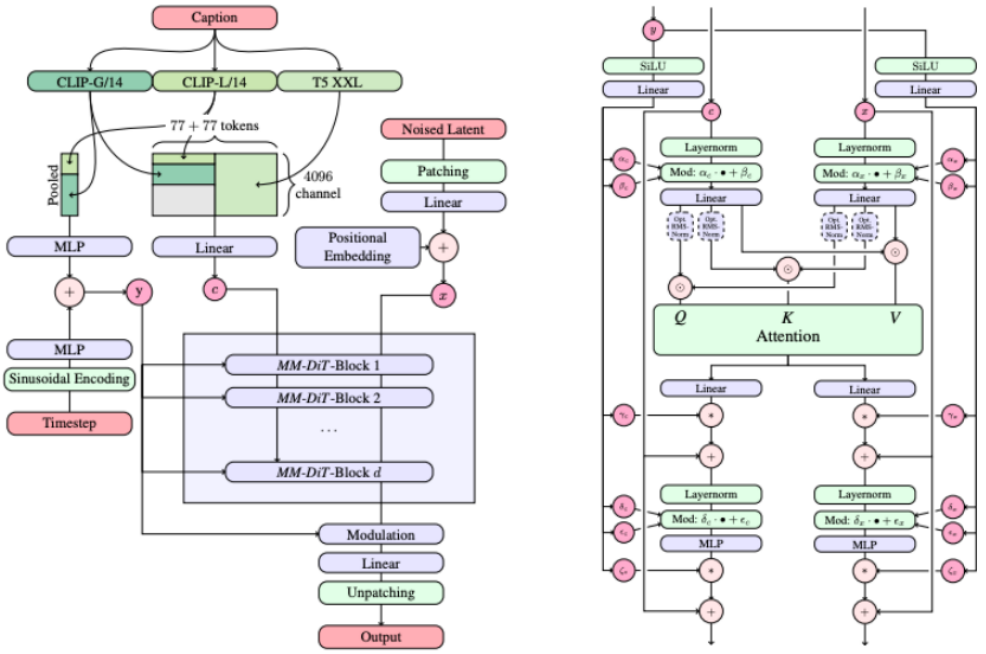


图 16: 文献 [14] 提出的多模态扩散 Transformer（MM-DiT）架构，图片亦取自文献 [14]。图中的 Caption / Noised Latent / Timestep 分别表示图像说明文本 / 加噪潜变量 / 时间步，Patching 为图像块化，Attention 为注意力，Output 为输出。译注：其余图中文字：Pooled 为池化；tokens / channel 为词元 / 通道；Linear 为线性层；Positional Embedding 为位置嵌入；Sinusoidal Encoding 为正弦编码；Modulation / Mod. 为调制；Unpatching 为逆图像块化；Layernorm 为层归一化；MLP 为多层感知机，SiLU 为激活函数；Q, K, V 分别为查询、键和值。MM-DiT-Block 为多模态 DiT 块，CLIP、T5 及调制参数保留原名和原记号。

6.3.2 Meta Movie Gen Video

接下来讨论 Meta 的视频生成器 Movie Gen Video（`https://ai.meta.com/research/movie-gen/`）。由于数据不是图像而是视频，数据 x 位于空间 $\mathbb{R}^{T\times C\times H\times W}$，其中 T 表示新增的时间维度（即帧数）。正如我们将看到的，在视频设定中许多设计选择都可以看作将已有图像技术（例如自编码器、扩散 Transformer 等）调整为处理这个额外时间维度。

Movie Gen Video 使用具有相同直线路径调度函数 $\alpha_t = t$、$\sigma_t = 1 - t$ 的条件流匹配目标。与 Stable Diffusion 3 类似，Movie Gen Video 也在冻结的预训练自编码器潜空间中工作。注意，使用自编码器减少内存消耗对视频而言比对图像更为重要——这也是目前大多数视频生成器在生成视频长度上很受限制的原因。

具体而言，作者提出引入时间自编码器（TAE）来处理新增的时间维度，它将原始视频 $x'_t \in$

$\mathbb{R}^{T'\times 3\times H\times W}$ 映射到潜变量 $x_t \in \mathbb{R}^{T\times C\times H\times W}$，其中 $T'/T = H'/H = W'/W = 8$ [33]。为容纳长视频，提出了一种时间分块过程：将视频切成若干片段，分别编码每一片段，然后将潜变量拼接起来 [33]。模型本身，即 $u_t^\theta(x_t)$，由类似 DiT 的主干给出，其中沿时间和空间维度对 x_t 分块。随后图像块通过一个 Transformer，它同时使用图像块间的自注意力及与语言模型嵌入的交叉注意力，类似于 Stable Diffusion 3 使用的 MM-DiT。

对于文本条件化，Movie Gen Video 使用三类文本嵌入：用于细粒度文本推理的 UL2 嵌入 [47]；用于关注字符级细节的 ByT5 嵌入（例如提示明确要求出现某些文字）[50]；以及在共享文本—图像嵌入空间中训练的 MetaCLIP 嵌入 [24, 33]。最终最大模型具有 300 亿参数。要获得明显更详细、广泛的讨论，我们鼓励读者查看 Movie Gen 技术报告本身 [33]。

7 离散扩散模型：用扩散构建语言模型

前几节我们研究了作为欧氏空间 $\mathbb{R}^d$ 上生成模型的流与扩散模型，它们使我们能够生成由向量 $z \in \mathbb{R}^d$ 表示的数据点。然而，并非所有数据都自然地建模为欧氏空间 $\mathbb{R}^d$ 中的点。文本或 DNA 等许多数据类型更自然地被视为离散状态空间 $\mathcal{S}$ 的元素。最重要的是，语言由我们希望建模的一串离散 token 构成。如何将流与扩散模型应用于这类数据？事实证明，前几节学到的原理同样可以推广到这类数据。在机器学习文献中，所得模型称为*离散扩散模型* [5, 16]。然而，重要的是记住这里没有数学意义上的扩散过程（离散状态空间中不存在 SDE）。我们使用连续时间 Markov 链（CTMC），而非 ODE/SDE。下面解释 CTMC 模型（第 7.1 节）及如何学习它们（第 7.2 节），从而使用流与扩散模型的原理构建大语言模型（LLM）。

7.1 连续时间 Markov 链（CTMC）模型

本节解释连续时间 Markov 链（CTMC）。可以将 CTMC 看作 SDE 的离散对应物，我们可以使用它构建生成离散状态的神经网络模型。此外，我们还将介绍 CTMC 模型，即允许利用 CTMC 生成文本等离散序列的神经网络模型。

首先刻画状态空间 $\mathcal{S}$。设 $\mathcal{V} = \{v_1, \dots, v_V\}$ 为词表。状态空间由 $\mathcal{S} = \mathcal{V}^d$ 给出，其中 $d \in \mathbb{N}$ 为序列长度，$V \in \mathbb{N}$ 为词表大小。对于语言，$\{v_1, \dots, v_V\}$ 可以枚举字母表或一组离散 token，而 $\mathcal{S}$ 表示长度为 d 的序列（或句子）的集合。对于 DNA，$\{v_1, \dots, v_V\}$ 可以是全部 4 种 DNA 碱基，$\mathcal{S}$ 是长度为 d 的所有 DNA 序列。接下来，设 X_t 是 $\mathcal{S}$ 上的随机过程，即 $\mathcal{S}$ 中的一条随机轨迹 $X : [0,1] \to \mathcal{S},\ t \mapsto X_t$。要求 X_t 是 Markov 过程，即没有记忆的过程。具体说来，这意味着满足以下条件：

$$\underbrace{p(X_{t+h} \mid X_t, X_{t_1}, \dots, X_{t_k})}_{\text{给定现在与过去时未来的概率}} = \underbrace{p(X_{t+h} \mid X_t)}_{\text{给定现在时未来的概率}}$$
$$(\text{对所有} 0 < h,\ 0 \le t_1 < t_2 < \cdots < t_k < t).$$

换言之，未来事件的概率只依赖现在，过去对未来不再有影响。注意 ODE/SDE 虽不在离散状态空间上，也是 Markov 过程。这里 X_t 位于离散空间，因此称为 Markov 链，具体而言是连续时间 Markov 链（CTMC）。量 $p_{t+h|t}(X_{t+h} \mid X_t)$ 是*转移概率*，它们与 Markov 链的初始分布 $X_0 \sim p_0$ 一起完全确定 CTMC。因此，说到 CTMC 时，也可以直接想到转移概率 $p_{t+h|t}(X_{t+h} \mid X_t)$。

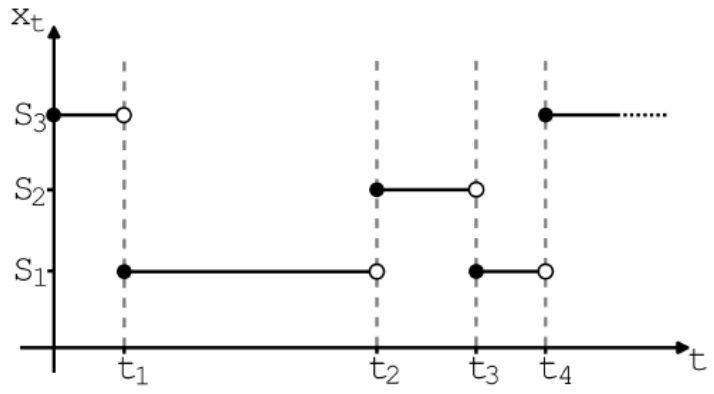


图 17: 状态空间为 $\mathcal{S} = \{S_1, S_2, S_3\}$（序列长度 $d = 1$）的 CTMC 轨迹示意。图改编自文献 [5]。

接下来推导离散情形中向量场的对应物。由于我们处于离散情形，只能在状态之间跳跃（或切换），不能像指定 ODE 时那样继续沿某个方向移动。因此定义*速率矩阵* $Q_t(y \mid x)$，它有效概括了从状态 $x \in \mathcal{S}$ 跳跃（或切换）到状态 $y \in \mathcal{S}$ 的速率。形式上，速率矩阵 Q_t 由一个关于时间连续的有界函数给出：

$$Q : \mathcal{S} \times \mathcal{S} \times [0, 1] \to \mathbb{R}, \qquad (x, y, t) \mapsto Q_t(y \mid x), \tag{84}$$

其中 $Q_t(y \mid x)$ 描述从 x 切换到 y 的速率，并满足

$$\text{（1）离开速率为正：} \quad Q_t(y \mid x) \geq 0 \quad \text{只要} x \neq y, \tag{85}$$

$$\text{（2）停留速率等于负离开速率：} \quad Q_t(x \mid x) = -\sum_{y \neq x} Q_t(y \mid x) \quad \text{对所有} x. \tag{86}$$

这两个条件直观明了：第一个条件说，从 x 切换到不同状态 $y \neq x$ 的速率只能非负（不切换对应 0，因此小于 0 的速率没有意义）。第二个条件说，停留在 x 的速率 $Q_t(x \mid x)$ 应抵消离开 x 的速率；它本质上是一致性条件，说明要么留在 x，要么离开（没有第三种选择）。特别注意，这些条件蕴含 $Q_t(x \mid x) \leq 0$。因此，$Q_t(y \mid x)$ 是对角元均非正、非对角元均非负的矩阵。

现在可以定义微分方程的对应物，即 CTMC “遵循” 速率矩阵的条件。其基本思想是 X 的分布或演化应遵循速率矩阵 Q_t。换言之，要求转移概率满足

$$\frac{d}{dh} p_{t+h|t}(X_{t+h} = y \mid X_t = x)\Big|_{h=0} = Q_t(y \mid x) \quad \text{对所有} x, y \in \mathcal{S},\ 0 \leq t. \tag{87}$$

左侧是从 x 切换到 y 的概率的无穷小变化率。我们要求这些概率按照速率矩阵所指定的方式变化。简单检查一下提出这些条件是否合理：如果按式 (87) 定义 $Q_t(y \mid x)$，它是否是有效速率矩阵？当 $h = 0$ 时，从 x 切换到 $y \neq x$ 的概率为零（因为没有经过时间），即对所有 $y \neq x$ 有 $p_{t|t}(y \mid x) = 0$。因此导数必须非负，只要 $y \neq x$ 就有 $Q_t(y \mid x) \geq 0$，验证了式 (85) 的第一个条

件。此外我们知道

$$\begin{aligned}\sum_{y\neq x} Q_t(y \mid x) &= \sum_{y\neq x} \frac{d}{dh} p(X_{t+h}=y \mid X_t=x)\Big|_{h=0} \\ &= \frac{d}{dh}\sum_{y\neq x} p(X_{t+h}=y \mid X_t=x)\Big|_{h=0} \\ &= \frac{d}{dh}\big(1-p(X_{t+h}=x \mid X_t=x)\big) = -Q_t(x \mid x),\end{aligned}$$

其中使用概率之和为 1。由此得到式 (86)，验证每个 CTMC 至少有一个满足式 (87) 的速率矩阵。但若反过来，给定 Q_t 是否有对应 CTMC，若有是否唯一？事实确实如此。

定理 33 (CTMC 的存在性与唯一性)**.** *对于任意速率矩阵 Q_t（有界且关于时间 t 连续），存在唯一 Markov 链 X_t（即唯一一组转移概率 $p_{t+h|t}(y \mid x)$）满足式 (87)。*

对有兴趣的读者，我们在附录 C 给出自包含证明。此定理的关键结论是：为机器学习目的，我们可以构造一个速率矩阵 Q_t（例如通过神经网络），并假设存在唯一对应于 Q_t 的 Markov 链。

例 34 (等跳跃速率的双状态 CTMC)**.** *设 $\mathcal{S}=\{a,b\}$，考虑以恒定速率 $\lambda>0$ 在两状态间切换的时间齐次 CTMC $(X_t)_{t\geq 0}$：*

$$Q = \begin{array}{c|cc} & a & b \\ \hline a & -\lambda & \lambda \\ b & \lambda & -\lambda \end{array}.$$

则时间增量 $h\geq 0$ 上的转移概率也不依赖时间 t，由下式给出：

$$\begin{pmatrix} p(X_{t+h}=a \mid X_t=a) & p(X_{t+h}=a \mid X_t=b) \\ p(X_{t+h}=b \mid X_t=a) & p(X_{t+h}=b \mid X_t=b) \end{pmatrix} = \frac{1}{2}\begin{pmatrix} 1+e^{-2\lambda h} & 1-e^{-2\lambda h} \\ 1-e^{-2\lambda h} & 1+e^{-2\lambda h} \end{pmatrix}.$$

可以手工验证式 (87) 成立，即这些转移概率确实对应于该速率矩阵。事实上，这些速率很直观：链以瞬时速率 λ 不断切换。指数项 $e^{-2\lambda h}$ 刻画初始状态的记忆如何衰减。经过无限时间，即 $h\to\infty$ 时，有

$$P(h) \to \begin{pmatrix} 1/2 & 1/2 \\ 1/2 & 1/2 \end{pmatrix},$$

所以链忘记起始位置，并以 $1/2$ 的概率处于 a 或 b。切换速率 $\lambda>0$ 越大，收敛越快。

CTMC 的模拟。 接下来思考如何模拟 CTMC 的一条轨迹。设 $h>0$ 为步长，p_{init} 是 $\mathcal{S}$ 上的初始分布，例如 $p_{\text{init}}=\text{Unif}_{\mathcal{S}}$ 是 $\mathcal{S}$ 上的均匀分布。可以通过设 $X_0\sim p_{\text{init}}$，并令

$$X_{t+h} \sim p_{t+h|t}(\cdot \mid X_t)$$

来迭代模拟。如果知道 $p_{t+h|t}(\cdot \mid X_t)$，这就可行。然而，除最简单的 CTMC 外，我们通常不知道转移核的闭式形式，只能得到速率矩阵 Q_t。尽管如此，由式 (87)：

$$\begin{aligned} p_{t+h|t}(X_{t+h}=y \mid X_t=x) &= p_{t|t}(X_t=y \mid X_t=x) + hQ_t(y \mid x) + R_t(h) \\ &= \mathbf{1}_{y=x} + hQ_t(y \mid x) + R_t(h), \end{aligned}$$

其中 $R_t(h)$ 为在 h 很小时可忽略的误差项。因此对小的 h，可令

$$p_{t+h|t}(X_{t+h} = y \mid X_t = x) \approx \mathbf{1}_{y=x} + hQ_t(y \mid x) =: \widetilde{p}_{t+h|t}(y \mid x).$$

根据对速率矩阵施加的条件，可以验证 h 较小时 $\widetilde{p}_{t+h|t}(y \mid x)$ 确实是有效概率分布。因此可以近似地采样下一点：

$$X_{t+h} \sim \widetilde{p}_{t+h|t}(\cdot \mid x) = (\mathbf{1}_{y=x} + hQ_t(y \mid x))_{y\in\mathcal{S}}. \tag{88}$$

由于这只是离散分布，可以用标准方法轻松采样。这是模拟 CTMC 的简单方式。

CTMC 模型。 接下来定义如何在神经网络中参数化 CTMC。CTMC 模型（或离散扩散模型）由 $\mathcal{S}$ 上的初始分布 p_{init} 和参数为 θ 的神经网络 Q_t^θ 给出，使得对每个输入 $x \in \mathcal{S}$，模型返回速率矩阵的一整列：

$$x \mapsto \{Q_t^\theta(y \mid x)\}_{y\in\mathcal{S}}.$$

我们希望模型返回整列，因为模拟 CTMC（式 (88)）需要它，即需要它来采样下一状态。

上述模型有一个复杂之处：空间 $\mathcal{S}$ 可能非常大。特别地，$|\mathcal{S}| = V^d$，其中 V 是词表大小，d 是序列长度。这种指数增长使得在内存中存储速率矩阵的一整列基本不可能——$\{Q_t^\theta(y \mid x)\}_{y\in\mathcal{S}}$ 无法在计算机中表示。因此必须约束模型。具体而言，几乎所有 CTMC 模型都是因子化的（见图 18），这实际上是一种稀疏性约束。因子化 CTMC 模型由一个 CTMC 模型 Q_t^θ 给出，使得对所有 $y = (y_1, \ldots, y_d), x = (x_1, \ldots, x_d) \in \mathcal{S} = \mathcal{V}^d$，都有

$$Q_t^\theta(y \mid x) = 0 \quad \text{当} y_i \neq x_i \text{ 的位置} i \text{ 超过一个时}.$$

我们将与 x 至多有一个 token 不同的所有 y 称为 x 的邻居 $\mathcal{N}(x)$。这样的因子化 CTMC 模型可写为

$$x \mapsto \{Q_t^\theta(y \mid x)\}_{y\in\mathcal{N}(x)} = \begin{pmatrix} Q_t^\theta(v_1, 1 \mid x) & \cdots & Q_t^\theta(v_V, 1 \mid x) \\ & \cdots & \\ Q_t^\theta(v_1, d \mid x) & \cdots & Q_t^\theta(v_V, d \mid x) \end{pmatrix},$$

其中 $Q_t(y \mid x) = Q_t^\theta(v_i, j \mid x)$ 给出从 $x = (x_1, \ldots, x_d)$ 转移到将第 j 个元素替换为 v_i 后的邻居 $y = (x_1, \ldots, x_{j-1}, v_i, x_{j+1}, \ldots, x_d)$ 的速率。每一行对应位置 $i = 1, \ldots, d$ 的一个速率矩阵，即要求

$$Q_t^\theta(v, i \mid x) \geq 0 \quad \text{若} v \neq x_i, \qquad\qquad Q_t(x_i, i \mid x) = -\sum_{v\neq x_i} Q_t^\theta(v, i \mid x).$$

这些条件容易在神经网络输出上施加，例如可以对长度为 d 的序列使用输出维数为 V 的 Transformer 模型。还要注意，因子化速率矩阵的输出形状为 $d \times V$，该大小随维数线性而非指数增长。

模拟 CTMC 模型。 为从 CTMC 模型采样，我们抽取 $X_0 \sim p_{\text{init}}$ 并迭代，按式 (88) 采样下一状态。算法 7 给出了具体步骤。如算法所示，对于因子化 CTMC 模型，可采用逐 token 并行 Euler 近似：在一个小步长 $h > 0$ 内，每个 token 独立更新。此近似与完整 CTMC Euler 步在 h 的一阶一致，但允许以 $O(h^2)$ 的概率同时更新多个 token。

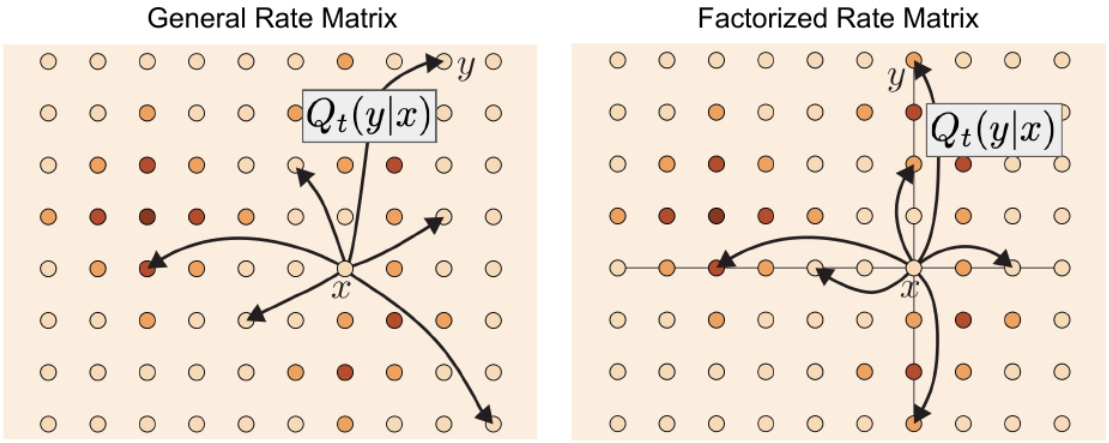


图 18: 因子化 CTMC 模型示意。仅当起点和终点只在一个维度上不同时（此处 $d = 2$），因子化 CTMC 才具有非零速率 $Q_t(y \mid x) \neq 0$。图取自文献 [26]；General Rate Matrix / Factorized Rate Matrix 分别为一般 / 因子化速率矩阵。译注：图注的“只在一个维度上不同”针对非对角跃迁率；对角元仍由离开速率之和的负值确定。

算法 7：从因子化 CTMC 模型采样。 **输入：**因子化速率网络 Q_t^θ、初始分布 p_{init}、步数 n。

1. 设 $t \leftarrow 0$，步长 $h \leftarrow 1/n$。
2. 抽取样本 $X_0 \sim p_{\text{init}}$，其中 $X_0 = (X_0^{(1)}, \ldots, X_0^{(d)}) \in \mathcal{V}^d$。
3. 对 $i = 1, \ldots, n$：计算因子化跳跃速率 $\{q_j(v)\}_{j=1..d,\, v\in\mathcal{V}} \leftarrow Q_t^\theta(\cdot \mid X_t)$。
4. 对 $j = 1, \ldots, d$ 并行执行：令 $x \leftarrow X_t^{(j)}$（当前位置的 token），定义逐位置 Euler 转移概率

$$\widetilde{p}_{j,t}(v \mid x) = \begin{cases} h\, q_j(v), & v \neq x, \\ 1 - h \sum_{v' \in \mathcal{V}\setminus\{x\}} q_j(v'), & v = x, \end{cases}$$

   并采样 $X_{t+h}^{(j)} \sim \text{Categorical}(\{\widetilde{p}_{j,t}(v \mid x)\}_{v\in\mathcal{V}})$。
5. 更新 $t \leftarrow t + h$ 并重复，最后返回 X_1。

7.2 训练 CTMC 模型

接下来讨论如何学习 CTMC 模型。其原理与流匹配相同：（1）构造在噪声与数据之间插值的概率路径；（2）推导条件速率矩阵和边缘速率矩阵；（3）以免模拟的方式学习边缘速率矩阵。现在逐步解释这一方法。

本节中，数据分布 p_{data} 是 $\mathcal{S}$ 上由概率质量函数刻画的分布，即 $p_{\text{data}} : \mathcal{S} \to \mathbb{R}_{\geq 0}$，$z \mapsto p_{\text{data}}(z)$，其中 $\sum_{z\in\mathcal{S}} p_{\text{data}}(z) = 1$。我们不知道 p_{data}，但训练期间可以以数据集的形式获取样本 $z \sim p_{\text{data}}$，例如互联网上的所有文本。目标是学会生成样本 $z \sim p_{\text{data}}$。我们要训练 CTMC 模型 Q_t^θ，使

$$X_0 \sim p_{\text{init}}, \quad X_t \text{ 是} Q_t^\theta \text{ 的 CTMC} \quad \Longrightarrow \quad X_1 \sim p_{\text{data}}.$$

因此可以看出，这与欧氏空间 $\mathbb{R}^d$ 的情形（见第二、三节）并无不同，只是使用了 CTMC 模型而非流 / 扩散模型。

7.2.1 条件与边缘概率路径

定义函数 $\delta_z(x)$：若 $x \neq z$ 则 $\delta_z(x) = 0$，若 $x = z$ 则 $\delta_z(x) = 1$。（离散）条件概率路径由 $x, z \in \mathcal{S}$、$0 \leq t \leq 1$ 时满足下式的一组分布 $p_t(x \mid z)$ 给出：

$$p_0(\cdot \mid z) = p_{\text{init}}, \qquad\qquad p_1(\cdot \mid z) = \delta_z.$$

与欧氏情形类似，离散条件概率路径在一个不依赖 z 的分布和一个将全部概率质量集中于 z 的分布之间插值。（离散）边缘概率路径由下式给出：

$$p_t(x) = \sum_{z \in \mathcal{S}} p_t(x \mid z) p_{\text{data}}(z).$$

容易验证边缘概率路径在“噪声”和数据之间插值：

$$p_0 = p_{\text{init}}, \qquad p_1 = p_{\text{data}}. \tag{89}$$

例 35 (因子化混合路径：对每个 token 独立加噪)**.** 设 $\mathcal{S} = \mathcal{V}^d$，并设 $p_{\text{init}}(x) = \prod_{j=1}^{d} p_{\text{init}}^{(j)}(x_j)$ 为因子化初始分布。固定满足 $0 \leq \kappa_t \leq 1$、$\kappa_0 = 0$、$\kappa_1 = 1$ 且 $\frac{d}{dt}\kappa_t \geq 0$ 的调度函数。[9]定义条件路径为

$$p_t(x \mid z) = \prod_{j=1}^{d} \left[(1 - \kappa_t) p_{\text{init}}^{(j)}(x_j) + \kappa_t \delta_{z_j}(x_j) \right].$$

等价地，可以通过独立同分布地抽取掩码 $m_j = 0, 1$ 和噪声 $\xi_j \sim p_{\text{init}}^{(j)}$，然后设置如下变量，来采样 $x \sim p_t(\cdot \mid z)$：

$$\begin{aligned} m_j &\sim \text{Bernoulli}(\kappa_t), \quad \xi_j \sim p_{\text{init}}^{(j)}, \\ x_j &= m_j z_j + (1 - m_j)\xi_j, \qquad j = 1, \ldots, d, \\ x &= (x_1, \ldots, x_d). \end{aligned}$$

我们称上述过程为因子化混合路径。上述过程实际上以概率 $1 - \kappa_t$ 对序列每个位置独立地“破坏”第 j 个 *token*：即在 $t = 0$ 时 $1 - \kappa_t = 1$，所有信息都被破坏；在 $t = 1$ 时 $1 - \kappa_t = 0$，没有信息被破坏。注意，它与例 *8* 的高斯概率路径相似，因为信息以调度函数 κ_t 确定的速度逐渐被破坏。但它也与高斯概率路径不同：因子化混合路径不移动 / 输运概率质量（由于在离散空间中，不存在方向），而只是使一个分布逐渐显现、另一个分布逐渐消退。

7.2.2 条件速率矩阵与边缘速率矩阵

下一步，我们构造离散流匹配的训练目标。首先构造条件速率矩阵，它对应于流匹配的条件向量场。对每个数据点 $z \in \mathcal{S}$，设 $Q_t^z(y \mid x)$ 是一个速率矩阵。若

$$X_0 \sim p_{\text{init}}, \quad X_t \text{ 是} Q_t^z \text{ 的 CTMC} \quad \Longrightarrow \quad X_t \sim p_t(\cdot \mid z),$$

则称它为条件速率矩阵。换言之，条件速率矩阵使其 CTMC“遵循”条件概率路径。条件速率矩阵可作为构件，构造遵循边缘概率路径的边缘速率矩阵：

[9]译注：原文此处对 κ_t 再求导；后续速率非负实际使用的是 $\kappa_t \geq 0$。保留原文记号，不将两个条件视为等价。

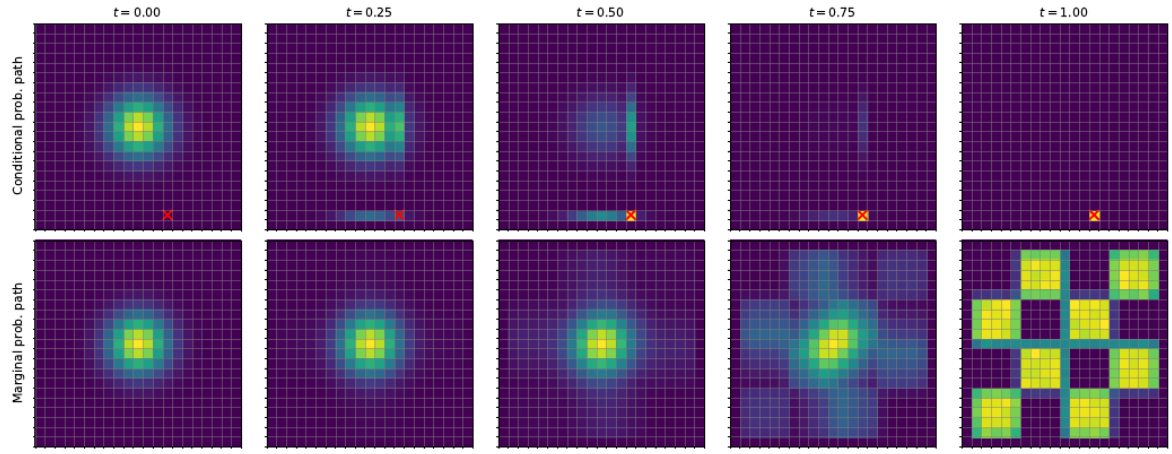


图 19: $d = 2$ 时离散概率路径示意。上排：在初始分布和 Dirac 分布之间插值的条件概率路径。下排：在初始分布和数据分布（此处为棋盘图案）之间插值。注意与图 5 的相似和不同：这里概率路径是“传送”的，即降低初始分布权重并提高终点分布权重。译注：Conditional prob. path / Marginal prob. path 分别为条件 / 边缘概率路径，五列时间为 $t = 0, 0.25, 0.5, 0.75, 1$。

定理 36 (离散边缘化技巧)**.** *以下定义的边缘速率矩阵*

$$Q_t(y \mid x) = \sum_{z \in \mathcal{S}} Q_t^z(y \mid x) \frac{p_t(x \mid z) p_{\text{data}}(z)}{p_t(x)} = \sum_{z \in \mathcal{S}} Q_t^z(y \mid x) p_{1|t}(z \mid x), \tag{90}$$
$$p_{1|t}(z \mid x) := \frac{p_t(x \mid z) p_{\text{data}}(z)}{p_t(x)}$$

是有效的速率矩阵，并满足条件

$$X_0 \sim p_{\text{init}}, \quad X_t \text{ 是} Q_t \text{ 的 } CTMC \quad \Longrightarrow \quad X_t \sim p_t.$$

特别地，由式 (89) 有 $X_1 \sim p_{\text{data}}$，即边缘速率矩阵对应的 CTMC 将噪声转换为数据。

要证明这一陈述，我们需要 CTMC 的一个基本方程——所谓 Kolmogorov 前向方程：

命题 2 (Kolmogorov 前向方程)**.** *设对每个 $0 \le t \le 1$，p_t 都是 $\mathcal{S}$ 上的一组分布。再设 X_t 为具有矩阵 Q_t 和初始分布 p_0 的 CTMC。那么，对所有 $0 \le t \le 1$ 有 $X_t \sim p_t$，当且仅当 Kolmogorov 前向方程（KFE）成立：*

$$\frac{d}{dt} p_t(x) = \sum_{y \in \mathcal{S}} Q_t(x \mid y) p_t(y).$$

KFE 的证明. 为证明 KFE 的必要性，假设 $p_t(x)$ 是 CTMC 的真实边缘分布，即对每个 $0 \le t \le 1$ 都有 $X_t \sim p_t$。则可计算：

$$\frac{d}{dt} p_t(x) \overset{(i)}{=} \left. \frac{d}{dh} \right|_{h=0} p_{t+h}(x) \overset{(ii)}{=} \left. \frac{d}{dh} \right|_{h=0} \sum_y p_{t+h|t}(x \mid y) p_t(y)$$
$$\overset{(iii)}{=} \sum_y \left. \frac{d}{dh} \right|_{h=0} p_{t+h|t}(x \mid y) p_t(y) \overset{(iv)}{=} \sum_y Q_t(x \mid y) p_t(y),$$

其中 (i) 只使用时间偏移，(ii) 使用转移概率定义，(iii) 交换求和与导数，(iv) 使用速率矩阵的定义，见式 (87)。

接下来为证明充分性，可将 KFE 改写为矩阵形式：$\frac{d}{dt}p_t = Q_t p_t$，其中将 $p_t = (p_t(x))_{x\in\mathcal{S}}$ 看作向量、$Q_t = (Q_t(y \mid x))_{x,y\in\mathcal{S}}$ 看作矩阵。注意上述方程是向量空间 $\mathbb{R}^{\mathcal{S}}$ 上的线性 ODE。其初始条件由定理中给出的 p_0 固定。因此若任何其他一组边缘分布 q_t 满足此方程，由 ODE 的唯一性（见定理 3）可得 $q_t = p_t$。这表明 KFE 也充分。 □

定理 36 的证明. 利用 KFE，剩下只需证明定理中定义的边缘速率矩阵（见式 (90)）满足 KFE：

$$\begin{aligned}
\frac{d}{dt}p_t(x) &\overset{(i)}{=} \frac{d}{dt}\sum_{z\in\mathcal{S}} p_t(x \mid z)p_{\text{data}}(z) \overset{(ii)}{=} \sum_{z\in\mathcal{S}} \frac{d}{dt}p_t(x \mid z)p_{\text{data}}(z) \\
&\overset{(iii)}{=} \sum_{z\in\mathcal{S}} \left[\sum_{y\in\mathcal{S}} Q_t^z(x \mid y)p_t(y \mid z)\right] p_{\text{data}}(z) \\
&\overset{(iv)}{=} \sum_{y\in\mathcal{S}} p_t(y) \left[\sum_{z\in\mathcal{S}} Q_t^z(x \mid y)\frac{p_t(y \mid z)p_{\text{data}}(z)}{p_t(y)}\right] \\
&\overset{(v)}{=} \sum_{y\in\mathcal{S}} p_t(y)Q_t(x \mid y),
\end{aligned}$$

其中 (i) 由边缘概率路径定义得出；(ii) 交换求和与导数；(iii) 对条件速率矩阵使用 KFE；(iv) 乘以并除以 $p_t(y)$；(v) 使用边缘速率矩阵 $Q_t(y \mid x)$ 的定义。这说明 KFE 成立，由命题 2 可得结论。 □

现在推导因子化混合路径的条件速率矩阵的具体例子。

例 37 (因子化混合路径的条件速率矩阵)**.** 令 $\frac{d}{dt}\kappa_t = \dot{\kappa}_t$。因子化混合路径具有因子化条件速率矩阵：[1011]

$$\begin{aligned}
Q_t^z(y \mid x) &= (Q_t^z(v_i, j \mid x_j))_{v_i,j}, \\
Q_t^z(v_i, j \mid x_j) &= \frac{\dot{\kappa}_t}{1-\kappa_t}(\delta_{z_j}(v_i) - \delta_{x_j}(v_i)) \\
&= \frac{\dot{\kappa}_t}{1-\kappa_t}\begin{cases} 0, & x_j = z_j, \\ 1, & v_i = z_j,\ x_j \neq z_j, \\ 0, & v_i \neq z_j,\ x_j \neq z_j, \\ -1, & v_i = x_j,\ x_j \neq z_j. \end{cases}
\end{aligned}$$

注意，这是非常简单的速率矩阵：它只允许跳转到 z^j，即若任一 *token* j 更新，就必须跳至终点数据 $z = (z_1, \ldots, z_d)$ 的相应 *token* 值；而且只有尚未到达该值时才跳到 z^j。

[10]译注：下方分段式的第三与第四种条件有重叠：当 $v_i = x_j \neq z_j$ 时，前一行写 0 而后一行写 -1。本稿保留原式；前面的 Kronecker delta 表达式可用于辨别其意图。

[11]译注：以下速率含 $\dot{\kappa}_t/(1-\kappa_t)$。若 $\kappa_1 = 1$，它在终点可能发散，例如 $\kappa_t = t$ 时为 $1/(1-t)$。因此前文关于闭区间上有界连续速率的存在唯一性定理不能直接涵盖该终点；需在 $t < 1$ 上处理并另行核查 $t \to 1$ 的极限。

证明. 注意因子化混合路径完全分解为独立分量，所建议的条件速率矩阵也如此。因此不失一般性可以假设 $d = 1$，只需对每个维度计算。于是可推导：

$$\begin{aligned}
\frac{d}{dt}p_t(x \mid z) &\overset{(i)}{=} \frac{d}{dt}[(1-\kappa_t)p_{\text{init}}(x) + \kappa_t \delta_z(x)] \\
&\overset{(ii)}{=} \dot{\kappa}_t \delta_z(x) - \dot{\kappa}_t p_{\text{init}}(x) \\
&\overset{(iii)}{=} \frac{\dot{\kappa}_t}{1-\kappa_t}\big(\delta_z(x) - [(1-\kappa_t)p_{\text{init}}(x) + \kappa_t\delta_z(x)]\big) \\
&\overset{(iv)}{=} \frac{\dot{\kappa}_t}{1-\kappa_t}(\delta_z(x) - p_t(x \mid z)) \\
&\overset{(v)}{=} \frac{\dot{\kappa}_t}{1-\kappa_t}\delta_z(x)(1 - p_t(x \mid z)) + \frac{\dot{\kappa}_t}{1-\kappa_t}(\delta_z(x) - 1)p_t(x \mid z) \\
&\overset{(vi)}{=} \sum_{y \neq x} \frac{\dot{\kappa}_t}{1-\kappa_t}\delta_z(x)p_t(y \mid z) + \frac{\dot{\kappa}_t}{1-\kappa_t}(\delta_z(x) - 1)p_t(x \mid z) \\
&\overset{(vii)}{=} \sum_{y \neq x} Q_t^z(x \mid y)p_t(y \mid z) + Q_t^z(x \mid x)p_t(x \mid z) \\
&\overset{(viii)}{=} \sum_{y \in \mathcal{S}} Q_t^z(x \mid y)p_t(y \mid z),
\end{aligned}$$

其中 (i) 使用 $d = 1$ 时因子化混合路径的定义；(ii) 通过求导并令 $\frac{d}{dt}\kappa_t = \dot{\kappa}_t$ 得到；(iii) 来自简单代数；(iv) 来自因子化混合路径定义；(v) 是简单代数；(vi) 由 $\sum_{y \in \mathcal{S}} p_t(y \mid z) = 1$ 得到；(vii) 使用速率矩阵定义；$(viii)$ 是简单代数。上述计算表明 KFE 成立，因此结论成立。 □

7.2.3 学习边缘速率矩阵

本节推导训练 CTMC 模型的基本算法。由定理 36, 可以通过学习边缘速率矩阵训练 CTMC 模型 $Q_t^\theta(y \mid x)$。

本节现在限制于因子化混合路径（见例 35），因为这是迄今大多数离散扩散 / 流匹配模型使用的路径。此时边缘速率矩阵具有非常直观的形式：

定理 38 (因子化混合路径的边缘化技巧)**.** *因子化混合路径的边缘速率矩阵是因子化的，并具有形式*

$$Q_t(v_i, j \mid x) = \frac{\dot{\kappa}_t}{1-\kappa_t}\big(p_{1|t}(z_j = v_i \mid x) - \delta_{x_j}(v_i)\big),$$

其中 $p_{1|t}(z_j = v_i \mid x)$ 是给定完整带噪序列 x 时，第 j 个位置（序列中的第 j 个 token）等于 v_i 的条件概率。

证明. 边缘速率矩阵为

$$Q_t(y \mid x) = \sum_{z \in \mathcal{S}} Q_t^z(y \mid x)p_{1|t}(z \mid x). \tag{91}$$

当 y 与 x 不相邻（有超过一个 token 不同）时，对每个 z 均有 $Q_t^z(y \mid x) = 0$，因此此时

$Q_t(y \mid x) = 0$。这说明边缘速率矩阵也因子化。于是有

$$Q_t(v_i, j \mid x) = \sum_{z \in \mathcal{S}} Q_t^z(v_i, j \mid x) p_{1|t}(z \mid x) \tag{92}$$

$$\overset{(i)}{=} \sum_{z \in \mathcal{S}} \frac{\dot{\kappa}_t}{1 - \kappa_t} (\delta_{z_j}(v_i) - \delta_{x_j}(v_i)) p_{1|t}(z \mid x) \tag{93}$$

$$\overset{(ii)}{=} \frac{\dot{\kappa}_t}{1 - \kappa_t} \left(\sum_{z \in \mathcal{S}} \delta_{z_j}(v_i) p_{1|t}(z \mid x) - \delta_{x_j}(v_i) \right) \tag{94}$$

$$\overset{(iii)}{=} \frac{\dot{\kappa}_t}{1 - \kappa_t} \big(p_{1|t}(z_j = v_i \mid x) - \delta_{x_j}(v_i) \big), \tag{95}$$

其中 (i) 由条件速率矩阵公式（见例 37）得出；(ii) 由 $\sum_{z \in \mathcal{S}} p_{1|t}(z \mid x) = 1$ 得出；(iii) 由边缘化得出。证明完成。 □

前一个定理很了不起：边缘速率矩阵实际上是概率 $p_{1|t}(z_j = v_i \mid x)$ 的重新参数化。这实际上无非是在每个 token 位置 $j = 1, \ldots, d$ 学习一个分类器。换言之，可直接将去噪概率网络定义为

$$p_{1|t}^\theta : \underbrace{x}_{\text{网络输入}} \mapsto \underbrace{(p_{1|t}^\theta(z_j = v_i \mid x))_{j=1,\ldots,d,\, v_i \in \mathcal{V}}}_{\text{网络输出}}.$$

注意网络输出形状为 $d \times V$。通过简单的 softmax 层可以获得各 token 位置的概率。网络本身可以是标准的序列到序列网络，例如 Transformer（见第 6.1.2 节）。

由于这只是每个位置 j 的分类器，可通过对每个 $j = 1, \ldots, d$ 使用交叉熵损失训练该网络。由此得到离散流匹配损失：

$$\mathcal{L}_{\text{DFM}}(\theta) = \mathbb{E}_{z \sim p_{\text{data}}, t \sim \text{Unif}_{[0,1]}, x \sim p_t(\cdot|z)} \left[\sum_{j=1}^{d} -\log p_{1|t}^\theta(z_j \mid x) \right].$$

这十分了不起：训练生成模型所需做的全部工作，就是对每个位置 j 训练分类器。正如连续流匹配归结为简单回归（见第三节），离散流匹配与离散扩散模型归结为简单的分类训练。算法 8 总结了训练算法；训练后可以通过算法 7 采样。

例 39 (掩码扩散语言模型)**.** *上述方法的一种特例是掩码扩散语言模型（MDLM）。MDLM 的思想是，我们可以向 token 词表 $\mathcal{V} = \{v_1, \ldots, v_V\}$ 中加入新 token `[mask]`，以表示该 token 缺失（或被掩码）。具体地，令 $\mathcal{V} = \{v_1, \ldots, v_V, \texttt{[mask]}\}$，初始点简单地取为 $\texttt{[mask]}^d$，即全掩码序列。形式上，这相当于在上述框架中设 $p_{\text{init}} = \delta_{\texttt{[mask]}^d}$。采样过程见图 20。*

算法 8：训练因子化 CTMC 模型（离散扩散）。 **输入：** 序列数据集 $z \sim p_{\text{data}}$，其中 $z = (z_1, \ldots, z_d) \in \mathcal{V}^d$；$\mathcal{V}$ 上的初始（噪声）token 边缘分布 $p_{\text{init}}^{(j)}$；调度函数 $\kappa_t \in [0, 1]$；返回每个位置在 $\mathcal{V}$ 上 logits 的后验网络 f_θ；优化器 Opt。

1. 对每次训练迭代，采样数据点 $z \sim p_{\text{data}}$。
2. 采样时间 $t \sim \text{Unif}[0, 1]$ 并计算 $\kappa \leftarrow \kappa_t$。

3. 从 $p_t(\cdot \mid z)$（因子化混合路径）采样带噪状态 x：对 $j = 1, \ldots, d$ 并行采样掩码 $m_j \sim \text{Bernoulli}(\kappa)$ 和噪声 token $\xi_j \sim p_{\text{init}}^{(j)}$，令 $x_j \leftarrow m_j z_j + (1 - m_j)\xi_j$，再令 $x \leftarrow (x_1, \ldots, x_d)$。

4. 用网络输出的 logits 预测终点 token 的后验：$\ell_j(\cdot) \leftarrow f_\theta(x, t)_j$，从而 $p^\theta_{1|t}(v \mid x)_j = \text{Softmax}(\ell_j)(v)$。

5. 计算离散流匹配损失（z 的逐 token 负对数似然）：$\mathcal{L}_{\text{DFM}}(\theta) \leftarrow \sum_{j=1}^{d} [-\log p^\theta_{1|t}(z_j \mid x)_j]$。

6. 更新参数 $\theta \leftarrow \text{Opt.step}(\nabla_\theta \mathcal{L}_{\text{DFM}}(\theta))$，对各次训练迭代重复。

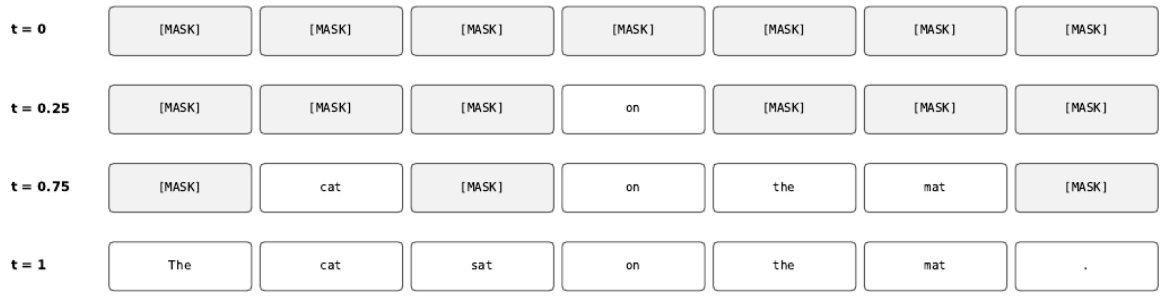


图 20: 掩码扩散语言模型轨迹示意。[MASK] 表示掩码；随时间增加，句子逐渐显现，最终得到 “The cat sat on the mat.”（猫坐在垫子上）。

至此，便完成了训练和采样 CTMC 模型的完整流程，使我们能够生成文本等离散序列。当前先进离散扩散模型 [4] 使用本文描述的方法，以网页规模的数据训练神经网络（通常是 Transformer）。

注 40 (生成元匹配)**.** 你可能会问，为什么流 / 扩散模型的原理可以如此无缝地转移到离散状态空间？事实证明，流匹配原理并非流或 *CTMC* 所独有，而是使用 *Markov* 过程构造生成模型的一般学习原理。由此得到生成元匹配框架 *[19]*，该框架扩展并统一离散和连续流与扩散模型。生成元是向量场 u_t 和速率矩阵 Q_t 的推广。对于任何数据模态和状态空间，都可以构造 *Markov* 过程及生成元。例如，可以为光滑流形（如几何数据）*[8, 10]*、混合状态空间（如联合文本和图像生成）*[6]* 及跳跃过程等其他 *Markov* 过程 *[19, 7]* 构造模型。

8 参考文献

[1] Michael S Albergo, Nicholas M Boffi, and Eric Vanden-Eijnden. “Stochastic interpolants: A unifying framework for flows and diffusions”. In: arXiv preprint arXiv:2303.08797 (2023).

[2] Brian DO Anderson. “Reverse-time diffusion equation models”. In: *Stochastic Processes and their Applications* 12.3 (1982), pp. 313–326.

[3] Yogesh Balaji et al. *eDiff-I: Text-to-Image Diffusion Models with an Ensemble of Expert Denoisers*. 2023. arXiv:2211.01324 [cs.CV]. url: https://arxiv.org/abs/2211.01324.

[4] Tiwei Bie et al. “Llada2. 0: Scaling up diffusion language models to 100b”. In: arXiv preprint arXiv:2512.15745 (2025).

[5] Andrew Campbell et al. “A continuous time framework for discrete denoising models”. In: *Advances in Neural Information Processing Systems* 35 (2022), pp. 28266–28279.

[6] Andrew Campbell et al. “Generative flows on discrete state-spaces: Enabling multimodal flows with applications to protein co-design”. In: arXiv preprint arXiv:2402.04997 (2024).

[7] Andrew Campbell et al. “Trans-dimensional generative modeling via jump diffusion models”. In: *Advances in Neural Information Processing Systems* 36 (2023), pp. 42217–42257.

[8] Ricky TQ Chen and Yaron Lipman. “Flow matching on general geometries”. In: arXiv preprint arXiv:2302.03660 (2023).

[9] Earl A Coddington, Norman Levinson, and T Teichmann. *Theory of ordinary differential equations*. 1956.

[10] Valentin De Bortoli et al. “Riemannian score-based generative modelling”. In: *Advances in Neural Information Processing Systems* 35 (2022), pp. 2406–2422.

[11] Prafulla Dhariwal and Alex Nichol. *Diffusion Models Beat GANs on Image Synthesis*. 2021. arXiv:2105.05233 [cs.LG]. url: `https://arxiv.org/abs/2105.05233`.

[12] Alexey Dosovitskiy. “An image is worth 16x16 words: Transformers for image recognition at scale”. In: arXiv preprint arXiv:2010.11929 (2020).

[13] Alexey Dosovitskiy et al. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. 2021. arXiv:2010.11929 [cs.CV]. url: `https://arxiv.org/abs/2010.11929`.

[14] Patrick Esser et al. *Scaling Rectified Flow Transformers for High-Resolution Image Synthesis*. 2024. arXiv:2403.03206 [cs.CV]. url: `https://arxiv.org/abs/2403.03206`.

[15] Lawrence C Evans. *Partial differential equations*. Vol. 19. American Mathematical Society, 2022.

[16] Itai Gat et al. “Discrete flow matching”. In: *Advances in Neural Information Processing Systems* 37 (2024), pp. 133345–133385.

[17] Jonathan Ho, Ajay Jain, and Pieter Abbeel. “Denoising diffusion probabilistic models”. In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 6840–6851.

[18] Jonathan Ho and Tim Salimans. *Classifier-Free Diffusion Guidance*. 2022. arXiv:2207.12598 [cs.LG]. url: `https://arxiv.org/abs/2207.12598`.

[19] Peter Holderrieth et al. “Generator matching: Generative modeling with arbitrary markov processes”. In: arXiv preprint arXiv:2410.20587 (2024).

[20] Peter Holderrieth et al. “GLASS Flows: Transition Sampling for Alignment of Flow and Diffusion Models”. In: arXiv preprint arXiv:2509.25170 (2025).

[21] Arieh Iserles. *A first course in the numerical analysis of differential equations*. Cambridge University Press, 2009.

[22] Alexia Jolicoeur-Martineau et al. "Adversarial score matching and improved sampling for image generation". In: arXiv preprint arXiv:2009.05475 (2020).

[23] Tero Karras et al. "Elucidating the design space of diffusion-based generative models". In: *Advances in Neural Information Processing Systems* 35 (2022), pp. 26565–26577.

[24] Samuel Lavoie et al. *Modeling Caption Diversity in Contrastive Vision-Language Pretraining*. 2024. arXiv:2405.00740 [cs.CV]. url: https://arxiv.org/abs/2405.00740.

[25] Yaron Lipman et al. "Flow matching for generative modeling". In: arXiv preprint arXiv:2210.02747 (2022).

[26] Yaron Lipman et al. "Flow Matching Guide and Code". In: arXiv preprint arXiv:2412.06264 (2024).

[27] Xingchao Liu, Chengyue Gong, and Qiang Liu. "Flow straight and fast: Learning to generate and transfer data with rectified flow". In: arXiv preprint arXiv:2209.03003 (2022).

[28] Nanye Ma et al. "Sit: Exploring flow and diffusion-based generative models with scalable interpolant transformers". In: arXiv preprint arXiv:2401.08740 (2024).

[29] Xuerong Mao. *Stochastic differential equations and applications*. Elsevier, 2007.

[30] William Peebles and Saining Xie. *Scalable Diffusion Models with Transformers*. 2023. arXiv:2212.09748 [cs.CV]. url: https://arxiv.org/abs/2212.09748.

[31] Ethan Perez et al. "Film: Visual reasoning with a general conditioning layer". In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 32. 1. 2018.

[32] Lawrence Perko. *Differential equations and dynamical systems*. Vol. 7. Springer Science & Business Media, 2013.

[33] Adam Polyak et al. *Movie Gen: A Cast of Media Foundation Models*. 2024. arXiv:2410.13720 [cs.CV]. url: https://arxiv.org/abs/2410.13720.

[34] Alec Radford et al. *Learning Transferable Visual Models From Natural Language Supervision*. 2021. arXiv:2103.00020 [cs.CV]. url: https://arxiv.org/abs/2103.00020.

[35] Colin Raffel et al. *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. 2023. arXiv:1910.10683 [cs.LG]. url: https://arxiv.org/abs/1910.10683.

[36] Robin Rombach et al. *High-Resolution Image Synthesis with Latent Diffusion Models*. 2022. arXiv:2112.10752 [cs.CV]. url: https://arxiv.org/abs/2112.10752.

[37] Robin Rombach et al. “High-resolution image synthesis with latent diffusion models”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2022, pp. 10684–10695.

[38] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. “U-net: Convolutional networks for biomedical image segmentation”. In: *Medical image computing and computer-assisted intervention–MICCAI 2015: 18th international conference, Munich, Germany, October 5-9, 2015, proceedings, part III 18*. Springer. 2015, pp. 234–241.

[39] Chitwan Saharia et al. *Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding*. 2022. arXiv:2205.11487 [cs.CV]. url: https://arxiv.org/abs/2205.11487.

[40] Simo Särkkä and Arno Solin. *Applied stochastic differential equations*. Vol. 10. Cambridge University Press, 2019.

[41] Jascha Sohl-Dickstein et al. “Deep unsupervised learning using nonequilibrium thermodynamics”. In: *International conference on machine learning*. PMLR. 2015, pp. 2256–2265.

[42] Yang Song and Stefano Ermon. “Generative modeling by estimating gradients of the data distribution”. In: *Advances in neural information processing systems* 32 (2019).

[43] Yang Song et al. *Score-Based Generative Modeling through Stochastic Differential Equations*. 2021. arXiv:2011.13456 [cs.LG]. url: https://arxiv.org/abs/2011.13456.

[44] Yang Song et al. “Score-Based Generative Modeling through Stochastic Differential Equations”. In: *International Conference on Learning Representations (ICLR)*. 2021.

[45] Yang Song et al. “Score-based generative modeling through stochastic differential equations”. In: arXiv preprint arXiv:2011.13456 (2020).

[46] Matthew Tancik et al. *Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains*. 2020. arXiv:2006.10739 [cs.CV]. url: https://arxiv.org/abs/2006.10739.

[47] Yi Tay et al. *UL2: Unifying Language Learning Paradigms*. 2023. arXiv:2205.05131 [cs.CL]. url: https://arxiv.org/abs/2205.05131.

[48] Arash Vahdat, Karsten Kreis, and Jan Kautz. “Score-based generative modeling in latent space”. In: *Advances in Neural Information Processing Systems* 34 (2021), pp. 11287–11302.

[49] Ashish Vaswani et al. *Attention Is All You Need*. 2023. arXiv:1706.03762 [cs.CL]. url: https://arxiv.org/abs/1706.03762.

[50] Linting Xue et al. *ByT5: Towards a token-free future with pre-trained byte-to-byte models*. 2022. arXiv:2105.13626 [cs.CL]. url: https://arxiv.org/abs/2105.13626.

[51] Jingfeng Yao, Bin Yang, and Xinggang Wang. “Reconstruction vs. generation: Taming optimization dilemma in latent diffusion models”. In: *Proceedings of the Computer Vision and Pattern Recognition Conference*. 2025, pp. 15703–15712.

A 概率论回顾

本附录简要回顾概率论中的基本概念，部分内容取自文献 [26]。

A.1 随机向量

考虑 d 维欧氏空间中的数据 $x = (x^1, \dots, x^d) \in \mathbb{R}^d$，使用标准欧氏内积 $\langle x, y\rangle = \sum_{i=1}^d x^i y^i$ 和范数 $\|x\| = \sqrt{\langle x, x\rangle}$。我们考虑具有连续概率密度函数（PDF）的随机变量 $X \in \mathbb{R}^d$；其密度 $p_X : \mathbb{R}^d \to \mathbb{R}_{\geq 0}$ 是连续函数，事件 A 的概率为

$$\mathbb{P}(X \in A) = \int_A p_X(x)\, dx, \tag{96}$$

其中 $\int p_X(x)\, dx = 1$。按照惯例，对整个空间积分时省略积分区间（$\int \equiv \int_{\mathbb{R}^d}$）。为简化记号，随机变量 X_t 的密度 p_{X_t} 简写为 p_t。我们用 $X \sim p$ 或 $X \sim p(X)$ 表示 X 服从分布 p。生成建模中常用的密度是 d 维各向同性高斯分布：

$$\mathcal{N}(x; \mu, \sigma^2 I) = (2\pi\sigma^2)^{-d/2} \exp\left(-\frac{\|x-\mu\|_2^2}{2\sigma^2}\right), \tag{97}$$

其中 $\mu \in \mathbb{R}^d$ 和 $\sigma \in \mathbb{R}_{>0}$ 分别为均值和标准差。随机变量的期望是在最小二乘意义下最接近 X 的常向量：

$$\mathbb{E}[X] = \operatorname*{arg\,min}_{z \in \mathbb{R}^d} \int \|x - z\|^2 p_X(x)\, dx = \int x p_X(x)\, dx. \tag{98}$$

计算随机变量函数的期望时，一个有用工具是无意识统计学家定律：

$$\mathbb{E}[f(X)] = \int f(x) p_X(x)\, dx. \tag{99}$$

必要时，我们写成 $\mathbb{E}_X f(X)$，以标明取期望的随机变量。

A.2 条件密度与条件期望

给定两个随机变量 $X, Y \in \mathbb{R}^d$，其联合密度 $p_{X,Y}(x, y)$ 的边缘密度为

$$\int p_{X,Y}(x, y)\, dy = p_X(x), \qquad \int p_{X,Y}(x, y)\, dx = p_Y(y). \tag{100}$$

图 21 展示了 $\mathbb{R}$ 中两个随机变量（$d = 1$）的联合密度。条件密度 $p_{X|Y}$ 描述给定 $Y = y$ 时随机变量 X 的密度，其中 $p_Y(y) > 0$：

$$p_{X|Y}(x \mid y) := \frac{p_{X,Y}(x, y)}{p_Y(y)}. \tag{101}$$

条件密度 $p_{Y|X}$ 类似定义。当 $p_X(x) > 0$ 时，Bayes 法则用 $p_{X|Y}$ 表示 $p_{Y|X}$：

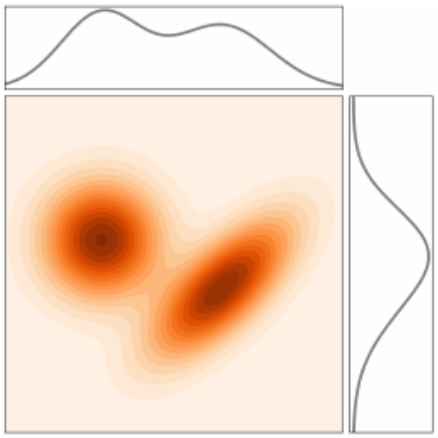


图 21: 联合概率密度 $p_{X,Y}$（阴影）及其边缘密度 p_X 与 p_Y（黑线）。图取自文献 [26]。

$$p_{Y|X}(y \mid x) = \frac{p_{X|Y}(x \mid y)p_Y(y)}{p_X(x)}. \tag{102}$$

条件期望 $\mathbb{E}[X \mid Y]$ 是在最小二乘意义下对 X 的最佳近似函数 $g^\star(Y)$：

$$\begin{aligned} g^\star &:= \operatorname*{arg\,min}_{g:\mathbb{R}^d\to\mathbb{R}^d} \mathbb{E}\big[\|X - g(Y)\|^2\big] \\ &= \operatorname*{arg\,min}_{g:\mathbb{R}^d\to\mathbb{R}^d} \int \|x - g(y)\|^2 p_{X,Y}(x,y)\,dx\,dy \\ &= \operatorname*{arg\,min}_{g:\mathbb{R}^d\to\mathbb{R}^d} \int \left[\int \|x - g(y)\|^2 p_{X|Y}(x \mid y)\,dx\right] p_Y(y)\,dy. \end{aligned} \tag{103}$$

因此，对满足 $p_Y(y) > 0$ 的 $y \in \mathbb{R}^d$，条件期望函数为

$$\mathbb{E}[X \mid Y = y] := g^\star(y) = \int x p_{X|Y}(x \mid y)\,dx, \tag{104}$$

第二个等号如同式 (98)，由对式 (103) 中方括号内的量取最小值得到。将 $g^\star$ 与随机变量 Y 复合，得到

$$\mathbb{E}[X \mid Y] := g^\star(Y), \tag{105}$$

它是 $\mathbb{R}^d$ 中的随机变量。$\mathbb{E}[X \mid Y = y]$ 与 $\mathbb{E}[X \mid Y]$ 常被统称为条件期望，但它们是不同的对象：前者是函数 $\mathbb{R}^d \to \mathbb{R}^d$，后者是取值于 $\mathbb{R}^d$ 的随机变量。为区分两者，本文采用这里引入的记号。塔式性质有助于简化两个随机变量 X, Y 的条件期望计算：

$$\mathbb{E}[\mathbb{E}[X \mid Y]] = \mathbb{E}[X]. \tag{106}$$

由于 $\mathbb{E}[X \mid Y]$ 自身是随机变量 Y 的函数，外层期望是对其再取期望。利用上述定义可以验证：

$$\begin{aligned} \mathbb{E}[\mathbb{E}[X \mid Y]] &= \int \left(\int x p_{X|Y}(x \mid y)\,dx\right) p_Y(y)\,dy \\ &\overset{(101)}{=} \int\int x p_{X,Y}(x,y)\,dx\,dy \overset{(100)}{=} \int x p_X(x)\,dx = \mathbb{E}[X]. \end{aligned}$$

最后，考虑任意随机变量 X, Y 构成的 $f(X,Y)$ 和 Y，由无意识统计学家定律及式 (104) 得到

$$\mathbb{E}[f(X,Y) \mid Y = y] = \int f(x,y) p_{X|Y}(x \mid y)\,dx. \tag{107}$$

B Fokker–Planck 方程的证明

本节给出 Fokker–Planck 方程的自包含证明，其中连续性方程是一个特例（定理 11）。本节并非理解其余内容所必需，数学上也更为深入；希望了解 Fokker–Planck 方程从何而来的读者可以阅读本节。

定理 41 (Fokker–Planck 方程)**.** *设 p_t 是满足 $p_0 = p_{\text{init}}$ 的概率路径，考虑 SDE*

$$X_0 \sim p_{\text{init}}, \qquad\qquad dX_t = u_t(X_t)\,dt + \sigma_t\,dW_t.$$

则对所有 $0 \le t \le 1$，X_t 服从分布 p_t，当且仅当 Fokker–Planck 方程成立：

$$\partial_t p_t(x) = -\operatorname{div}(p_t u_t)(x) + \frac{\sigma_t^2}{2}\Delta p_t(x) \quad \text{对所有} x \in \mathbb{R}^d,\ 0 \le t \le 1. \tag{108}$$

首先证明 Fokker–Planck 方程的必要性：若 $X_t \sim p_t$，则该方程成立。证明的技巧是使用测试函数 f，即无限可微（光滑）且仅在有界区域内非零（具有紧支撑）的函数 $f : \mathbb{R}^d \to \mathbb{R}$。对于任意可积函数 $g_1, g_2 : \mathbb{R}^d \to \mathbb{R}$，使用如下事实：

$$g_1(x) = g_2(x) \quad \text{对所有} x \in \mathbb{R}^d \quad \iff \quad \int f(x) g_1(x)\,dx = \int f(x) g_2(x)\,dx \quad \text{对所有测试函数} f. \tag{109}$$

换言之，可以将逐点相等表示为积分相等。测试函数的有用之处在于它们光滑，因而可以求梯度及更高阶导数。特别地，利用分部积分，对任意测试函数 f_1, f_2 有

$$\int f_1(x) \frac{\partial}{\partial x_i} f_2(x)\,dx = -\int f_2(x) \frac{\partial}{\partial x_i} f_1(x)\,dx, \tag{110}$$

条件是 f_1, f_2 及其乘积 $f_1 f_2$ 可积。结合散度和 Laplace 算子的定义（见式 (22)），得到

$$\int \nabla f_1^T(x) f_2(x)\,dx = -\int f_1(x) \operatorname{div}(f_2)(x)\,dx \qquad (f_1 : \mathbb{R}^d \to \mathbb{R},\ f_2 : \mathbb{R}^d \to \mathbb{R}^d), \tag{111}$$

$$\int f_1(x) \Delta f_2(x)\,dx = \int f_2(x) \Delta f_1(x)\,dx \qquad (f_1 : \mathbb{R}^d \to \mathbb{R},\ f_2 : \mathbb{R}^d \to \mathbb{R}). \tag{112}$$

现在开始证明。使用 SDE 轨迹的随机更新（式 (6)）：

$$X_{t+h} = X_t + h u_t(X_t) + \sigma_t(W_{t+h} - W_t) + h R_t(h) \tag{113}$$

$$\approx X_t + h u_t(X_t) + \sigma_t(W_{t+h} - W_t). \tag{114}$$

为便于阅读，暂时忽略误差项 $R_t(h)$，因为最终将取 $h \to 0$。于是计算如下：

$$\begin{aligned}
f(X_{t+h}) - f(X_t) &\overset{(114)}{=} f(X_t + h u_t(X_t) + \sigma_t(W_{t+h} - W_t)) - f(X_t) \\
&\overset{(i)}{=} \nabla f(X_t)^T \big(h u_t(X_t) + \sigma_t(W_{t+h} - W_t)\big) \\
&\quad + \frac{1}{2}\big(h u_t(X_t) + \sigma_t(W_{t+h} - W_t)\big)^T \nabla^2 f(X_t) \big(h u_t(X_t) + \sigma_t(W_{t+h} - W_t)\big) \\
&\overset{(ii)}{=} h \nabla f(X_t)^T u_t(X_t) + \sigma_t \nabla f(X_t)^T (W_{t+h} - W_t) \\
&\quad + \frac{1}{2} h^2 u_t(X_t)^T \nabla^2 f(X_t) u_t(X_t) + h \sigma_t u_t(X_t)^T \nabla^2 f(X_t)(W_{t+h} - W_t) \\
&\quad + \frac{1}{2} \sigma_t^2 (W_{t+h} - W_t)^T \nabla^2 f(X_t)(W_{t+h} - W_t).
\end{aligned}$$

其中 (i) 在 X_t 附近使用 f 的二阶 Taylor 近似；(ii) 使用 Hessian 矩阵 $\nabla^2 f$ 的对称性。注意 $\mathbb{E}[W_{t+h} - W_t \mid X_t] = 0$，且 $W_{t+h} - W_t \mid X_t \sim \mathcal{N}(0, hI_d)$。因此

$$\begin{aligned}
&\mathbb{E}[f(X_{t+h}) - f(X_t) \mid X_t] \\
&= h\nabla f(X_t)^T u_t(X_t) + \frac{1}{2}h^2 u_t(X_t)^T \nabla^2 f(X_t) u_t(X_t) + \frac{h}{2}\sigma_t^2 \mathbb{E}_{\epsilon_t \sim \mathcal{N}(0, I_d)}[\epsilon_t^T \nabla^2 f(X_t)\epsilon_t] \\
&\overset{(i)}{=} h\nabla f(X_t)^T u_t(X_t) + \frac{1}{2}h^2 u_t(X_t)^T \nabla^2 f(X_t) u_t(X_t) + \frac{h}{2}\sigma_t^2 \operatorname{trace}(\nabla^2 f(X_t)) \\
&\overset{(ii)}{=} h\nabla f(X_t)^T u_t(X_t) + \frac{1}{2}h^2 u_t(X_t)^T \nabla^2 f(X_t) u_t(X_t) + \frac{h}{2}\sigma_t^2 \Delta f(X_t),
\end{aligned}$$

其中 (i) 使用 $\mathbb{E}_{\epsilon_t \sim \mathcal{N}(0, I_d)}[\epsilon_t^T A \epsilon_t] = \operatorname{trace}(A)$；$(ii)$ 使用 Laplace 算子与 Hessian 的定义。由此得到

$$\begin{aligned}
\partial_t \mathbb{E}[f(X_t)] &= \lim_{h \to 0} \frac{1}{h}\mathbb{E}[f(X_{t+h}) - f(X_t)] \\
&= \lim_{h \to 0} \frac{1}{h}\mathbb{E}[\mathbb{E}[f(X_{t+h}) - f(X_t) \mid X_t]] \\
&= \mathbb{E}\left[\lim_{h \to 0} \frac{1}{h}\left(h\nabla f(X_t)^T u_t(X_t) + \frac{1}{2}h^2 u_t(X_t)^T \nabla^2 f(X_t) u_t(X_t) + \frac{h}{2}\sigma_t^2 \Delta f(X_t)\right)\right] \\
&= \mathbb{E}\left[\nabla f(X_t)^T u_t(X_t) + \frac{1}{2}\sigma_t^2 \Delta f(X_t)\right] \\
&\overset{(i)}{=} \int \nabla f(x)^T u_t(x) p_t(x)\, dx + \int \frac{1}{2}\sigma_t^2 \Delta f(x) p_t(x)\, dx \\
&\overset{(ii)}{=} -\int f(x) \operatorname{div}(u_t p_t)(x)\, dx + \int \frac{1}{2}\sigma_t^2 f(x) \Delta p_t(x)\, dx \\
&= \int f(x)\left(-\operatorname{div}(u_t p_t)(x) + \frac{1}{2}\sigma_t^2 \Delta p_t(x)\right) dx.
\end{aligned}$$

其中 (i) 使用 p_t 是 X_t 分布的假设；(ii) 使用式 (111) 与式 (112)。注意，使用这些式子要求乘积 $p_t(x)u_t(x)$ 可积，即

$$\int p_t(x)\|u_t(x)\|\, dx < \infty.$$

在机器学习中这一条件几乎总是成立（由于数值精度限制，数据和函数有界）。因此对所有测试函数 f、$x \in \mathbb{R}^d$ 和 $0 \le t \le 1$，有

$$\partial_t \mathbb{E}[f(X_t)] = \int f(x)\left(-\operatorname{div}(p_t u_t)(x) + \frac{\sigma_t^2}{2}\Delta p_t(x)\right) dx \tag{115}$$

$$\overset{(i)}{\iff} \quad \partial_t \int f(x) p_t(x)\, dx = \int f(x)\left(-\operatorname{div}(p_t u_t)(x) + \frac{\sigma_t^2}{2}\Delta p_t(x)\right) dx \tag{116}$$

$$\overset{(ii)}{\iff} \quad \int f(x)\partial_t p_t(x)\, dx = \int f(x)\left(-\operatorname{div}(p_t u_t)(x) + \frac{\sigma_t^2}{2}\Delta p_t(x)\right) dx \tag{117}$$

$$\overset{(iii)}{\iff} \quad \partial_t p_t(x) = -\operatorname{div}(p_t u_t)(x) + \frac{\sigma_t^2}{2}\Delta p_t(x). \tag{118}$$

其中 (i) 使用 $X_t \sim p_t$；(ii) 交换导数与积分；(iii) 使用式 (109)。这就完成了必要性的证明。

最后解释充分性。Fokker–Planck 方程是一类偏微分方程，更具体地说是所谓的抛物型偏微分方程。类似于定理 3，这类方程在给定初始条件时具有唯一解（例如见文献 [15, 第 7 章]）。若 p_t 满足式 (108)，我们刚刚证明 X_t 的真实分布 q_t（即 $X_t \sim q_t$）也必须满足它；换言之，p_t 和 q_t 都是该抛物型偏微分方程的解。此外，由插值概率路径的构造可知初始条件相同，即 $p_0 = q_0 = p_{\text{init}}$。因此由解的唯一性，对所有 $0 \le t \le 1$ 有 $p_t = q_t$，即 $X_t \sim q_t = p_t$，这正是需要证明的结论。

C 连续时间 Markov 链的存在性与唯一性

本节证明定理 33。

证明. **唯一性**。需要证明满足式 (87) 的转移核 $p_{t'|t}(X_{t'} = y \mid X_t = x)$ 至多只有一个。首先注意，式 (87) 蕴含

$$\frac{d}{dt'} p_{t'|t}(X_{t'} = y \mid X_t = x) \tag{119}$$

$$= \frac{d}{dh} p_{t'+h|t}(X_{t'+h} = y \mid X_t = x)\Big|_{h=0} \tag{120}$$

$$= \frac{d}{dh}\left[\sum_{z\in\mathcal{S}} p_{t'+h|t'}(X_{t'+h} = y \mid X_{t'} = z) p_{t'|t}(X_{t'} = z \mid X_t = x)\right]\Bigg|_{h=0} \tag{121}$$

$$= \sum_{z\in\mathcal{S}} Q_{t'}(y \mid z) p_{t'|t}(X_{t'} = z \mid X_t = x). \tag{122}$$

固定 x, t，可将 $t' \mapsto p_{t'|t}(X_{t'} = y \mid X_t = x)$ 视作向量值函数；上式是该函数的线性 ODE（实际上是 Kolmogorov 前向方程，见命题 2），初始条件已知：$p_{t|t}(X_t = y \mid X_t = x) = \delta_y(x)$。由于每个线性 ODE 都有唯一解（见定理 3），$p_{t'|t}(X_{t'} = y \mid X_t = x)$ 也必然唯一。

存在性。反过来，任一线性 ODE 都存在解，即对每个 x, t 都存在 $p_{t'|t}(X_{t'} = y \mid X_t = x)$，使得

$$p_{t|t}(X_t = y \mid X_t = x) = \delta_y(x), \tag{123}$$

$$\frac{d}{dt'} p_{t'|t}(X_{t'} = y \mid X_t = x) = \sum_{z\in\mathcal{S}} Q_{t'}(y \mid z) p_{t'|t}(X_{t'} = z \mid X_t = x). \tag{124}$$

当 $t' = t$ 时，这特别蕴含式 (87)。剩下要证明此时 $p_{t'|t}(X_{t'} = y \mid X_t = x)$ 是有效的转移核，即满足以下三个性质：

$$\sum_{y\in\mathcal{S}} p_{t'|t}(X_{t'} = y \mid X_t = x) = 1, \tag{125}$$

$$p_{t'|t}(X_{t'} = y \mid X_t = x) \geq 0, \tag{126}$$

$$\sum_{z\in\mathcal{S}} p_{t_2|t_1}(X_{t_2} = y \mid X_{t_1} = z) p_{t_1|t_0}(X_{t_1} = z \mid X_{t_0} = x) = p_{t_2|t_0}(y \mid x). \tag{127}$$

对于第一个性质，初始条件表明它在 $t' = t$ 时成立，且

$$\frac{d}{dt'} \sum_{y\in\mathcal{S}} p_{t'|t}(X_{t'} = y \mid X_t = x) \tag{128}$$

$$= \sum_{y\in\mathcal{S}} \frac{d}{dt'} p_{t'|t}(X_{t'} = y \mid X_t = x) \tag{129}$$

$$= \sum_{z\in\mathcal{S}} \left[\sum_{y\in\mathcal{S}} Q_{t'}(y \mid z)\right] p_{t'|t}(X_{t'} = z \mid X_t = x) \tag{130}$$

$$= 0, \tag{131}$$

这里使用了速率矩阵各列之和为零。对于第二个性质，注意它在 $t' = t$ 时成立。此外，当 $p_{t'|t}(X_{t'} = y \mid X_t = x) = 0$ 时，必有

$$\frac{d}{dt'} p_{t'|t}(X_{t'} = y \mid X_t = x) = \sum_{z \neq y} \underbrace{Q_{t'}(y \mid z)}_{\geq 0} p_{t'|t}(X_{t'} = z \mid X_t = x) \geq 0.$$

因此，只要 $p_{t'|t}(X_{t'} = y \mid X_t = x) = 0$，它只能增加，所以永远不会为负。

为了证明第三个性质，定义

$$q_{t_2|t_0}(y \mid x) = \sum_{z \in \mathcal{S}} p_{t_2|t_1}(X_{t_2} = y \mid X_{t_1} = z) p_{t_1|t_0}(X_{t_1} = z \mid X_{t_0} = x).$$

于是

$$q_{t_2 = t_1|t_0}(y \mid x) = \sum_{z \in \mathcal{S}} \delta_y(z) p_{t_1|t_0}(X_{t_1} = z \mid X_{t_0} = x) = p_{t_1|t_0}(X_{t_1} = y \mid X_{t_0} = x),$$

并且

$$\begin{aligned}
\frac{d}{dt_2} q_{t_2|t_0}(y \mid x) &= \sum_{z \in \mathcal{S}} \frac{d}{dt_2} p_{t_2|t_1}(X_{t_2} = y \mid X_{t_1} = z) p_{t_1|t_0}(X_{t_1} = z \mid X_{t_0} = x) \\
&= \sum_{z \in \mathcal{S}} \sum_{\widetilde{z} \in \mathcal{S}} Q_{t_2}(y \mid \widetilde{z}) p_{t_2|t_1}(X_{t_2} = \widetilde{z} \mid X_{t_1} = z) p_{t_1|t_0}(X_{t_1} = z \mid X_{t_0} = x) \\
&= \sum_{\widetilde{z} \in \mathcal{S}} Q_{t_2}(y \mid \widetilde{z}) \left[\sum_{z \in \mathcal{S}} p_{t_2|t_1}(X_{t_2} = \widetilde{z} \mid X_{t_1} = z) p_{t_1|t_0}(X_{t_1} = z \mid X_{t_0} = x) \right] \\
&= \sum_{\widetilde{z} \in \mathcal{S}} Q_{t_2}(y \mid \widetilde{z}) q_{t_2|t_0}(\widetilde{z} \mid x).
\end{aligned}$$

这表明 $p_{t_2|t_0}(z \mid x)$ 和 $q_{t_2|t_0}(z \mid x)$ 满足同一 ODE，因此必有

$$\sum_{z \in \mathcal{S}} p_{t_2|t_1}(X_{t_2} = y \mid X_{t_1} = z) p_{t_1|t_0}(X_{t_1} = z \mid X_{t_0} = x) = q_{t_2|t_0}(y \mid x) = p_{t_2|t_0}(y \mid x).$$

这证明了第三个性质。因此 $p_{t'|t}(y \mid x)$ 确为满足式 (87) 的转移核，证明完成。 □

D 关于 VAE 的其他视角

本节扩展正文对 VAE 的讨论，给出式 (83) 中 VAE 总损失的变分推导。首先注意，编码器和解码器都给出 x 和潜变量 z 的联合分布：

$$\begin{aligned}
q_\phi(x, z) &= p_{\text{data}}(x) q_\phi(\cdot \mid x) && \text{（编码器联合分布），} \\
p_\theta(x, z) &= p_\theta(x \mid z) p_{\text{prior}}(z) && \text{（解码器联合分布）．}
\end{aligned}$$

因此，可以把训练 VAE 理解为学习 ϕ 和 θ，使编码器和解码器的联合分布足够相似。我们可以借助潜变量与数据联合分布的 KL 散度来实现：

$$\begin{aligned}
D_{\text{KL}}(q_\phi(x, z) \| p_\theta(x, z)) &= D_{\text{KL}}(p_{\text{data}}(x) q_\phi(z \mid x) \| p_\theta(x \mid z) p_{\text{prior}}(z)) \\
&= \mathbb{E}_{\blacksquare} \left[\log \frac{p_{\text{data}}(x) q_\phi(z \mid x)}{p_\theta(x \mid z) p_{\text{prior}}(z)} \right] \\
&= \mathbb{E}_{\blacksquare}[\log p_{\text{data}}(x)] + \mathbb{E}_{\blacksquare} \left[\log \frac{q_\phi(z \mid x)}{p_{\text{prior}}(z)} \right] - \mathbb{E}_{\blacksquare}[\log p_\theta(x \mid z)], \qquad (132)
\end{aligned}$$

其中 ■ 表示 $x \sim p_{\text{data}}(x)$, $z \sim q_\phi(z \mid x)$。依次考察剩余的三项。首先，

$$\mathbb{E}_{\blacksquare}[\log p_{\text{data}}(x)] = \mathbb{E}_{x\sim p_{\text{data}}(x)}[\log p_{\text{data}}(x)] = C, \tag{133}$$

其中常数 C 与 ϕ 和 θ 无关。其次，

$$\mathbb{E}_{\blacksquare}\left[\log \frac{q_\phi(z \mid x)}{p_{\text{prior}}(z)}\right] = \mathbb{E}_{x\sim p_{\text{data}}(x)}[D_{\text{KL}}(q_\phi(z \mid x)\|p_{\text{prior}}(z))] \tag{134}$$

促使 $q_\phi(z \mid x)$ 接近先验 $p_{\text{prior}}(z)$。最后，

$$-\mathbb{E}_{x\sim p_{\text{data}}(x),\, z\sim q_\phi(z|x)}[\log p_\theta(x \mid z)] \tag{135}$$

对应平均负对数似然，因而用于最小化重构损失。忽略常数项后，将先验惩罚与重构项合并，得到 VAE 损失实际上就是数据与潜变量联合空间中的 KL 散度：

$$\mathcal{L}_{\text{VAE}}(\phi,\theta) = \underbrace{\mathbb{E}_{x\sim p_{\text{data}}(x)}[D_{\text{KL}}(q_\phi(z \mid x)\|p_{\text{prior}}(z))]}_{\text{先验约束损失}} - \underbrace{\mathbb{E}_{x\sim p_{\text{data}}(x),\, z\sim q_\phi(z|x)}[\log p_\theta(x \mid z)]}_{\text{重构损失}} \tag{136}$$

$$= D_{\text{KL}}(q_\phi(x,z)\|p_\theta(x,z)) + \text{const.} \tag{137}$$

因此可以将 VAE 理解为潜变量与图像联合空间中的 KL 散度。

VAE 作为生成模型。 现在解释如何将 VAE 理解为生成模型。我们可以令 $z \sim p_{\text{prior}} = \mathcal{N}(0, I_k)$，再从解码器采样 $x \sim p_\theta(\cdot \mid z)$。所得分布为

$$p_\theta(x) = \int_z p_\theta(x \mid z) p_{\text{prior}}(z)\, dz.$$

我们希望说明 VAE 学习近似地从 p_θ 采样。为此需要以下结果：

命题 3 (链式法则)**.** *设 $q(x,z), p(x,z)$ 为两个变量 $x \in \mathbb{R}^{l_1}$、$z \in \mathbb{R}^{l_2}$ 上的分布，则*

$$D_{\text{KL}}(q(z,x)\|p(z,x)) = D_{\text{KL}}(q(x)\|p(x)) + \mathbb{E}_{x\sim q}[D_{\text{KL}}(q(z \mid x)\|p(z \mid x))].$$

特别地，根据式 (76) 第二项非负，于是得到数据处理不等式

$$D_{\text{KL}}(q(x)\|p(x)) \le D_{\text{KL}}(q(z,x)\|p(z,x)). \tag{138}$$

证明.

$$\begin{aligned} D_{\text{KL}}(q(z,x)\|p(z,x)) &= \mathbb{E}_q\left[\log \frac{q(z,x)}{p(z,x)}\right] = \mathbb{E}_{(x,z)\sim q}\left[\log \frac{q(z \mid x)q(x)}{p(z \mid x)p(x)}\right] \\ &= \mathbb{E}_{(x,z)\sim q}\left[\log \frac{q(z \mid x)}{p(z \mid x)}\right] + \mathbb{E}_{x\sim q}\left[\log \frac{q(x)}{p(x)}\right] \\ &= D_{\text{KL}}(q(x)\|p(x)) + \mathbb{E}_{x\sim q}[D_{\text{KL}}(q(z \mid x)\|p(z \mid x))], \end{aligned}$$

其中反复使用了 KL 散度的定义。 □

由命题 3 可以证明

$$\mathcal{L}_{\text{VAE}}(\phi,\theta) = D_{\text{KL}}(q_\phi(x,z)\|p_\theta(x,z)) + \text{const} \ge D_{\text{KL}}(p_{\text{data}}(x)\|p_\theta(x)) + \text{const}, \tag{139}$$

其中使用了 $q_\phi(x,z)$ 关于 x 的边缘分布是 p_{data}。换言之，VAE 损失最小化数据分布 p_{data} 和 VAE 生成的分布之间 KL 散度的一个上界，因此 VAE 本身可看作生成模型。同样可以证明

$$\mathcal{L}_{\text{VAE}}(\phi,\theta) = D_{\text{KL}}(q_\phi(x,z)\|p_\theta(x,z)) + \text{const} \ge D_{\text{KL}}(q_\phi(z)\|p_{\text{prior}}(z)) + \text{const}. \tag{140}$$

即 VAE 目标最小化潜变量分布与先验之间 KL 散度的上界。

为什么不止使用 VAE？ 根据以上讨论，VAE 本身可以作为生成模型；编码器仅用于辅助训练解码器，后者将高斯分布变为目标数据分布。然后可通过 $z \sim p_{\text{prior}}$、$x \sim p_\theta(x \mid z)$ 获取样本。那么为何还要在学得的潜空间中训练单独的生成模型？答案与式 (139) 和式 (140) 两边之间所谓的摊销间隙有关，它恰好对应信息处理不等式中的间隙。当且仅当 $q_\phi(z \mid x) = p_\theta(z \mid x)$ 时，这一间隙为零，此时编码器表示真实后验。[12]因此，例如 $D_{\text{KL}}(q_\phi(x,z)\|p_\theta(x,z))$ 被最小化蕴含 $D_{\text{KL}}(q_\phi(z)\|p_{\text{prior}}(z))$ 被最小化（见式 (140)），但前者的下降不一定意味着后者同样幅度的下降。因此在训练结束时，$D_{\text{KL}}(q_\phi(x,z)\|p_\theta(x,z))$ 和摊销间隙

$$D_{\text{KL}}(q_\phi(x,z)\|p_\theta(x,z)) - D_{\text{KL}}(q_\phi(z)\|p_{\text{prior}}(z)) \tag{141}$$

都没有完全最小化，从而 $q_\phi(z) \neq p_{\text{prior}}(z)$。最后注意，训练时解码器学习从 $q_\phi(z)$ 重构，而非从 $p_{\text{prior}}(z)$ 重构；因此推理时切换到从 $p_{\text{prior}}(z)$ 重构，便偏离训练分布。不过实践中这种不匹配是一种特性，而非缺陷。实践表明，流与扩散模型通常比实现 VAE 解码器的卷积网络更强，因此将部分生成复杂度交给潜空间生成模型是合理的。我们将在后续讨论中回到这一思路。此外，超出本讲义范围的是，扩散和流模型的变分形式本身也可视为 VAE。

证据下界。 适当重排式 (132) 中的各项，可以得到几个互补的视角。其中一个是所谓的证据下界。对固定的 x，有

$$\begin{aligned}\mathbb{E}_{z\sim q_\phi(z|x)}\left[\log \frac{q_\phi(z \mid x)}{p_\theta(x \mid z)p_{\text{prior}}(z)}\right] &= \mathbb{E}_{z\sim q_\phi(z|x)}\left[\log \frac{q_\phi(z \mid x)}{p_\theta(z \mid x)}\right] - \log p_\theta(x) \\ &= D_{\text{KL}}(q_\phi(z \mid x)\|p_\theta(z \mid x)) - \log p_\theta(x),\end{aligned} \tag{142}$$

第一个等号来自

$$p_\theta(z \mid x) = \frac{p_\theta(x \mid z)p_{\text{prior}}(z)}{p_\theta(x)}.$$

因此重排可得

$$\mathbb{E}_{z\sim q_\phi(z|x)}\left[\log \frac{p_\theta(x \mid z)p_{\text{prior}}(z)}{q_\phi(z \mid x)}\right] + D_{\text{KL}}(q_\phi(z \mid x)\|p_\theta(z \mid x)) = \log p_\theta(x), \tag{143}$$

由此得到

$$\underbrace{\mathbb{E}_{z\sim q_\phi(z|x)}\left[\log \frac{p_\theta(x \mid z)p_{\text{prior}}(z)}{q_\phi(z \mid x)}\right]}_{:=\text{ELBO}(x;\phi,\theta)} \leq \underbrace{\log p_\theta(x)}_{\text{证据}}. \tag{144}$$

因此左侧通常称为证据下界（ELBO）。现在可用 ELBO 重写式 (136) 中的 $\mathcal{L}_{\text{VAE}}$：

$$\begin{aligned}\mathcal{L}_{\text{VAE}} &= D_{\text{KL}}(q_\phi(x,z)\|p_\theta(x,z)) + \text{const} \\ &= \mathbb{E}_{x\sim p_{\text{data}}, z\sim q_\phi(z|x)}\left[\log \frac{p_{\text{data}}(x)q_\phi(z \mid x)}{p_\theta(x \mid z)p_{\text{prior}}(z)}\right] + \text{const} \\ &= \mathbb{E}_{x\sim p_{\text{data}}}[\log p_{\text{data}}(x) - \text{ELBO}(x;\phi,\theta)] + \text{const} \\ &= -\mathbb{E}_{x\sim p_{\text{data}}}[\text{ELBO}(x;\phi,\theta)] - \underbrace{H(p_{\text{data}})}_{\text{const}} + \text{const} \\ &= -\mathbb{E}_{x\sim p_{\text{data}}}[\text{ELBO}(x;\phi,\theta)] + \text{const}.\end{aligned} \tag{145}$$

[12]译注：原文把两个边缘界的间隙合并解释。链式法则中，数据边缘界的间隙涉及 $z \mid x$，潜变量边缘界的间隙则涉及 $x \mid z$；两者不能无条件使用同一个零间隙判据。

因此原 VAE 目标可看作试图最大化期望 ELBO。最后考虑完美训练 VAE 的极限。

注 42 ($q_\phi(x,z) \approx p_\theta(x,z)$ 时会怎样？)**.** 首先，训练潜空间生成模型所用的采样分布是边缘分布

$$q_\phi(z) = \int_x q_\phi(z \mid x) p_{\text{data}}(x)\, dx.$$

如果 $q_\phi(x,z) = p_\theta(x,z)$，则特别有 $q_\phi(z) = p_\theta(z) = p_{\text{prior}}(z)$。因此 $q_\phi(x,z) \approx p_\theta(x,z)$ 蕴含潜空间采样分布受到正则化。其次，$q_\phi(x,z) \approx p_\theta(x,z)$ 意味着变分近似 $p_\theta(x \mid z) \approx q_\phi(x \mid z)$ 是好的，进而意味着较低的重构误差。

注 43 (VAE 的变分性体现在哪里？)**.** 为什么不能直接令 $q_\phi(\cdot \mid x) = p_\theta(\cdot \mid x)$，从而保证 $q_\phi(x,z) = p_\theta(x,z) = 0$？原因是尽管我们知道似然 $p_\theta(x \mid z)$，后验

$$p_\theta(z \mid x) = \frac{p_\theta(x \mid z) p_{\text{prior}}(z)}{p_\theta(x)}$$

通常仍难以计算，因为我们无法获得似然 $p_\theta(x)$。因此 *VAE* 中的“变分”一词来自 $q_\phi(\cdot \mid x)$ 充当难以计算的后验 $p_\theta(\cdot \mid x)$ 的替代物，即变分近似。

重构与生成。 给定编码器 $q_\phi(z \mid x)$、解码器 $p_\theta(x \mid z)$ 以及训练为从 $q_\phi(z)$ 采样的潜空间生成模型 r_ψ，可以考虑以下两个生成模型：

$$r^{\text{recon}}_{\psi,\theta}(x_{\text{out}}) = \int_{z,x_{\text{in}}} p_\theta(x_{\text{out}} \mid z) q_\phi(z \mid x_{\text{data}}) p_{\text{data}}(x_{\text{data}})\, dz\, dx_{\text{in}} \qquad \text{（重构采样器），}$$

$$r^{\text{gen}}_{\psi,\phi}(x_{\text{out}}) = \int_{z_{\text{gen}}} p_\theta(x_{\text{out}} \mid z_{\text{gen}}) r_\psi(z_{\text{gen}})\, dz_{\text{gen}} \qquad \text{（生成采样器）.}$$

换言之，重构采样器从 $x_{\text{data}} \in p_{\text{data}}$ 出发，编码为 z 后解码得到 x_{out}；生成采样器则从生成模型的 $z_{\text{gen}} \in r_\psi$ 出发，再经过解码器。计算两个采样器各自的分布与 p_{data} 之间的 Fréchet inception distance，得到重构 FID（rFID）和生成 FID（gFID）。也可以通过平均失真（重构的均方根误差）衡量重构采样器的质量，但这一指标对生成采样器没有意义。重构采样器质量与生成采样器质量之间存在一种自然的张力。较低的 rFID（高质量重构采样器）通常表明潜变量的信息损失较低，使潜变量分布 $q_\phi(z)$ 很大程度上类似 p_{data}，因此学习潜空间生成模型的任务可能更难，导致 gFID 上升。相反，较高的 rFID 通常表明较高的信息损失和更容易学习的潜变量分布 $q_\phi(z)$，从而降低 gFID。图 22 展示了这一现象。

分工。 重构采样器与生成采样器之间的权衡迫使我们考虑如何在自编码器与潜空间生成模型 r_ψ 之间分配信息损失。直观地说，r_ψ 通过某个学得的向量场 $u^\psi_t(z_t)$ 将标准高斯输运到 $q_\phi(z) \approx p_{\text{prior}}$，随后解码器 $p_\theta(x \mid z)$ 将 $q_\phi(z)$ 输运到 p_{data}。现在（不精确地）把*码率*定义为潜变量分布 $q_\phi(z)$ 与 $p_{\text{prior}}(z)$ 相匹配的程度，进而表示生成任务交给潜空间生成模型的程度。[13]这种分工可通过绘制码率和失真之间的 Pareto 前沿来展示，如图 22 所示。特别地，码率高时失真低，反之亦然，这为此前关于重构与生成采样器质量的讨论提供了第二种视角。我们以如下见解结束讨论。

直觉 44 (分工)**.** 图 *22* 的关键见解是，在 *Pareto* 前沿的“拐点”处存在最优分工：此时可以获得低码率（高压缩率!），而失真并不高。换言之，这一点对应的压缩程度既降低了训练底层生成模型的难度，又保持了合理的重构质量。

[13]关于码率更技术性的讨论留待下一小节。

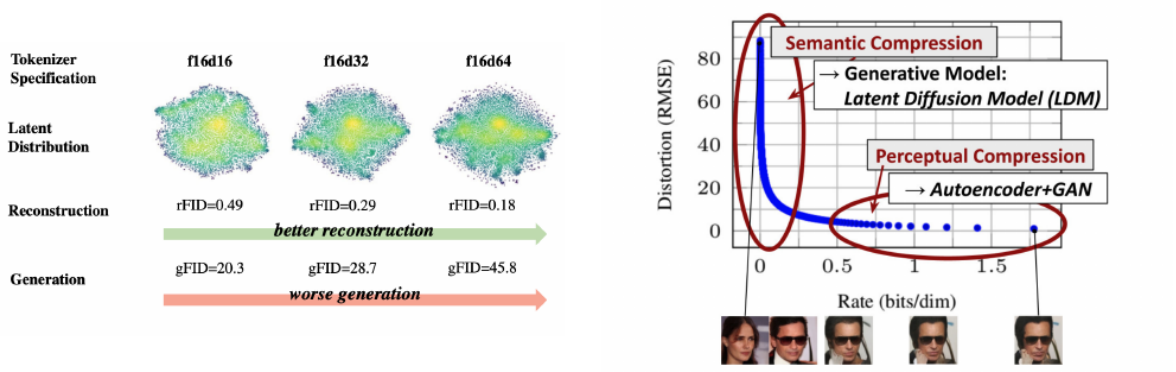


图 22: 右：gFID 与 rFID 的权衡，图取自文献 [51]；f 为下采样因子，d 为潜变量通道维数。右：失真（重构质量）与码率，取自文献 [17, 37]。此曲线由 DDPM（一种 VAE）生成；尽管失真与码率计算的一些技术细节可能不同于本文给出的不精确定义，总体直观相同。译注：原文两处均写“右”；原图左半是 gFID/rFID 对照，右半是码率—失真曲线。左图 Tokenizer Specification / Latent Distribution / Reconstruction / Generation 分别为视觉标记化器配置 / 潜分布 / 重构 / 生成；better reconstruction / worse generation 为更好的重构 / 更差的生成。右图 Distortion (RMSE) / Rate (bits/dim) 为失真（均方根误差）/ 码率（比特/维）；Semantic Compression / Perceptual Compression 为语义 / 感知压缩；Generative Model: Latent Diffusion Model (LDM) 为生成模型：潜扩散模型，Autoencoder+GAN 为自编码器加 GAN。

E 扩散模型文献指南

文献中有一整个扩散模型和流匹配相关的模型家族。阅读这些论文时，很可能会发现它们采用与本课程不同但等价的方式呈现材料，有时因此令人困惑。为此，我们简要概述不同框架及其差异，并介绍历史背景。**这对于理解本文其余内容并非必需**，而是为阅读文献提供帮助。

离散时间与连续时间。 最早的去噪扩散模型论文 [41, 42, 17] 没有使用 SDE，而是在离散时间构造 Markov 链，即时间步 $t = 0, 1, 2, 3, \ldots$。至今仍有许多文献采用这种离散时间形式。这种构造因简单而有吸引力，但其缺点是必须在训练前选择时间离散化。此外，损失函数需要用证据下界（ELBO）近似；顾名思义，它只是我们实际希望最小化的损失的下界。后来 Song 等人 [45] 表明，这些构造本质上是连续时间 SDE 的近似。此外，在连续时间情形中 ELBO 损失变紧（即不再只是下界；例如注意定理 12 和定理 22 是等式而非下界，离散时间情形有所不同）。这使 SDE 构造受到欢迎，因为它被认为在数学上更“干净”，并且可以在训练后通过 ODE/SDE 采样器控制模拟误差。但需要注意，这两类模型使用相同的损失，并无根本差别。

“前向过程”与概率路径。 第一波去噪扩散模型 [41, 42, 17, 45] 没有使用概率路径一词，而是通过所谓前向过程构造数据点 $z \in \mathbb{R}^d$ 的加噪过程。它是如下形式的 SDE：

$$\bar{X}_0 = z, \qquad d\bar{X}_t = u_t^{\text{forw}}(\bar{X}_t)\, dt + \sigma_t^{\text{forw}}\, d\bar{W}_t. \tag{146}$$

其思想是抽取数据点 $z \sim p_{\text{data}}$ 后模拟前向过程，从而破坏数据或给数据“加噪”。前向过程被设计为其分布在 $t \to \infty$ 时收敛到高斯分布 $\mathcal{N}(0, I_d)$。换言之，当 $T \gg 0$ 时近似有 $\bar{X}_T \sim \mathcal{N}(0, I_d)$。

注意，这本质上对应一条概率路径：给定 $\bar{X}_0 = z$ 时 $\bar{X}_t$ 的条件分布是一条条件概率路径 $\bar{p}_t(\cdot \mid z)$，对 $z \sim p_{\text{data}}$ 边缘化后的 $\bar{X}_t$ 分布则对应一条边缘概率路径 $\bar{p}_t$。[14]

不过，在这种构造下，我们需要知道 $X_t \mid X_0 = z$ 的分布的闭式形式，以便训练模型时避免模拟 SDE。这本质上将向量场 u_t^{forw} 限制为使 $\bar{X}_t \mid \bar{X}_0 = z$ 分布具有已知闭式形式的向量场。因此，扩散模型文献中的前向过程向量场总是仿射形式，即对某个连续函数 a_t 有 $u_t^{\text{forw}}(x) = a_t x$。选择此形式后，可使用已知的条件分布公式 [40, 44, 23]：

$$\bar{X}_t \mid \bar{X}_0 = z \sim \mathcal{N}(\alpha_t z, \beta_t^2 I), \qquad \alpha_t = \exp\left(\int_0^t a_r \, dr\right), \qquad \beta_t^2 = \alpha_t^2 \int_0^t \frac{(\sigma_r^{\text{forw}})^2}{\alpha_r^2} \, dr.$$

注意，这只是高斯概率路径。因此可以说前向过程是构造（高斯）概率路径的一种特定方式。流匹配 [25] 引入概率路径这一术语，以同时简化并推广构造。第一，扩散模型的“前向过程”实际上从不被模拟（训练时只从 $\bar{p}_t(\cdot \mid z)$ 采样）；第二，前向过程只在 $t \to \infty$ 时收敛（即永远不会在有限时间到达 p_{init}）。因此本文选择使用概率路径。

时间反演与求解 Fokker–Planck 方程。 扩散模型最初并非通过 Fokker–Planck 方程（或连续性方程）构造训练目标 u_t^{target} 或 $\nabla \log p_t$，而是通过前向过程的时间反演 [2]。时间反演 $(X_t)_{0 \le t \le T}$ 是一种轨迹分布在时间上反向的 SDE，即

$$\mathbb{P}[\bar{X}_{t_1} \in A_1, \dots, \bar{X}_{t_n} \in A_n] = \mathbb{P}[X_{T-t_1} \in A_1, \dots, X_{T-t_n} \in A_n] \tag{147}$$

$$\text{对所有} 0 \le t_1, \dots, t_n \le T, \quad A_1, \dots, A_n \subset \mathcal{S}. \tag{148}$$

Anderson [2] 表明，可由以下 SDE 得到满足上述条件的时间反演：

$$dX_t = [-u_t(X_t) + \sigma_t^2 \nabla \log p_t(X_t)] \, dt + \sigma_t \, dW_t, \qquad u_t(x) = u_{T-t}^{\text{forw}}(x), \qquad \sigma_t = \bar{\sigma}_{T-t}.$$

由于 $u_t(X_t) = a_t X_t$，上式对应于命题 1 中推导的训练目标的一个特例（由于使用了不同的时间约定，这一点并非立即显然；推导可参见文献 [26]）。不过，对于生成建模，我们常常只使用 Markov 过程的终点 X_1（如生成图像），舍弃更早的时间点。因此，许多应用中过程是否为“真正的”时间反演，还是沿概率路径运动，并不重要。因此使用时间反演并非必需，且往往导致次优结果；例如概率流 ODE 通常更好 [23, 28]。不同于时间反演的所有扩散模型采样方式都再次依赖 Fokker–Planck 方程。我们希望这说明了为何如今许多人直接通过 Fokker–Planck 方程构造训练目标——正如文献 [25, 27, 1] 开创的做法及本课程所采用的做法。

流匹配 [25] 与随机插值 [1]。 本文介绍的框架与流匹配和随机插值（SI）框架最为接近。我们已经看到，流匹配局限于流。事实上，流匹配的一项关键创新是表明，无需通过前向过程和 SDE 构造，单独使用流模型即可进行可扩展的训练。由于这一限制，从流匹配模型采样是确定性的（只有初始值 $X_0 \sim p_{\text{init}}$ 是随机的）。随机插值既包含纯流，也包含本文通过“Langevin 动力学”使用的 SDE 扩展（见定理 17）。随机插值得名于用于在两个分布之间插值的插值函数 $I(t, x, z)$。在本文的术语中，它对应于构造条件和边缘概率路径的另一种（大体）等价方式。相对于扩散模型，流匹配与随机插值的优点既包括简单，也包括一般性：其训练框架十分简单，同时允许从任意分布 p_{init} 变换到任意分布 p_{data}，而去噪扩散模型只适用于高斯初始分布及高斯概率路径。这为生成建模开辟了新的可能性，本课程稍后会简要涉及。

[14] 但注意这里采用了反向时间约定：$\bar{p}_0(\cdot \mid z) = p_{\text{data}}$。

总结 45 (扩散模型的其他形式)**.** 文献中常见的扩散模型替代形式通常包含以下一种或多种要素：

1. **离散时间：** 常用离散时间 *Markov* 链近似 *SDE*。
2. **反向时间约定：** 常用 $t = 0$ 对应 p_{data} 的时间约定（与本文的 $t = 0$ 对应 p_{init} 相反）。
3. **前向过程：** 前向过程（或加噪过程）是构造（高斯）概率路径的方法。
4. **通过时间反演构造训练目标：** 也可以通过 *SDE* 的时间反演构造训练目标，它是本文构造的一种特例（采用反向时间约定）。